



Joint Interpretation Library

Attack Methods for Smartcards and Similar Devices

Distribution controlled by member schemes of the JIWG
This document shall not be sent without JIWG authorisation and shall not be published.

This page is intentionally left blank.

Table of content

1	Introduction	6
1.1	Motivation	6
1.2	Meaning of paragraphs (under descriptors of attack methods)	7
1.3	Proposed use of this document	8
2	Attack Methods	9
2.1	Physical Attacks	9
2.1.1	Description of Attack	9
2.1.2	Effect of Attack	9
2.1.3	Impact on TOE	9
2.1.4	Characteristics of the Attack	10
2.1.5	Example: Probing a bus.....	11
2.1.6	Example: Probing Sensitive wires of the glue logic	17
2.1.7	Example: Reading ROM	18
2.2	Overcoming sensors and filters	24
2.2.1	Description of Attack	24
2.2.2	Effect of Attack	24
2.2.3	Impact on TOE	24
2.2.4	Characteristics of the Attack	25
2.2.5	Example: Combination of attacks	25
2.3	Perturbation Attacks.....	26
2.3.1	Description of Attack	26
2.3.2	Effect of Attack	26
2.3.3	Impact on TOE.....	27
2.3.4	Characteristics of the Attack	28
2.3.5	Example: cryptographic failure.....	32
2.3.6	Example: authentication bypass.....	34
2.3.7	Example: Changing output address range and dumping memory.....	39
2.3.8	Example: Accessing forbidden file	43
2.3.9	Example: PIN brute force using PIN UNBLOCK	47
2.3.10	Example: perturbation / cryptographic attack on 3DES.....	49
2.3.11	Example: Register localization and modification	58
2.3.12	Example: Double Perturbation attack on AES engine	59
2.3.13	Example: Double Perturbation attack on AES engine and CPU.....	63
2.4	Retrieving keys with FA	68
2.4.1	Description of Attack	68
2.4.2	Effect of Attack	68
2.4.3	Impact on TOE.....	69
2.4.4	Characteristics of the Attack	69
2.4.5	Example: Perturbation attack combined with brute force.....	70
2.4.6	Example: DFA attack on (3)DES / AES	71
2.4.7	Example: DFA attack on RSA	77
2.4.8	Further References	80
2.5	Side-channel Attacks – Non-invasive retrieving of secret data	81
2.5.1	Description of Attack	81
2.5.2	Effect of Attack	82
2.5.3	Impact on TOE	84
2.5.4	Characteristics of the Attack	85
2.5.5	Note about Profiled Side-Channel Attacks	94

2.5.6	Example: SPA/SEMA to retrieve DES key	96
2.5.7	Example: DPA to retrieve DES key during DES calculation	98
2.5.8	Example: higher-order DPA to retrieve DES-key.....	100
2.5.9	Example: SPA on RSA	101
2.5.10	Example: DPA on RSA (Cross-Correlation Attack).....	103
2.5.11	Example: DPA on RSA-CRT.....	104
2.5.12	Profiled Side-Channel Attack on AES	105
2.5.13	Profiled Side-Channel Attack on RSA.....	107
2.5.14	Example: combined Side-channel and brute force attack on ISO 9797-1 (MAC Algo3)	109
2.5.15	Machine Learning as alternative for Profiled Side Channel Attacks	110
2.5.16	Further References	114
2.6	Exploitation of Test Features	115
2.6.1	Description of Attack	115
2.6.2	Effect of Attack	115
2.6.3	Impact on TOE	115
2.6.4	Characteristics of the Attack	116
2.6.5	Example: Attack against test mode with physical fuse	116
2.6.6	Example: Attack against test mode with lifecycle control	117
2.6.7	Example: Enter test mode protected by access code with error counter	120
2.7	Attacks on RNG	124
2.7.1	Description of Attacks.....	124
2.7.2	Effect of Attacks.....	125
2.7.3	Impact on the TOE	125
2.7.4	Characteristics of the Attack	126
2.7.5	Example: attack on True Random Number Generator.....	127
2.7.6	Example: attack on Pseudo Random Number Generator.....	130
2.8	Software Attacks	132
2.8.1	Introduction	132
2.8.2	Software attack detailed objective.....	133
2.8.3	Information gathering on commands	135
2.8.4	Direct protocol attacks	137
2.8.5	Man-in-the-middle and replay attacks.....	138
2.8.6	Buffer overflow or stack overflow	139
2.8.7	Malicious code execution.....	140
2.8.8	Communication interface switching.....	140
2.8.9	Firewall bypass.....	141
2.9	Application isolation	144
2.9.1	Overview of the section.....	144
2.9.2	References	144
2.9.3	Definitions	145
2.9.4	Introduction	145
2.9.5	Assets	146
2.9.6	Threats	147
2.9.7	List of relevant technologies and standards for application isolation	147
2.9.8	Rating of attacks defeating application isolation	153
2.9.9	Partial attack bricks applicable to the countermeasures.....	153
2.9.10	Full attack path	173
3	Countermeasure list.....	185
3.1	Hardware	185

3.2	Software countermeasures.....	186
4	Glossary.....	189
5	References	191

1 Introduction

1.1 Motivation

- 1 Common Criteria is a very powerful tool to evaluate and rate the security properties of an IT product. But it can only be as good as the underlying security test methods.
- 2 Responsibility for ensuring the quality of these security tests lies with the certification bodies.
- 3 Attack methods for the product range of smart cards cover diverse fields of expertise such as physics, informatics and cryptography. The use of these different types of expertise for attacks is very complex. This makes it very difficult for a single organisation to ensure “state of the art” coverage of the whole range of attacks when relying only on its own resources. Ideally, the experts in security and security testing of a defined product range in IT would come together, pool their knowledge and compose a list of test methods representing the state-of-the-art.
- 4 This document is the result of the cooperation of such a group, comprising representatives of IC-vendors, card vendors, evaluation labs, certification bodies and service providers. It refines the attack methods given in the JIL-paper about application of attack potential to smartcards [JIL-AP]. The refinement serves two main purposes:
 - It provides guidance as to which attack methods have to be considered in a smart card evaluation. By describing the key factors of these methods, a harmonisation of vulnerability assessment and penetration testing in evaluations can be achieved.
 - The document also helps the standardisation of the security rating of smart card products. To this end, the attack methods provide guidelines as well as examples for the attack rating. It should be noted that for consistency the concept of “Exploitation” and “Identification” defined in the [JIL-AP] was maintained. The authors of the document are aware that for risk management purposes information about the resulting combined rating may not be enough. However, this is an issue to be solved in the CC process (perhaps by introducing an “ETR for Risk Management”).
- 5 Important note:
 - The presented guidance is a minimum set of methods that have to be considered. For special applications and products there may exist attacks that are not specially mentioned here.
 - Not all methods will be applicable for every product. It is in the responsibility of the ITSEF and the CB to select the appropriate methods.
 - For each attack one or more examples are provided. These examples differ – among others – by the countermeasures the TOE is supposed to have. These countermeasures are by no means meant to be mandatory for a TOE and only serve to illustrate what sort of complications an attack path may entail and how that affects the rating.

1.2 Meaning of paragraphs (under descriptors of attack methods)

Description of Attack

- 6 Gives a short description of the purpose and method of the attack

Effect of Attack

- 7 Contains a more detailed attack description and how to recognise when this attack has succeeded. This may include variations of a basic attack (e.g. different causes of perturbations).

Impact on TOE

- 8 Examples of how the attack may result in an exploitable vulnerability in the TOE. This may explain how a hardware attack is used in the context of software (or how an OS attack is used in the context of an application). This description is in terms of issues that would need to be notified to a developer of software that uses the TOE.
- 9 This will also explain the motivation for carrying out this attack.

Characteristics of the Attack

- Factors that make the attack difficult or easy to carry out or to apply to a real TOE.
 - Skills and tools required to carry out the attack, in the terminology of the JIL attack potential document [JIL-AP].
 - References to books, papers or standard methods, where appropriate. This list of references will probably not be complete – more techniques are used in labs than are published – but they may give an understanding of the basics of the attack or attack techniques.
- 10 This part gives an agreed set of attacks (or attack variants) which should be considered as “obvious attacks”.

Examples of Attack Potential Ratings

- 11 These examples illustrate in more detail what is behind the different attack methods. The presentation of these ratings helps to come to consistent and commonly agreed ways to interpret the existing JIL attack potential table [JIL-AP].

Required demonstration of attack to justify this rating

- 12 Describes how much of the attack has to be demonstrated to decide whether it is feasible or not. This does not automatically mean the evaluator has to perform the attack; the evaluator may have other considerations to decide that this attack is not relevant to a specific TOE.

1.3 Proposed use of this document

- 13 This document pertains to the product range of smartcards. It refines the CEM for this product range concerning the aspects of class AVA and especially of the component AVA_VAN with CC v3.1. In contrast to the CEM it also covers AVA_VAN.5.
- 14 For the planning of the vulnerability assessment and penetration testing, it provides guidance as to which attack methods should be considered. The examples give an indication of what is regarded as “state of the art”.

Note: “State of the art” is not static and may change over time. The list does also not claim to be complete but describes only a minimum set.

- 15 For each evaluation it has to be decided which of the attack methods are applicable for the product under evaluation and how the attacks should be best implemented. It might be possible to exclude whole classes of attacks just by considering specific properties of the TOE (such as its use or environment).
- 16 The examples in the document also give guidance about how the attack potential was decided upon. Even if the attacks performed for another TOE do not exactly match the examples, evaluators should be able to build an equivalent rating.

Note: Even though the examples consider a selection of commonly used countermeasures it is very important to note that the ratings given reflect only an average security level; but the security level in reality strongly depends on the implementation. There may also be other countermeasures that are not considered here. In consequence, the ratings for any particular evaluation may be different from the examples given.

- 17 Implied in the application of this list is that the lab conducting the evaluation has the expertise and skill to select the appropriate attack methods and is able to perform them adequately or to subcontract special tasks.

2 Attack Methods

2.1 Physical Attacks

2.1.1 Description of Attack

- 18 Microelectronic tools enable either access to or modification of an IC by removing or adding material (etching, FIB, etc). Depending on the tool and on its use, the interesting effect for the attacker is to extract internal signals or manipulate connections inside the IC by adding or cutting wires inside the silicon.
- 19 Memories could also be physically accessed for, depending on the memory technology, reading or setting bit values.

2.1.2 Effect of Attack

- 20 Various types of attack can be performed on an IC depending on whether the intended effect is to remove material, to add material, or both:
- De-processing the IC and removing all the material above the upper metal layer of the whole chip or in dedicated areas. This is a prerequisite for the following attacks, also allowing direct access to internal wires and thereby signals, for example by use of a microprobe. This access can be used either to retrieve information or to force internal signals.
 - “Layering” the IC: metal layers are removed successively. The objective is to get one sample with each metal layer exposed. This preparation is the basis for reverse engineering.
 - Cutting connections: a “wire” in the IC can be cut. Transmission of data or data storage itself could be forced into permanent faults. This is also one approach to disconnecting physically IC security features (e.g. sensors).
 - Adding connections: using a FIB, connections can be added to the IC. This could allow bypassing security features, or changing the functionality of the IC, or connecting internal signals (such as bus wires) to an external pin of the IC.
 - Using microscopes: ROM memory values can be read out.

2.1.3 Impact on TOE

- 21 The attack is directed against the IC and is often independent of the embedded software (i.e., it could be applied to any embedded software and is independent of software countermeasures).
- 22 The main impacts are:
- Access to secret data such as cryptographic keys (by extracting internal signals or reading out memories)

- Disconnecting IC security features to make another attack easier (DPA, perturbation)
- Forcing internal signals
- Even unknown signals could be used to perform or aid some attacks (e.g., as trigger for DPA or perturbation attacks)

23 The potential use of these techniques is very wide and has to be carefully considered in the context of each evaluation.

2.1.4 Characteristics of the Attack

24 Using this attack requires performing of an accurate reverse engineering first, at least to identify the potentially interesting areas (e.g. memories, memory output, ALU, busses, etc). Modern circuits with small feature sizes generated with automatic synthesis tools are very difficult to analyze and “understand” the individual functions thereof. Therefore, this first step is very complex.

25 Moreover, an active shield (anti-probing layer) often protects circuits. A physical attack has first to bypass this protection either by removing it (and modifying the IC to avoid detection), by penetrating through it without detection (extensive use of FIB), or by starting the attack from the backside of the TOE (also extensive use of FIB).

26 In all the cases, the expertise required for physical manipulation is very high and the EXPERT rating of the table has to be used.

27 The equipment can typically be found in a research lab or in an IC manufacturer’s characterization lab. The basic tools (plasma etching, microscope, etc) are SPECIALIZED and the most complex tools are BESPOKE.

28 *Remark:* Some companies offer services around complex equipment such as FIB (renting the equipment and a trained operator to perform some experiments). The rating of such equipment could then be different for a specific TOE and attack. It might depend on the sophistication of the required equipment demanded by the IC and its feature sizes.

29 The equipment typically used for these attacks is:

Observation

30 High-end microscope (optical, SEM): SPECIALIZED – Observation of the IC. There is a minimum feature size to allow optical observation of the IC. It is difficult to give a value, but 0.12 microns looks realistic. In contrast, a low-end visible-light microscope can only be regarded as STANDARD.

Removing materials (“large” attack area):

31 Plasma etching: SPECIALIZED
Chemical liquid etching: STANDARD or SPECIALIZED
Laser cutter: SPECIALIZED

Removing or adding material (small areas):

32 FIB: SPECIALIZED or BESPOKE
Micro-probe station: SPECIALIZED

2.1.5 Example: Probing a bus

Countermeasures of the IC (taken from the countermeasure list):

None

2.1.5.1 Attack path

- 33 The attack is performed on an IC with embedded software. The objective is to record the cryptographic key bits during their transfer on a data bus by probing (positioning of probe needles on the upper metal layer).

Step 1: De-processing

- 34 The circuit is first de-processed to gain access to the upper metal layer using etching techniques (either chemical liquid etching or plasma etching).

Step 2: Reverse engineering

- 35 The upper layer is examined through an optical microscope to get an image of the wiring. This image is used to perform reverse engineering and to detect the bus tracks. Extra reverse engineering and some probing during the execution of specific commands, with an SPA-based analysis, will help the attacker in finding the right order of the wires and the right time for data sampling, corresponding to e.g. a key value transfer.

Step 3: Probing/Acquisition of data

- 36 After setting up a scenario (key loading), the commands are executed and an acquisition of the signal (analogue or digital) of the bus activity is performed. This acquisition can be done in various ways:
- Probing 8 wires (or 9 wires with the clock signal). This can be done by putting 8 (or 9) probe needles in contact with the bus wires. It could be difficult to ensure that this number of needles is correctly placed and to keep contact during the complete data sampling.
 - Probing fewer wires at a time. For example probing with 3 or 4 needles is much easier. The complete data set can be obtained by re-running the execution with the probes on different bus lines until all lines are sampled and then re-assembling the data to reconstruct the 8 bit bus. (Some countermeasures in the software, such as random execution, key loading strategy, etc., could obstruct this attack method.)
 - Creating contacts on each wire with a FIB to allow an easier 8 bit probing, or even creating pads and bonding each wire to an external pin of the IC.

Step 4: Exploitation of data

- 37 An extra phase is often needed to obtain the bit values from the acquired signals. Depending on the complexity of the software countermeasures, this phase could be anywhere from easy (e.g. interpreting binary values from analogue signals) to impossible (reconstructing a consistent value from partial acquisitions of blinded signals).

2.1.5.2 Example rating 1 without countermeasures

38 For this example, we consider a simple, straightforward IC (typically an 8-bit micro-controller) with no countermeasures from the list, in particular:

- No active shield (anti-probing layer)
- No data encryption during bus transfer
- “Classical” IC design (the layout reflects the block structure of the IC and an optical analysis, using a microscope, of the upper layer of the IC can allow a specialist to identify the bus wires).

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Preparation of the IC (de-processing), reverse engineering to locate the buses, data acquisition and interpretation require a significant time, certainly more than 1 week and probably less than 1 month for the simple IC taken into account in this example. For exploitation, preparation of the IC and data acquisition has to be performed. Several days are required.	<1 month (3)	<1 week (4)
Expertise	Reverse engineering is typically at the EXPERT level. Exploiting the attack is typically at PROFICIENT level.	Expert (5)	Proficient (2)
Knowledge of TOE	Knowledge of the commands (APDUs) is required.	Restricted (2)	Public (0)
Open Samples/ Known Key	Not required	NA	NA
Access to TOE	More than one sample may be necessary, certainly one per layer to be reverse engineered, but less than ten.	< 10 samples (0)	< 10 samples (0)
Equipment	For this attack, we consider that the inspection equipment, the probe equipment and plasma etching belong to one testbench. So no points are added. Also, it is assumed that the technology node of the TOE is sufficiently advanced so that it is not possible to use simpler tools instead (0.35μ or better)	Specialized (3)	Specialized (4)
Sub Total		13	10
Total		23	

39 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3) with CC v3.

2.1.5.2.1 Required demonstration of attack to justify this rating

40 The required demonstration is the successful probing of 1 wire and demonstrating that all the other relevant wires can be accessed by probing.

2.1.5.3 Example rating 2 with selected countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Randomised internal execution (to avoid multiple partial acquisitions being combined)

Note: This countermeasure could be effective but is often limited. Invariants exist (either corresponding to interesting critical code sections or corresponding to random execution) that could allow critical code to be identified.

- Wide Bus width (for example 32 bits vs. 8 bits):

41 Probing 32 wires on an IC is certainly very complex and probably not realistic, but, if the signals could be rearranged after multiple acquisitions, the attack is still possible.

42 The “FIB” attack (connecting each bus wire to a contact PAD) is still possible but more complex (adding time and complexity, so the number of samples used in the exploitation phase would probably increase).

43 The rating of such an attack could be:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Randomization and bus width certainly increase the identification time and the exploitation time.	>1 month (5)	<1 week (4)
Expertise	EXPERT is also put in exploitation because of the added complexity (probing directly or by FIB preparation 32 lines)	Expert (5)	Expert (4)
Knowledge of TOE	Knowledge of the commands (APDUs) is required.	Restricted (2)	Public (0)
Access to TOE	Some samples will certainly be killed during the test, justifying to move to more than 10 samples in identification.	< 100 samples (2)	< 10 samples (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	If a FIB preparation for 32-bit buses is required then a BESPOKE rating in exploitation is required (in identification not all the lines have to be acquired).	Specialized (3) or Bespoke (5)	Specialized (4) or Bespoke (6)
Sub Total		17 or 19	12 or 14
Total		29 or 33	

44 The TOE would be resistant for this attack to an attacker with high attack potential (VAN.5) when bespoke equipment is used, otherwise it is moderate only (VAN.4).

2.1.5.3.1 Required demonstration of attack to justify this rating

45 Sufficiently many wires must be probed to justify in terms of countermeasure designs that the remaining wires can be probed and data synchronised.

Discussion:

46 An expert level is required even for exploitation due to the complexity of the IC preparation, and/or because of the complexity of the post-processing in case of partial acquisition.

47 It could be considered to subcontract a simple FIB preparation without too much problem, reducing the rating for equipment, but a 32-wire preparation could be difficult to keep confidential.

2.1.5.4 Example rating 3 with selected countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Active shield or barrier shielding (simple implementation)

48 Even with an active layer it is possible to create undetected “holes” in the shield by avoiding sensors. The extent to which this is possible depends on the technology node of the TOE. For the following, we will again assume a bus 8 bits wide.

49 The reverse engineering phase is more complex because the mechanisms related to the active shield have to be identified and the (even partial) disconnection analysed and defined. It is assumed that no special countermeasures are taken to hide the shield mechanism.

50 The rating could be:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Extra reverse engineering as well as IC preparation. However, once everything has been identified, exploitation is as straight-forward as before	> 1 month (5)	< 1 week (4)
Expertise	Same rating as the initial example	Expert (5)	Proficient (2)
Knowledge of TOE	Knowledge of the commands (APDUs) is required.	Restricted (2)	Public(0)
Access to TOE	Less than 10 samples is typical	<10 samples (0)	<10 samples (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Probably a FIB will be required, but for a limited and very well defined preparation that could easily be subcontracted (because of the simple	Specialized (3)	Specialized (4)

	implementation).		
Sub Total		15	10
Total		25	

51 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

2.1.5.4.1 Required demonstration of attack to justify this rating:

52 Sufficiently many wires must be probed to justify in terms of countermeasure designs that the remaining wires can be probed. This must include justifying that the shield can be breached in the necessary number of places, leaving the device still operational. The evaluator records the number of samples destroyed and the elapsed time during the first breach and uses this to calculate the number of samples and the elapsed time required for the complete attack taking into account the success rate.

2.1.5.5 Example rating 4 with selected countermeasures:

53 Countermeasures of the IC (taken from the countermeasure list):

- Randomized location of components
- State of the art design methods use an automatic synthesis of the IC. The effect is that there is no simple block structure in the layout and reverse engineering (e.g. finding the bus wires and finding their right order) could become very complex and even not practical.

54 Rather than spending a lot of time searching for them, there is still the alternative of using VERY CRITICAL knowledge (design documentation including layout) to find the interesting wires.

Note: In this example only the randomized location is used. It is considered that the IC has no active shield or barrier.

55 The corresponding rating could be:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Because of an extensive use of the knowledge of the TOE there is no need for a significant reverse engineering phase.	< 1 week (2)	<1 week (4)
Expertise	Understanding the design at the implementation level and deriving an attack path requires the EXPERT level in identification.	Expert (5)	Proficient (2)
Knowledge of TOE	This attack makes full use of the knowledge of the TOE at the implementation level (schematics). Once scripted, no more knowledge is required for the exploitation phase.	Very critical design (9)	Public (0)
Access to TOE	Less than 10 samples are typically required.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Using the design information	Bespoke	Specialized

	requires some more expensive and complex tools than normally are required for reverse engineering. These tools could be specific to an IC developer or even the IC family itself. Coupled with the physical preparation of the samples, this justifies to moving the equipment level to BESPOKE. Once identified, the exploitation phase could be subcontracted reasonably easily, justifying a SPECIALIZED level.	(5)	(4)
Sub Total		21	10
Total		31	

56 This TOE would be resistant for this attack to an attacker with high attack potential (VAN.5).

2.1.5.5.1 Required demonstration of attack to justify this rating

57 The required demonstration is the successful probing of 1 wire and verification that all the other relevant wires can be identified and accessed by probing.

Discussion:

58 Using the design information requires some expensive and complex tools. These tools could be specific to an IC developer or even the IC family itself. Coupled with the physical preparation of the samples, this justifies moving the equipment level to BESPOKE. Once identified, the exploitation phase could be subcontracted reasonably easily, justifying a SPECIALIZED level.

2.1.5.6 Example rating 5 with selected countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- On-chip encryption of internally handled data.
This countermeasure is certainly the most effective if implemented correctly. As a minimum, the implementation has to be checked for that:
 - The detailed knowledge of the TOE does not give the keys used for encryption (for example randomly generated key inside the IC).
 - There is no unencrypted data transfer between different parts of the IC in the upper layers.
 - No easy coupled acquisition of plain text is possible – ciphered messages used only.
 - The encryption scheme used is not only based on address encryption alone, as this would allow for attacks based on some suitable assumptions that the corresponding plaintexts are related to each other.
 - The key is TOE-specific.

59 In such a case, the attack will be NOT PRACTICAL because the acquisition of the signals does not give the clear data (e.g. key values).

2.1.6 Example: Probing Sensitive wires of the glue logic

Countermeasures of the IC (taken from the countermeasure list):

- Randomized location of components
- State of the art design methods use an automatic synthesis of the IC. The effect is that there is no simple block structure in the layout and reverse engineering (e.g. finding the bus wires and finding their right order) could become very complex and even not practical.

2.1.6.1 Attack path

60 The attack is performed on an IC with embedded software in case of a composite evaluation. In case of hardware evaluation the attack is performed with a (rudimentary) operating system and/or other related test software and/or with optional cryptographic libraries, provided from the hardware vendor. The objective is to record the cryptographic key bits during their transfer/use on specific wires of the glue logic by probing (positioning of probe needles on the upper metal layer). Assumptions: no active shield over the glue logic.

61 Note that this attack is targeting the cryptographic keys, but it could aim to retrieve any sensitive data possibly handled by wires at the surface of the glue logic.

Step 1: De-processing

62 The circuit is first de-processed to gain access to the upper metal layer using etching techniques (either chemical liquid etching or plasma etching).

Step 2: Localisation

63 Using signal analysis by standard power analysis allows localizing the cryptographic computation (temporal positioning).

64 Random probing over the surface when executing the crypto primitive allows the detection of areas where wires are involved in the cryptographic calculation (key loading, sub-keys).

65 When an interesting area is located, more systematic probing is performed in this area. That allows detecting wires that are only involved with key or subkey operations (for this, the ITSEF may be able to launch a crypto calculation with different keys to identify wires only involved with the key - or part of it).

Step 3: Probing/Acquisition of data

66 After setting up a scenario (key loading), the commands are executed and an acquisition of the signal (analogue or digital) of the bus activity is performed.

Step 4: Exploitation of data

67 An extra phase is often needed to obtain the bit values from the acquired signals. Depending on the complexity of the software countermeasures, this phase could be anywhere from easy (e.g. interpreting binary values from analogue signals) to impossible (reconstructing a consistent value from partial acquisitions).

2.1.6.2 Example rating 1 with countermeasures

68 For this example, we consider an IC (typically an 8-bit micro-controller) with glue logic as countermeasures, but in particular featuring:

- No active shield (anti-probing layer) over the glue logic

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Preparation of the IC (de-processing), systematic search to locate the relevant wires, data acquisition and interpretation require a significant time, certainly more than 1 week and probably less than 1 month for the IC taken into account in this example. For exploitation, preparation of the IC and data acquisition has to be performed. Several days are required.	<1 month (3)	<1 week (4)
Expertise	Doing the random probing requires to be experienced in such manipulation, both in identification and exploitation.	Expert (5)	Expert (4)
Knowledge of TOE	Knowledge of the commands (APDUs) is required.	Restricted (2)	Public (0)
Open Samples/ Known Key	In case keys can be chosen and are known	(0)/(4)	NA
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 samples (0)	< 10 samples (0)
Equipment	For this attack, we consider that the inspection equipment, the probe equipment and plasma etching belong to one testbench. So no points are added.	Specialized (3)	Specialized (4)
Sub Total		13/17	12
Total		25/29	

69 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

2.1.6.2.1 Required demonstration of attack to justify this rating

70 The required demonstration is the successful probing of a cryptographic key or a part of it (subkey). A rationale shall be provided to explain how to retrieve the full key.

2.1.7 Example: Reading ROM

2.1.7.1 Attack path

71 In this example the attacker uses a staining attack to read the contents of (ion-implanted) ROM. For this to be exploitable as an attack in its own right we assume that a secret master key (e.g., enabling access to confidential information or other protected assets for an entire class of devices), or other authentication data is held in the ROM. Therefore this ROM reading attack leads to a direct vulnerability. In other situations it could provide further information to assist other attacks, as for example revealing

details of DPA or DFA countermeasures in a cryptographic function implementation. Once the master key has been retrieved, no further work is required to compromise all TOEs having the same master key. Consequently, no points can be awarded for the Exploitation phase.

- 72 In principle a similar approach to the one taken here can be pursued when rating other non-volatile memories such as, e.g., EEPROM, or Flash memory. However, it should be kept in mind that making content visible in those types of memory is considerably harder to achieve – likely requiring an AFM – and even that has been demonstrated only for a few bits yet but not for an entire memory. As a consequence attacks along these lines on these non-volatile memories are currently considered “Not Practical”. And even if these types of memories could be read out at some point in the future, they can be encrypted die-individual, and hence the Exploitation phase will have to be considered as well then.

2.1.7.2 Example rating 1 without countermeasures

- 73 We assume that there is no encryption or scrambling of the ROM contents. Although this general type of attack is well known, we assume that there is some work to do during identification in order to locate the data required for an attack.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the attacker requires very limited physical experiments to confirm a logical-to-physical location mapping. In addition to the time required to extract the physical image some further time is required for (at least partially) reverse engineering the binary thus obtained.	< 1 week (2)	NA
Expertise	An Expert rating is required, because of the de-processing techniques and equipment operation required, as well as the ability to identify and interpret the required data in ROM.	Expert (5)	NA
Knowledge of TOE	The attack is assumed to require Restricted information to help locate target data in ROM, and to make sense of its structure.	Restricted (2)	NA
Access to TOE	More than 1 sample may be necessary, but less than ten.	< 10 samples (0)	NA
Open Samples / Known Key	Not required	NA	NA
Equipment	De-processing, staining and inspection equipment. Tools for extracting and analysing (disassembling) code	Specialized (3)	NA
Sub Total		12	0
Total		12	

- 74 This TOE would achieve no rating (VAN.1) with CC v3.
- 75 If the ROM to be extracted is very large (>32kB) it might take more than a week to get reliable data, but the TOE would still achieve no rating.

2.1.7.2.1 Required demonstration of attack to justify this rating

- 76 Perform successful photographing of the ROM, showing that it can be read clearly, and a method for identifying and using the data of interest (e.g. decompiling source code or recognising keys). As a result, the key could be extracted.

2.1.7.3 Example rating 2 with 1 countermeasure

- 77 For the following example we assume a TOE featuring a 100kB ROM of 8 bits granularity, designed at a 0.18µm technology node.

Countermeasures of the IC (taken from the countermeasure list):

- Hardware encryption of memories (data scrambling only, so no address scrambling, no non-linear encryption)

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	The attacker can discover the scrambling mechanism, but reading out the entire ROM and descrambling the ROM would still take up to one month. (Note that stronger encryption mechanisms could make all the difference here. Also, a wider width of the memory would increase the time required to perform the descrambling / decoding.)	< 1 month (3)	NA
Expertise	An expert is necessary to etch and perform the reverse engineering of the IC. When statistical analysis is necessary to descramble the data, the expertise level is still Expert for the identification of the attack, but more time is needed to perform the entire attack.	Expert (5)	NA
Knowledge of TOE	The attack is assumed to require Restrictive information to help locate target data in ROM, and to make sense of its structure.	Restricted (2)	NA
Access to TOE	Access to TOE will in practice be in the order of less than 100 samples, even though the attack makes the sample unusable after the attack has been performed.	< 100 (2)	NA
Open Samples / Known Key	Not required	NA	NA
Equipment	If smaller dimensions make the use of a FIB necessary, this means specialized or bespoke equipment is necessary.	Specialized (3)	NA
Sub Total		15	0
Total		15	

- 78 This TOE would achieve no rating (VAN.1) with CC v3.

2.1.7.4 Example rating 2 with 2 countermeasures

- 79 For the following example we assume a TOE featuring a 100kB ROM of 8 bits

granularity, designed at a 0.18 μ m technology node.

Countermeasures of the IC (taken from the countermeasure list):

- Hardware encryption of memories (data scrambling, address scrambling, non-linear encryption)

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	The attacker can discover the scrambling mechanism, but reading out the entire ROM and descrambling the ROM would still take up to one month. (Note that stronger encryption mechanisms could make all the difference here. Also, a wider width of the memory would increase the time required to perform the descrambling / decoding.)	Not practical	NA
Expertise	An expert is necessary to etch and perform the reverse engineering of the IC. When statistical analysis is necessary to descramble the data, the expertise level is still Expert for the identification of the attack, but more time is needed to perform the entire attack.	Expert (5)	NA
Knowledge of TOE	The attack is assumed to require Restrictive information to help locate target data in ROM, and to make sense of its structure.	Restricted (2)	NA
Access to TOE	Access to TOE will in practice be in the order of less than 100 samples, even though the attack makes the sample unusable after the attack has been performed.	< 100 (2)	NA
Open Samples / Known Key	Not required	NA	NA
Equipment	If smaller dimensions make the use of a FIB necessary, this means specialized or bespoke equipment is necessary.	Specialized (3)	NA
Sub Total		Not practical	0
Total		Not practical	

80 The physical matrix image of a ROM memory can be read out with rather standard methods and thus one has to assume that the bit file image of the ROM can be known by an attacker. A natural protection is thus to encrypt the ROM bit file so that the attacker can only retrieve a ciphertext when reading out the physical image of the memory. [Note: For other types of memory such as EEPROM or Flash it is substantially harder to retrieve this physical image and hence at this point in time the ROM and its encryption is in the focus of the discussion.]

81 The encryption of memories serves multiple purposes such as IP protection for the vendor, making it harder for attackers to analyse SW countermeasures (e.g. against side-channel or fault attacks) by simply hiding these from the attacker, and lastly to protect secret keys and other assets that may be stored in this memory. [Note: It is discouraged

to store global system secrets in memories that do not have diversified keys such as ROM.]

82 Although reverse-engineering of memories has been known for a long time, this subject got renewed interest with recent publications by Tarnovsky and Nohl. [Ref] In particular the appearance of freely available tools on the internet to support the reverse engineering process such as the DEGATE program [Ref] means that there is a high chance of substantial progress in this area in the next few years. The reverse engineering and bit file image retrieval can be rendered harder thanks to advanced ROM technologies and the use of HW masking techniques. However, the memory protections cannot rely only on these features. The use of cryptographic techniques to conceal the bit file image is unavoidable.

83 From a cryptographic point of view it would be desirable to implement strong encryption schemes that have been thoroughly vetted by the cryptographic community. However, this view is neglecting a number of major differences in the type and nature of challenges encountered when reverse engineering a smart card memory compared to a classical cryptanalysis, such as

Differences in context:

84 ROM ciphertext is relatively difficult to obtain for an attacker since one has to carry out several other attack steps before. The assumption is that the evaluation will show that the cipher text is also adequately protected in the development and production environments.

85 On top of the encryption of the ROM data, there is also address scrambling, which makes analysing the mapping between plaintext and ciphertext much harder. If this mapping is not known, many standard cryptographic attacks simply fail. If it needs to be reverse-engineered then this adds significantly to the attack potential.

86 Memory size is also a factor: bigger memory is more difficult to prepare for being read out physically (if we need all of it)

Differences in implementation constraints:

- A smart card IC cannot employ a conventional strong block encryption algorithm, such as AES, for various reasons, including performance, power and area. It is believed that this is an acceptable constraint because of the other difficulties in accessing plaintext and reverse engineering the encryption design.
- What is currently feasible?
 - Nonlinear encryption with “a few rounds”
 - 32-64 bit key
 - Address scrambling
- It is discouraged to use a linear encryption in the absence of address scrambling, as this would then boil down to a completely linear problem, which is trivial to solve.
- On the other hand, there is some merit seen in keeping proprietary memory encryption algorithms secret, despite knowing that not having exposed those algorithms to a lengthy cryptographic peer review process does mean that cryptographic weaknesses in these algorithms are likely to exist.

Differences in cryptanalysis:

- It is difficult for an attacker to generate **chosen** plaintexts – only a member of the IC or OS developer’s staff should be able to do this, and doing it too many times would arouse suspicion.
 - It probably requires significant collusion, but it should be checked in each case whether there is a single critical tool available for this.
 - ROM encryption designers and people with knowledge of ROM content are assumed to be separate in a typical organisation.
 - It is difficult for an attacker to use **known** plaintexts, but this might be less true for some parts of the ROM if, say, byte code is reused between TOEs. An example where reuse is quite common is the Firmware, which is often standard and stored in ROM (even at same locations?). Other examples may be crypto libraries. And then there are standard protocols with standard – known – data structures. If parts of those are kept in ROM, then those may also serve as known plaintexts.
 - Assuming that state of the art ROM encryption does not use one static key for the entire memory space, then this will force an attacker again to perform a deeper analysis, even if he could guess the plain text of the first bytes output. This is essentially the requirement of address dependent encryption.
 - It is encouraged to use different keys for different ROM masks in order to limit the damage of any attack to one product only, and not have the entire family affected.
 - Even IF the entire ROM code were known, the total number of plaintext / ciphertext pairs available is typically of the order of 2^{18} , which is much less than what is required for many *theoretical* attacks to succeed. Thus, a possible guidance to determine a sufficient strength of the ROM encryption could be that we are ok with cryptographic attacks that require 2^{18} pairs of plaintext/ciphertext.

2.1.7.5 Countermeasures

- 87 As a general principle, the reasoning above is not meant to be a list of hard requirements, but rather is intended to serve as a guideline for the analysis of the strength of the memory encryption algorithm. It is not the intent to limit innovation in this area by imposing certain rules. It is also admitted that the sole implementation of an encryption or a scrambling mechanism is not sufficient to ensure a long resistance to an attacker with Medium to High potential.
- 88 The best way to achieve a maximum protection of the ROM content is the use of all possible protections (environmental, advanced HW technology, encryption and scrambling techniques).
- 89 Given that a single successful attack will necessarily reveal the secret key used in all TOEs having the same ROM code, as a general rule, it is always recommended not to store critical data such as master keys or password values in ROM.

2.2 Overcoming sensors and filters

2.2.1 Description of Attack

90 This attack covers ways of deactivating or avoiding the different types of sensors that an IC may use to monitor the environmental conditions and to protect itself from conditions that would threaten correct operation of the TOE. Hardware or software may use the outputs from sensors to take action to protect the TOE.

91 Sensors and filters may be overcome by:

- Disconnection (and/or bypassing)
- Changing the behaviour of the sensor
- Finding gaps in the coverage of the monitored condition (e.g. voltage), or of the timing of monitoring.

92 Sensors may also be misused in order to exploit activation of a sensor as a step in an attack. This misuse of sensors is a separate attack.

93 The different types of sensors and filters include:

- Voltage (e.g. high voltage or voltage spike)
- Frequency (e.g. high frequency or frequency spike)
- Temperature
- Light (or other radiation)

2.2.2 Effect of Attack

94 The chip continues to operate under environmental conditions outside the specified range¹. For example, a chip might be operated outside the safe operating voltage, whether or not it returns incorrect results.

95 Effects may be temporary or permanent.

2.2.3 Impact on TOE

96 Under this attack, the correct operation of a chip can no longer be guaranteed outside the safe operating conditions. The impact of operating under these conditions may be of many sorts. For example:

- Contents of memory or registers may be corrupted
- Program flow may be changed
- Failures in operations may occur (e.g. CPU, coprocessors, RNG)

¹ The specified range is inside the safe range (see [PP0084] section 146)

- Change of operating mode and/or parameters (e.g. from user to supervisor mode)
- Change in other operating characteristics (e.g. changed leakage behaviour; enable other attacks like RAM freezing, electron beam scanning).

97 If a chip returns incorrect cryptographic results then this may allow a DFA attack, see section 2.4. Other possible consequences are described under general perturbation effects in section 2.3.

2.2.4 Characteristics of the Attack

98 Disconnection requires a physical manipulation (see section 2.1). As a result the intended sensor alarm is not activated. As a first step the sensor has to be located and analysed.

99 Changing the behaviour of the sensor may be achieved by physical manipulation, but also by changing environmental conditions of the chip and exploiting effects of environmental conditions applied globally or locally to the TOE. These other operating conditions may or may not be monitored by sensors of the chip.

Note: Changes may be caused directly or indirectly by changing stored parameters.

Note: To select the right tests and parameters an experienced decision is required.

2.2.5 Example: Combination of attacks

100 Overcoming sensors and filters represents an attack step that must be combined with others to create an exploitable attack. In particular, the general physical attacks in section 2.1 and the perturbation attacks in section 2.3 are relevant.

101 In general, this attack step would be expected to follow the calculation for a physical attack or perturbation attack (using the same sort of equipment, expertise, and number of TOE samples). However, the attack step may increase the Elapsed Time for Identification and Exploitation, and/or the Knowledge of TOE required for Identification.

2.3 Perturbation Attacks

2.3.1 Description of Attack

- 102 Perturbation attacks change the normal behaviour of an IC in order to create an exploitable error in the operation of a TOE.
- 103 The attacks aim at protecting mechanisms and typically include the following:
- Reducing the strength of cryptographic operations,
 - Tampering with memory protection mechanisms,
 - Affecting non-volatile monotonic counter values.
- 104 Creating faults can be used to recover keys or plaintext, to change the results of validation such as authentication or lifecycle state checks, to change the program flow, to provide unauthorized access to protected memory or to enable rollback and replay attacks.
- 105 Section 2.2 is concerned more with the methods to induce meaningful faults whereas Section 2.3 describes how these induced faults may be used to create exploitable vulnerabilities.

2.3.2 Effect of Attack

- 106 Various types of attack can be launched on an IC by trying to change the normal behaviour of the IC hardware. Typically the IC behaviour is changed either by applying an external source of energy during the operation of the IC, or by operating the IC outside its intended operating environment (usually characterised in terms of temperature, VCC and the externally supplied clock frequency). This type of attack is referred to as ‘applying a perturbation’.
- 107 Under perturbations, the normal behaviour of an IC can sometimes be altered. The physical effects depend on the type of stress (e.g. light, laser, voltage/clock glitches) but the external effect can be seen either as a change to data, or as an incorrect execution of software. When data is changed, a bit (or a group of bits) may either invert its current value (0 to 1 or 1 to 0), or become set to a specific value (either 0 or 1). The effect can be permanent (e.g. if a value in memory is changed) or temporary. Temporary effects may occur only while the perturbation is applied (e.g., during read-out from memory, or when affecting registers in the CPU), or may last for some time after the perturbation has been stopped (e.g., until the next reset occurs, or the data have been overwritten). In other cases, where no data is changed, the general effect is that an instruction is executed in a way that does not comply with its specification. For example, an instruction may be skipped, or replaced by another instruction (benign for the purposes of the attacker), or the wrong branch of a conditional jump instruction may be taken².
- 108 The effect of a perturbation may be predictable, or it may be less certain. In some cases, the attack will succeed on every attempt, whereas in other cases the probability of the effect will be substantially less than one.
- 109 Despite the fact that high probabilities tend to be more useful to an attacker, low probabilities can also lead to successful exploitation, if the attack operation can be

² In some cases, the cause of the incorrect behaviour can be explained in terms of a data corruption. For example, a corruption to the Program Counter may cause an instruction to be skipped, while a corruption to the Zero condition flag may cause the wrong branch of a conditional to be taken. However, it is useful to distinguish between perturbations that cause corruptions to data values that are part of software data structures and perturbations that leave software data unchanged but change the execution behaviour.

repeated many times. The latter is valid, for example, if the operation is not subject to a threshold check such as a limit on the number of successive PIN check failures.

110 If the odds for a single perturbation attack are suitably high, it is also worthwhile considering to apply two or more perturbations in a row, in an attempt to defeat simple countermeasures.

111 An attacker may use perturbations to attempt a relatively complex theoretical attack such as differential fault analysis (see section 2.4), in which an error in cryptographic processing enables the key to be recovered from one (or a few) pairs of correct and incorrect calculations. On the other hand, the attacker may simply use trial and error to see what observable effects perturbations can cause at different times during the execution of software. For example, if the IC could be influenced to take the wrong branch of a conditional jump instruction then a PIN check might be bypassed.

2.3.3 Impact on TOE

112 Perturbations may be applied to either a hardware TOE (an IC) or a software/composite TOE (an OS or application running on an IC).

113 For attackers, the typical external effects on an IC running a software application are as follows:

- Modifying a value read from memory during the read operation: The value held in memory is not modified, but the value that arrives at the destination (e.g. CPU or coprocessor) is modified. This may concern data or address information.
- Modifying a value that is stored in volatile memory, including auxiliary CPU and MPU registers. The modified value is effective until it is overwritten by a new value, and could therefore be used to influence the processing results or the security policy of the device.
- Modifying non-volatile monotonic counter values used to ensure the data freshness. Glitching or reducing the voltage of the power supply at the counter increment can result in a marginal value giving a possibility to roll the counter back, thus enabling replay attacks (if no special countermeasures, like a checksum of a counter, are implemented).
- Changing the characteristics of random numbers generated (e.g. forcing RNG output to be all 1's) – see section 2.7 for more discussion of attacks on random number generators.
- Modifying the program flow: the program flow is modified and various effects can be observed or deduced:
 - Skipping an instruction
 - Replacing an instruction (typically a jump) with another (“benign”) one
 - Inverting a test result / conditional branch
 - Generating a jump
 - Generating calculation errors
 - ...

114 It is noted that it is relatively easy to cause communication errors, in which the final data returned by the IC is modified. However, these types of errors are not generally

useful to an attacker, since they indicate only the same type of errors as may naturally occur in a communication medium: they have not affected the behaviour of the IC while it was carrying out a security-sensitive operation (e.g. a cryptographic calculation or access control decision).

- 115 The range of possible perturbation techniques is large, and typically subject to a variety of parameters for each technique. This large range and the further complications involved in combining perturbations mean that developing a perturbation attack is usually preceded by investigating what types of perturbation cause any observable effect, and then refining this technique both in terms of the parameters of the perturbation (e.g. small changes in power, location or timing) and in terms of what parts of software are attacked. For example, if perturbations can be found to change the value of single bits in a register, then this may be particularly useful if software in a TOE uses single-bit flags for security decisions. The application context (i.e. how the TOE is used in its intended operating environment) may determine whether the perturbation effect needs to be precise and certain, or whether a less certain modification (e.g. one modification in 10 or 100 attempts) can still be used to attack the TOE.

2.3.4 Characteristics of the Attack

- 116 A modern smart card IC will normally implement various countermeasures against perturbations: voltage/frequency filters and threshold detectors, glitch detectors, temperature detectors and (discrete) light detectors are all commonly included in a smart card IC. In addition, there may be hardware and/or software measures that detect some types of modification to data (e.g. integrity checksums on memory or data structures, and detectors for invalid instruction codes or illegal addresses). Therefore, the evaluator attempts to find a perturbation technique that is not detected or countered by the hardware, or else he applies the perturbations as part of a combined attack after attempting to disable the countermeasures. This is based on a combination of testing and design analysis.
- 117 Perturbations attack particular features on a die (e.g. laser attacks). The first difficulty in carrying out an attack is therefore to find a location (in time and space) at which the perturbation is effective. In the absence of appropriate hardware countermeasures, the edges of memories such as ROM, EEPROM, and Flash are often very useful points to start the investigations that are perhaps easier to interpret than perturbation attacks on other parts of the TOE. This does not mean, though, that other parts of the TOE are not equally susceptible to these attacks – it is only a matter of finding them. This may be guided by design information but will also typically rely on extensive trial-and-error in areas such as the random logic, where suitable targets are difficult to identify from design information (at least without access to the developer's design tools).
- 118 Where perturbations are used to change particular data values in memory, it may be difficult to locate the specific locations required (especially if memory addressing and content is scrambled or encrypted). A successful attack in this case may require a combination of perturbation with reverse engineering of the scrambling/encryption mechanism and/or physical modification of the chip to overcome this countermeasure.
- 119 The effect of perturbations can also be highly sensitive to the synchronisation with stages of an operation. In some cases, an attack may only be successful if the perturbation is applied precisely at the time of a particular decision (such as a PIN check result) in software. Sometimes the synchronisation may need to be even more precise: perhaps during a specific part of the instruction cycle, or during a precise stage in a cryptographic coprocessor calculation. In general, perturbation is not a fully predictable

process. An effect might be seen with a low probability (1% for example), and the interesting effect could in turn be seen with a low probability amongst these events (also 1% for example, in which case a successful attack could occur in only 0.01% of all cases). Synchronisation may be one of the factors in the success, and is thus a critical parameter of the attack. This may result in linkage between the observable power behaviour of a chip (as used in SPA) and the use of synchronisation tools by the evaluator. In some cases, the TOE may allow the attacker to produce events to aid synchronisation (e.g. if the evaluator's own applications can be loaded).

120 The precise effect of perturbations such as changing a bit value depends, of course, on which bit is changed and when the change occurs. In general, without studying specific details of an individual TOE, it is impossible to predict the effect of a perturbation, or even to predict whether anything at all will happen. The only way of using this attack technique is to make experiments in order to create a characterisation of the IC with the following parameters:

- Type of perturbation
 - For glitches:
 - Intensity & profile, duration
 - interface used (VCC, clock,)
 - For light or lasers
 - Type of laser (power, wavelength)
 - Mode (pulsed or continuous)
 - Dimension of the laser spot
 - Where to apply it on the IC
 - Apply the laser on the structure or the substrate side of the IC (back side or lateral side)
 - Other types of perturbation (e.g. electromagnetic pulses, radiation, or capacitively coupled AC) are possible, and may be classified in terms of similar parameters indicating how the perturbation is caused, and where it is applied (either in terms of a functional interface to the IC or a location on the die).
- Effect
- Test program running tests, loops, calculations, ...
- Synchronisation
- When to apply the perturbation (which cycle(s), when in the cycle, etc)

121 Because it is difficult to defend intrinsically against all types of perturbation attacks, yet it may be possible to write software to defend against the *effects* of perturbations, an evaluation should consider the effectiveness of any user guidance concerning the effects of perturbations for an IC TOE.

122 In the case of a composite TOE that includes software and hardware, the software may apply countermeasures for perturbations caused to the hardware, and this may render some perturbation attacks ineffective. However, it will usually be necessary to carry out experiments to test the practical effectiveness of the countermeasures.

123 Because there is a variety of ways to cause perturbations, the equipment (and associated expertise required) can vary significantly. Some of the common attacks use equipment

classified as Standard in [JIL-AP] to cause the basic perturbation – for example flashguns, lasers (of the commonly available kind, such as laser pointers), or signal generators.

124 However, in other cases more complex equipment (Specialised or even Bespoke in terms of [JIL-AP]) may be needed to generate the perturbation (e.g. a more sophisticated pulse laser) and/or to synchronise the perturbation with a point in the operation of the TOE (e.g. a digital oscilloscope might be used to help generate the perturbation at a particular point in a DES operation).

125 The expertise to carry out the attack depends to some degree on the equipment used (see [JIL-AP] section 3.3), but may also depend on the detailed knowledge and experience required to locate the point – in space and time – for applying the perturbation and interpreting the result. For example, an attacker may need to reverse engineer the operation of software from oscilloscope traces in order to locate a suitable point to apply the perturbation. The TOE may then output incorrect results, but it may require some relatively complex analysis to recognise what the result is (relative to the correct operation) and how to exploit it. Hence in a simple case, such as where an authentication check is avoided or a flag is corrupted, and where the synchronisation is easy, expertise may be no more than Proficient in [JIL-AP] terms. Where reverse engineering is required, and/or a complex interpretation is made from the results in order to recognise an exploitable fault in a DES operation, then the expertise may be rated as Expert.

126 Some applications of perturbation attacks in public literature may be found in the following papers:

- *Low Cost Attacks on Tamper-Resistant Devices*, Anderson & Kuhn, proceedings of 5th Security Protocols Workshop, 1997
- *Optical Fault Induction Attacks*, Skorobogatov & Anderson, proceedings of CHES 2002
- *Design Principles for Tamper-Resistant Smart CardProcessors*, Kommerling & Kuhn, proceedings of Usenix 1999
- *On the Importance of Checking Cryptographic Protocols for Faults*, Boneh, DeMillo & R. J. Lipton, proceedings of Eurocrypt 97
- *Differential Fault Analysis of Secret-Key Cryptosystems*, Biham & Shamir, proceedings of CRYPTO'97

127 In summary: There are many potential ways of applying perturbations to an IC, and evaluators will always be justified in applying any perturbation techniques that they can demonstrate in the laboratory. However, the following is a set of perturbations that may be considered as ‘obvious attacks’:

- High and low voltage (outside intended operating environment)
- High and low frequency (outside intended operating environment)
- VCC glitches (short duration)
- Clock glitches (short duration)
- Laser attacks to selected locations (from front-side, lateral-side and back-side of TOE)

- 128 Flashgun attacks, although widely known and often referred to, are not specifically included in this list. This is because laser attacks are generally more effective and they can typically be applied to a smaller area of the die and more precisely synchronised with activity on the chip. However, it may be important to assess whether an attack can actually be made with a flashgun (or other simple equipment) because the work function is reduced if that is the case.

Note: In some cases it may be possible to demonstrate from design information that a TOE is resistant to certain types of perturbation (e.g., because of the use of integrity checks on critical data, provided of course that perturbations cannot be used to bypass the integrity check).

2.3.5 Example: cryptographic failure

2.3.5.1 Attack path

129 In this example an attacker applies a laser to a TOE while sending commands that involve a DES calculation that is used as a MAC value to check the integrity and authenticity of a message. The attacker observes the outputs and finds that under this type of perturbation the DES calculation can sometimes be forced to give an output of all 1's. He therefore makes an attack based on repeatedly sending a fake message with a MAC consisting of all 1's (the attack assumes that the message is not encrypted), whilst applying the perturbation until the TOE accepts the command. The attacker knows the command has been accepted because of the software values returned.

- The attack then involves the following steps:

- 1) *Prepare the sample by providing laser access to the silicon. Depending on the characteristics of the light source (laser, flashgun, wavelength, power) and on the IC characteristics (protection, sensitivity, light sensors) this step could be very simple (removing packaging) or more complex, which could include following actions: remove the IC from its packaging, de-processing down to the passivation layer (this may use chemical etch) or lay open the substrate of the IC (this may be done by still milling the protecting material or even thinning the substrate). The laser light can be applied from the top, the lateral or the back of the IC.*
- 2) Connect the IC to a test harness that repeatedly sends commands to the IC in order to cause DES encryptions to be carried out, and records responses that include the result of the DES encryptions.
- 3) Repeatedly apply the laser while operating the IC and monitor the results³.

130 After observing the intermittent failure of the DES when using the laser at certain locations with certain timing, the attacker can then exploit the attack.

- 1) Create a fake command, adding a MAC value of all 1's. Repeatedly send the command whilst applying the laser in the same way (same location, same timing) as before until the response (or subsequent TOE behaviour) indicates the message has been accepted. This step involves identifying a suitable command that can be exploited in this way, and constructing the values necessary for exploitation. It is assumed for the purposes of this example that this information is not publicly available, but can be found in (or deduced from) a limited-distribution functional specification and observation of commands sent to the TOE in its 'live' environment.

131 One can assume that obtaining a perfect result (forcing the DES output to be all 1's) occurs rarely, but in general it can be achieved within about 2 days of repetition (using manual adjustment of energy, timing and location).

³ For a more sophisticated fault attack, it may be difficult to recognise a fault, or to analyse the fault in a way that allows it to be exploited, without knowing the key. However, in this case we assume that the 'all 1's' situation is observed directly in the response.

2.3.5.2 Example rating 1 without countermeasure

Countermeasures of the IC (taken from the countermeasure list):

- For this attack, an IC without any special countermeasures against light attacks is assumed.

132 The attack potential calculation is then as is shown in the table below:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the de-processing and initial testing to identify the failure take less than 1 week (assuming the restricted command information is available). We assume exploitation based on a written description of the attack and the commands to use.	< 1 week (2)	< 1 day (3)
Expertise	The attacker can recognise the fault by simple observation of a DES output, so even a Proficient attacker can do the identification. A Proficient rating is also required for exploitation, because of the de-processing techniques and equipment operation required. (A more sophisticated DES fault, such as corruption of the key to mostly 1's, which is not evident by direct observation of the result might justify an 'Expert' rating for identification, as well as a longer elapsed time.)	Proficient (2)	Proficient (2)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	In the Identification phase, the attacker needs to be able to load freely chosen keys in order to perform the analysis.	Sensitive (4)	Public (0)
Equipment	Minimum requirement is equipment to de-process the chip, bond out the pads, and then to generate and analyse the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		13	9
Total		22	

133 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3) with CC v3.

134 It should be noted that in this example it is not possible to achieve a high rating even when adding SW countermeasures as, e.g., the lack of any HW fault detection for the memories means that code cannot be executed reliably. For the same reason it is also not possible to operate a fault counter reliably. (Yet, even when the code fetch from memory is protected, there may still be "easy" attacks targeting, e.g., registers in the

glue logic.)

2.3.5.2.1 Required demonstration of attack to justify this rating:

135 It has to be demonstrated that the message has been accepted.

2.3.6 Example: authentication bypass

2.3.6.1 Attack path

136 In this example, an attacker applies a laser to a TOE while an authentication check is carried out. The attacker repeatedly sends inauthentic commands and attempts to get a command accepted, which he can judge by observing either subsequent TOE behaviour or the response value.

137 The attack then involves the following steps:

- 1) *Prepare the sample by providing laser access to the silicon. Depending on the characteristics of the light source (laser, flashgun, wavelength, power) and on the IC characteristics (protection, sensitivity, light sensors) this step could be very simple (removing packaging) or more complex, which could include following actions: remove the IC from its packaging, de-processing down to the passivation layer (this may use chemical etch) or lay open the substrate of the IC (this may be done by still milling the protecting material or even thinning the substrate). The laser light can be applied from the top, the lateral or the back of the IC.*
- 2) Connect the IC to a test harness that repeatedly sends inauthentic commands to the IC in order to cause the authentication check to be carried out and records responses.
- 3) Repeatedly apply the laser while operating the IC and monitor the results. It is assumed that in the absence of any suitable countermeasures the TOE is in particular vulnerable to attacks in the area of the memory blocks that the target code is being executed from (e.g., ROM, EEPROM, Flash), which are usually easy to locate. However, vulnerable spots are also equally likely to exist in the glue logic.

138 After observing occasional acceptance of an inauthentic command when using the laser at certain locations with certain timing, the attacker can then exploit the attack.

- 1) Create a fake command with the desired exploitation effect (e.g. writing some critical data, such as a lifecycle state). Repeatedly send the command whilst applying the laser in the same way (same location, same timing) as before until the response (or subsequent TOE behaviour) indicates the message has been accepted. This step involves identifying a suitable command that can be exploited in this way, and constructing the values necessary for exploitation. It is assumed for the purposes of this example that this information is not publicly available, but can be found in (or deduced from) a limited-distribution functional specification and observation of commands sent to the TOE in its 'live' environment.

139 For this example, knowledge of the TOE, and in particular of the implementation of the TOE software, may make a significant difference to the time taken to find a suitable

point at which to apply the perturbation. In this case, the identification part of attack potential calculations using [JIL-AP] assumes only knowledge sufficient to operate the TOE in ways that the attacker can use to repeat the same processing in order to develop the attack in terms of where and when to apply the perturbation.

- 140 Depending on the level and number of countermeasures implemented in the TOE, it may also make a significant difference to the attack time if the attacker has ‘open’ samples of the TOE available – i.e. if the attacker can load his own code onto the same chip (using either the TOE, or some other product that uses the same chip but offers the ability to download code).

2.3.6.2 Example rating 1 with countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Light sensors

- 141 It is assumed that obtaining a perfect result (i.e. continued operation of the TOE as though authentication had succeeded) does not occur on every attempt, and hence the exploitation may not be immediate. However, in most cases repeating the actual attack, once the location and timing for the perturbation has been identified, will require less than an hour.

- 142 The HW countermeasure “light sensors” is required in this example, as otherwise a simple flashgun may be used for mounting the attack. But even then, it is not too difficult to cover parts of the TOE IC with opaque enamel paint and leave only, e.g., the memory blocks open. This is assumed for the exploitation phase.

- 143 It should be noted that as long as the memories from which the code is being executed are vulnerable to attacks, the countermeasure “randomized location of components” (taken from the countermeasure list) cannot be considered effective as often the memory blocks will be tried first by the attacker.

- 144 The attack potential calculation is then as is shown in the table below:

Factor	Comment	Ident’n	Exploit’n
Elapsed Time	We assume the de-processing and initial testing to identify the failure combined with demonstrating the vulnerability take less than a few days (assuming the restricted command information is available). We assume exploitation based on a description of the attack and the commands to use.	< 1 week (2)	< 1 day (3)
Expertise	The attacker can recognise the fault by simple observation of an output (showing that the inauthentic command has been accepted, not rejected), so even a Proficient attacker can do the identification. A Proficient rating is also required for exploitation, because of the de-processing techniques and equipment operation required.	Proficient (2)	Proficient (2)
Knowledge of	The attack requires Restricted information to	Restricted	Public

TOE	identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	(2)	(0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Minimum requirement is equipment to de-process the chip, bond out the pads, and then to generate and analyse the required commands to drive the IC. A simple laser and optical microscope are required in the identification phase to generate the perturbations (whilst a triggered flashgun and covering the TOE with opaque enamel paint is enough in the exploitation phase), and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		9	9
Total		18	

145 This TOE would be resistant for this attack to an attacker with basic attack potential (VAN.2) with CC v3.

146 It should be noted that in this example it is not possible to achieve a High rating even when adding all sorts of SW countermeasures, as the lack of any more effective HW countermeasures such as, e.g., HW fault detection for the memories means that code cannot be executed reliably. (Yet, even when the code fetch from memory is protected, there may still be “easy” attacks targeting, e.g., registers in the glue logic.) For the same reason it is also not possible to operate a fault counter reliably. But even if that were the case, a fault counter would only add to the number of samples required in the Identification phase (+3 points max, but likely to be less – this depends on the number of light sensors). No further points can be awarded in the Exploitation phase, since by then the precise location of the light sensors are known and thus no further samples need to be wasted. It should also be considered that light sensors may be made visible by reverse engineering techniques (e.g., from the back side), in which case they do not really add to the number of samples required. A fault counter may slightly increase the time in the Identification phase, but not exceeding a month. Hence, the maximum rating that could be achieved this way is 23 points only.

2.3.6.2.1 Required demonstration of attack to justify this rating

147 It has to be demonstrated that the authentication is bypassed.

148 There are possible further countermeasures at various levels that affect the attack potential calculation. Examples are:

- Hardware detectors above and beyond simple light sensors, which may detect some sorts of corruptions induced by the perturbations during the identification stage.

- Perturbation-resilient software, which takes account of possible corruptions of data or program flow and reacts appropriately if these are detected (in this case it is important to consider the nature of the perturbation effect, and hence whether the software checks can themselves be defeated by perturbations). Note that the effectiveness of this software depends heavily on how reliably code can be executed in the first place, i.e., on the effectiveness of the hardware countermeasures available in the IC.
- Perturbation-response in software, which may shut down the TOE if a threshold of hardware detection events (or possibly software detection events) is reached. This may make the attack require more samples (if more than 10 samples are required then the attack potential increases). Again, the software needs to run reliably on the IC for this to be effective. Also, it should be considered whether writing to such counters can be detected (and avoided) by way of a power analysis and yet another fault attack.
- Randomised internal execution will make it difficult to repeat an attack because the timing is changing for every try. Additional effort would be needed to get a trigger point for example by analysing the power traces.

149 Some TOEs may make it difficult to synchronise attacks reliably (e.g., where perturbation-resilient software means that there are only small time windows in which an attack can be applied), and in these cases the weakest path for an attack may need to consider a higher level of TOE knowledge (e.g. software source code) rather than assuming a higher Expertise and longer elapsed time.

2.3.6.3 Example rating 2 with countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Light sensors (in Glue Logic)
- Hardware fault detection for memories (that are effective in detecting when code is not read correctly from memory, leading to, e.g., a reset, but not necessarily a non-functional TOE)
- Randomized location of components (e.g., Glue Logic)

150 An efficient HW fault detection for (code-executing) memories forces the attacker to scan the randomised logic for possible loopholes as he cannot immediately identify individual components of the logic design to attack. It also is the basis for adding further layers of SW countermeasures.

151 The attack potential calculation is then as is shown in the table below:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the de-processing and initial testing to identify the failure combined with demonstrating the vulnerability take less than 1 month (assuming the restricted command information is available). We assume exploitation based on a description of the attack and the commands	< 1 month (3)	< 1 week (4)

	to use.		
Expertise	The attacker can recognise the fault by simple observation of an output (showing that the inauthentic command has been accepted, not rejected), so even a Proficient attacker can do the identification. A Proficient rating is also required for exploitation, because of the de-processing techniques and equipment operation required.	Proficient (2)	Proficient (2)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	Not Required	NA	NA
Equipment	Minimum requirement is equipment to de-process the chip, bond out the pads, and then to generate and analyse the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		10	10
Total		20	

152 This TOE would be resistant for this attack to an attacker with basic attack potential (VAN.2) with CC v3.

153 Yet, in contrast to the previous example, there is a much better chance now that additional SW countermeasures can raise the attack potential.

2.3.6.3.1 Required demonstration of attack to justify this rating

154 It has to be demonstrated that the authentication is bypassed.

2.3.7 Example: Changing output address range and dumping memory

2.3.7.1 Attack path

155 As already mentioned in the general description of the attack, it is always possible to perturb the IC in some way and normally the defence is a combination between hardware and software countermeasures. To get a clear separation no software countermeasures are taken into account here. The attacker uses a laser flash to perturb the TOE while the addresses for an output routine are handled. The attacker uses an application command, which transfers data from the TOE to the environment, and the target of his manipulation is to dump secret memory contents.

156 The attack then involves the following steps:

- 1) *Prepare the sample by providing laser access to the silicon. Depending on the characteristics of the light source (laser, flashgun, wavelength, power) and on the IC characteristics (protection, sensitivity, light sensors) this step could be very simple (removing packaging) or more complex, which could include following actions: remove the IC from its packaging, de-processing down to the passivation layer (this may use chemical etch) or lay open the substrate of the IC (this may be done by still milling the protecting material or even thinning the substrate). The laser light can be applied from the top, the lateral or the back of the IC.*
- 2) Connect the IC to a test harness that repeatedly sends output request commands to the IC.
- 3) Monitor the power traces during the execution of the command and identify distinct sequences of the command. Use characteristic power traces as a trigger for the laser pulse. Knowledge of the TOE, and in particular of the implementation of the TOE software, may make a significant difference to the efficiency of defining the right trigger and timing.
- 4) Repeatedly apply the laser at different locations while operating the IC and monitor the results. It is assumed that the TOE uses randomised logic and hence that the attacker cannot immediately identify individual components of the logic design to attack (although certain targets such as memory blocks and their interfaces will inevitably be visible – but for the following it is assumed that those are not susceptible to this attack). If the light sensors are well implemented, the area covered by the laser light with one shot will have to be very small (of the order of 10 – 50 μm square). In most cases, a light sensor reset will be caused. Therefore, the time to discover the areas where the TOE can be forced to make errors may be considerable. However, attention should also be paid to the possibility that the light sensors may be made visible from the backside and thus can be avoided in the further laser scans.
- 5) Laser pulses at different combinations of trigger and location have to be applied and the output sampled and analysed for interesting memory content. Due to the randomised internal code execution, the trigger cannot be defined at a fixed position during the execution time but has to be derived from a pattern recognition algorithm (or trial and error).
- 6) The dumped memory content has to be analysed, interpreted and put together

for an image of the memory.

- 157 For this example, knowledge of the TOE, and in particular of the implementation of the TOE software, may make a significant difference to the time taken to find a suitable point at which to apply the perturbation. In this example, the identification part of attack potential calculations using [JIL-AP] assumes only knowledge sufficient to operate the TOE in ways that the attacker can use to repeat the same processing in order to develop the attack in terms of where and when to apply the perturbation.
- 158 It may also make a significant difference to the attack time if the attacker has ‘open’ samples of the TOE available – i.e. if the attacker can load his own code onto the same chip (using either the TOE, or some other product that uses the same chip but offers the ability to download code). With this it is possible for him to create a test output routine and scan the entire open-sample surface for weak spots. Having found the weak spot he can then go on and determine the right moment in time to attack the real TOE.
- 159 It is assumed that obtaining a perfect result (i.e. dump of confidential memory content) does not occur on every attempt, and hence the exploitation may not be immediate. However, in most cases repeating the attack, once the location and timing for the perturbation has been identified, will require less than a week.

2.3.7.2 Example rating 1 with countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Randomised location of components
- Light sensors
- Hardware fault detection for memories (that are effective in detecting when code is not read correctly from memory, leading to, e.g., a reset, but not necessarily a non-functional TOE)
- Randomised internal execution

Countermeasures of the software (taken from the countermeasure list):

- Redundancy
- random delay

160 The attack potential calculation is then as is shown in the table below⁴:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the de-processing and initial testing to identify the failure combined with demonstrating the vulnerability takes more than 1 month to find the right combination of trigger and location (assuming the restricted command information is available). We assume exploitation based on a description of the attack and the commands to use.	> 1 month (5)	< 1 week (4)
Expertise	To find the right combination of trigger and	Expert	Proficient

⁴ Please keep in mind that these values are given for what is considered an average product. For especially good or bad implementations, the rating might be different. Nonetheless, the rating gives a baseline.

	location the attacker needs expert knowledge. He has to analyse the power traces and define the pattern recognition. A Proficient rating is required for exploitation, because of the de-processing techniques and equipment operation required.	(5)	(2)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 samples (0)	< 10 samples (0)
Open Sample / Known Key	Regarding time spent in identification, it is assumed that open samples are not used	NA	NA
Equipment	Minimum requirement is equipment to deprocess the chip, bond out the pads, and then to generate and analyse the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		15	10
Total		25	

161 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

162 As an alternative, open samples are used which results in the following table:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the deprocessing and initial testing to identify the failure combined with demonstrating the vulnerability takes less than 1 month using open samples to find the right combination of trigger and location (assuming the restricted command information is available). We assume exploitation based on a description of the attack and the commands to use.	< 1 month (3)	< 1 week (4)
Expertise	To find the right combination of trigger and location the attacker needs expert knowledge. He has to analyse the power traces and define the pattern recognition. A Proficient rating is required for exploitation, because of the deprocessing techniques and equipment operation required.	Expert (5)	Proficient (2)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the	Sensitive (4)	Public (0)

	command is assumed to be scripted for the exploitation phase (hence Public).		
Access to TOE	More than one sample may be necessary, but less than ten.	<10 samples (0)	< 10 samples (0)
Open Sample / Known Key	It is assumed that open samples are managed accordingly.	Sensitive (4)	NA
Equipment	Minimum requirement is equipment to deprocess the chip, bond out the pads, and then to generate and analyse the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		19	10
Total		29	

163 As well, this TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4). Applying the rule that minimum rating has to be taken, the final rating would be 25 points and (VAN.4).

2.3.7.2.1 Required demonstration of attack to justify this rating

164 It is not enough for this attack to show that a perturbation can be caused: A substantial part of the real memory content has to be dumped.

165 There are additional possible countermeasures at various levels that affect the attack potential calculation. Examples are:

- Additional hardware detectors that may detect some sorts of corruptions induced by the perturbations during the identification stage
- Perturbation-resilient software, which takes account of possible corruptions of data or program flow and reacts appropriately if these are detected (in this case it is important to consider the nature of the perturbation effect, and hence whether the software checks can themselves be defeated by perturbations)
- Perturbation-response in software, which may shut down the TOE if a threshold of hardware detection events (or possibly software detection events) is reached. This may make the attack require more samples (if more than 10 samples are required then the attack potential increases).

166 Some TOEs may make it difficult to synchronise attacks reliably (e.g. where perturbation-resilient software means there are only small time windows in which an attack can be applied), and in these cases the weakest path for an attack may need to consider a higher level of TOE knowledge (e.g. software source code) rather than assuming a higher Expertise and longer Elapsed time.

167 In the case of a pure hardware evaluation, application guidelines have to explain what countermeasures the software can take to achieve a higher rating.

2.3.8 Example: Accessing forbidden file

2.3.8.1 Attack path

168 Let us define a product implementing a file system structure with some access conditions to each file. Some of the files have an ‘*always*’ condition (meaning no protections, file is freely accessible), some have a ‘*never*’ condition (no access). We will consider that the software implements a *read* primitive allowing trying to access any file, but checking that the access conditions are satisfied before returning the content of the file.

169 The attack consists in accessing a forbidden file by applying a light perturbation during the read operation. The objective is to modify the execution of the program by “skipping” or “inverting” the tests on the access conditions.

170 This example uses the same approach as the previous one, and the same remarks on the IC protections apply.

171 The attack involves the following steps:

- 1) *Prepare the sample by providing laser access to the silicon. Depending on the characteristics of the light source (laser, flashgun, wavelength, power) and on the IC characteristics (protection, sensitivity, light sensors) this step could be very simple (removing packaging) or more complex, which could include following actions: remove the IC from its packaging, de-processing down to the passivation layer (this may use chemical etch) or lay open the substrate of the IC (this may be done by still milling the protecting material or even thinning the substrate). The laser light can be applied from the top, the lateral or the back of the IC.*
- 2) *Connect the IC to a test harness that repeatedly sends output request commands to the IC.*
- 3) Monitor the power traces during the executions of commands. Traces of access to authorized files and of (unsuccessful) access attempts to forbidden files are recorded and analysed. Simple implementations will show a common sequence at the beginning of the command and a difference after the tests. The divergence time is probably the place of the testing. The strategy is then to apply the light perturbation when the test is executed. The attacker will also use the traces to try to find synchronization events.
- 4) Run access attempts to a forbidden file, repeatedly applying the laser at different locations (in the absence of any other countermeasures it is likely that the edges of memory blocks are good targets, but other areas in the Glue Logic will be equally useful) while operating the IC and monitoring the results.
- 5) Laser pulses at different combinations of trigger and location have to be applied and the output checked.
- 6) If the attack succeeds, a ‘no error’ response is returned with the forbidden data available.

2.3.8.2 Example Rating 1 with Countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Light sensors

- 172 The HW countermeasure “light sensors” is required in this example as otherwise a simple flashgun may be used for mounting the attack. But even then, it is not too difficult to cover parts of the TOE IC with opaque enamel paint and leave only the memory blocks open. This is assumed for the exploitation phase.
- 173 It should be noted that as long as the memories from which the code is being executed are vulnerable to attacks, the countermeasure “randomized location of components” (taken from the countermeasure list) cannot be considered effective as often the memory blocks will be tried first by the attacker.
- 174 The basic attack potential calculation is then as is shown in the table below:

Factor	Comment	Ident’n	Exploit’n
Elapsed Time	Sample preparation, trace recording and analysis, sensitive area exploration	< 1 week (2)	< 1 day (3)
Expertise	Finding the attack path and tuning the parameters require a Proficient level only. Once defined and described, a proficient level is, in most cases, sufficient to reproduce the attack.	Proficient (2)	Proficient (2)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Minimum requirement is equipment to deprocess the chip, bond out the pads, and then to generate and analyse the required commands to drive the IC, a simple laser, an optical microscope and a digital oscilloscope. However, for the exploitation phase a simple triggerable flashgun and opaque enamel paint may also be ok.	Specialized (3)	Specialized (4)
Sub Total		9	9
Total		18	

- 175 This TOE would be resistant for this attack to an attacker with basic attack potential (VAN.2) with CC v3.
- 176 It should be noted that in this example it is not possible to achieve a High rating even when adding all sorts of SW countermeasures, as the lack of any HW fault detection for the memories means that code cannot be executed reliably. (Yet, even when the code fetch from memory is protected, there may still be “easy” attacks targeting, e.g., registers in the glue logic.) For the same reason it is also not possible to operate a fault counter reliably. But even if that were the case, a fault counter would only add to the number of samples required in the Identification phase (+3 points max, but likely to be

less – this depends on the number of light sensors). No further points can be awarded in the Exploitation phase, since by then the precise location of the light sensors are known and thus no further samples need to be wasted. It should also be considered that light sensors may be made visible by reverse engineering techniques (e.g., from the back side), in which case they do not really add to the number of samples required. A fault counter may slightly increase the time in the Identification phase, but not exceeding a month. Hence, the maximum rating that could be achieved this way are 23 points only.

2.3.8.2.1 Required demonstration of attack to justify this rating

- 177 The attack has to be successfully demonstrated by accessing any forbidden file.
- 178 There are additional possible countermeasures at various levels that can affect the attack potential calculation. Examples are:
- IC characteristics (packaging, shielding, active counter-measures – e.g. light or error detectors with reactions that stop or disable the TOE) could make the attack more difficult, increasing time (moving to more than 1 week for identification, more than 1 day for exploitation) or the number of samples (more than 10 for identification).
 - Perturbation-resilient software masking, and multiple redundant checks of the critical test may prevent successful attacks. If the countermeasure is efficient, the attack will probably be ‘not practical’ rather than simply increasing the attack potential. However, knowledge of the source code (Knowledge of the TOE rated Critical) could help the attacker in finding either a better synchronization event or an efficient attack path (when to apply perturbation(s)).

2.3.8.3 Example Rating 2 with countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Randomised location of components
 - Light sensors
 - Hardware fault detection for memories (that are effective in detecting when code is not read correctly from memory, leading to, e.g., a reset, but not necessarily a non-functional TOE)
 - Randomised internal execution
- 179 The countermeasure “hardware fault detection for memories” is required as otherwise focusing on the memories would be an easier attack and the randomised location of components would not prevent this attack.
- 180 If this attack is possible, the rating could be as follows:

Factor	Comment	Ident’n	Exploit’n
Elapsed Time	Sample preparation, trace recording and analysis, sensitive area exploration	> 1 month (5)	< 1 week (4)
Expertise	Finding the attack path and tuning the parameters probably require an expert level. Once defined and described, a proficient level is, in most cases, sufficient	Expert (5)	Proficient (2)

	to reproduce the attack.		
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Minimum requirement is equipment to prepare the chip, a laser, an optical microscope and a digital oscilloscope.	Specialized (3)	Specialized (4)
Sub Total		15	10
Total		25	

181 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

182 As an alternative, open samples are used with results in the following table:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Sample preparation, trace recording and analysis, sensitive area exploration	< 1 month (3)	< 1 week (4)
Expertise	Finding the attack path and tuning the parameters probably require an expert level. Once defined and described, a proficient level is, in most cases, sufficient to reproduce the attack.	Expert (5)	Proficient (2)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	It is assumed that open samples are managed appropriately.	Sensitive (4)	NA
Equipment	Minimum requirement is equipment to prepare the chip, a laser, an optical microscope and a digital oscilloscope.	Specialized (3)	Specialized (4)
Sub Total		17	10
Total		27	

183 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4). Applying the rule that minimum rating has to be taken the final rating would be 25 points and (VAN.4).

2.3.8.3.1 Required demonstration of attack to justify this rating

184 The attack has to be successfully demonstrated by accessing any forbidden file.

2.3.9 Example: PIN brute force using PIN UNBLOCK

2.3.9.1 Attack path

- 185 This attack requires a sophisticated laser manipulation set-up that is capable of using advanced triggering of the perturbation as well as prevention of write actions to non-volatile memory.
- 186 Let us define a card product implementing a PIN UNBLOCK command, accompanied by a MAC as access condition. The card will calculate the correct MAC using several DES operations and then verifies the MAC value in a conditional branch. The DES operations are clearly visible in the power consumption profile of the card and can be used as reference for the triggering of the perturbation. As a countermeasure against perturbation, the integrity of the conditional branch is monitored using a perturbation flag. Before the actual PIN UNBLOCK action is performed (resetting the PTC to three), the perturbation flag is set.
- 187 The attack consists of an automated sequence of testing two PIN values followed by a PIN UNBLOCK until the PIN is found.
- 188 During the execution of the PIN UNBLOCK command the program flow in the card is changed using laser manipulation to skip the checking of the correct MAC. However, in 95% of the attempts, the changed program flow is detected. In case the card tries to write the perturbation flag to non-volatile memory, this action is identified by an advanced trigger option in the digital oscilloscope. This trigger will initiate a power cut to the card in a very short time, preventing the completion of the write action and hence the card from blocking irreversibly.
- 189 After reset, the manipulation of the PIN UNBLOCK can be repeated until the command is successful completed, allowing the attacker to test another three PIN values. This attack can be repeated over and over again.
- 190 For this example, knowledge of the TOE software makes a significant difference to the time taken to find a suitable point (after the three DES operations) at which to apply the perturbation. Without knowledge of the TOE, many samples will be irreversibly blocked during the identification of the attack in terms of where and when to apply the perturbation.
- 191 The attack involves the following steps:
- 1) *Prepare the sample by providing laser access to the silicon. Depending on the characteristics of the light source (laser, flashgun, wavelength, power) and on the IC characteristics (protection, sensitivity, light sensors) this step could be very simple (removing packaging) or more complex, which could include following actions: remove the IC from its packaging, de-processing down to the passivation layer (this may use chemical etch) or lay open the substrate of the IC (this may be done by still milling the protecting material or even thinning the substrate). The laser light can be applied from the top, the lateral or the back of the IC.*
 - 2) *Connect the IC to a test harness that repeatedly sends PIN UNBLOCK commands to the IC.*

- 3) Monitor the power traces during the execution of PIN UNBLOCK commands. The strategy is then to apply the light perturbation when the MAC comparison is executed, i.e. after the DES operations.
- 4) Run the PIN UNBLOCK command repeatedly, applying the laser at different locations (probably registers in the Glue Logic) and timing while operating the IC and monitoring the results.
- 5) Laser pulses at different combinations of trigger and location have to be applied and the output checked.
- 6) If the attack on the PIN UNBLOCK succeeds, three additional attempts to brute force the PIN value are available.

2.3.9.2 Example Rating with countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Randomised location of components
- Light sensors
- Randomised internal clock execution
- Secure messaging for integrity (MAC-ing)
- Randomised internal execution
- Secure branching against execution perturbation and fault attacks
- Blocking card mechanism
- Irreversible card termination

192 The hardware countermeasures “randomised location of components”, “Light Sensors” and “Randomized internal clock execution” are only delaying the identification of the attack.

193 If this attack is possible, the rating could be as follows:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Sample preparation, trace recording and analysis, sensitive area exploration	> 1 month (5)	< 1 week (4)
Expertise	Finding the attack path and tuning the parameters probably require an expert level. Because of the complexity of the combination of equipment, an Expert is needed to reproduce the attack.	Expert (5)	Expert (4)
Knowledge of TOE	The attack requires Sensitive information to identify exploitable commands and the blocking mechanism, but the command is assumed to be scripted for the exploitation phase (hence Public).	Sensitive (4)	Public (0)
Access to TOE	Because of the Irreversible card termination countermeasure, more than 1 sample will be necessary, especially during Identification.	< 10 samples (0)	< 10 samples (0)

Open Samples / Known Key	Not required	NA	NA
Equipment	This attack requires the equipment to prepare the chip, a laser, an optical microscope and 2 digital oscilloscopes are needed. Additionally dedicated software needs to be developed to automate the attack and advanced triggering options are needed to trigger the laser and simultaneously prevent the setting of the perturbation flag during the attack. In both the identification phase and the exploitation phase the same equipment is needed.	Specialized (3)	Specialized (4)
Sub Total		17	12
Total		29	

194 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

2.3.9.2.1 Required demonstration of attack to justify this rating

195 The attack has to be successfully demonstrated by retrieving the PIN value of one sample.

2.3.10 Example: perturbation / cryptographic attack on 3DES

196 This section gives an attack potential assessment example for an attack path that combines two attack steps, where both steps are completely different in terms of means, expertise and approach. The example considers two attacks A and B where the results of attack A are the inputs for attack B.

197 Defining a general rule for this situation, such as adding both attack potentials (for A and B) seems neither obvious nor accurate. This section introduces an approach to the problem of combining the attack potentials based on this example.

2.3.10.1 Attack path

2.3.10.1.1 Description

198 The objective of the attack is to reduce the triple-DES (3DES) algorithm by skipping the execution of the last single DES. This allows the attacker in a second step to brute force the two single DES keys successively.

199 Let us assume that an embedded device allows one to encrypt a 64-bit plaintext block P by applying the 3DES algorithm using two 56-bit DES keys K1 and K2, i.e.:

$$C3 = \text{DES}_{K1}(\text{DES}_{K2}^{-1}(\text{DES}_{K1}(P)))$$

where:

- P is the known input message for the 3DES algorithm
- C3 is the known output of the 3DES algorithm

- K1 and K2 are the keys used for 3DES computation
- $\text{DES}_{K1}(\cdot)$ means DES encryption with key K1
- $\text{DES}_{K2}^{-1}(\cdot)$ means DES decryption with key K2

In the following, we also denote by C1 the output of the first single DES (i.e. $C1 = \text{DES}_{K1}(P)$) and C2 the output of the second single DES (i.e. $C2 = \text{DES}_{K2}^{-1}(C1)$).

- 200 The attack consists in applying a perturbation cancel the last single DES execution, allowing us to obtain the value of C2. At this step, the attacker thus knows the pair (C2, C3) which is made of a plaintext block and the related cipher text encrypted using DES algorithm and key K1.
- 201 Then, the key K1 can be recovered by using a single DES brute force that checks all the 2^{56} possible values⁵.
- 202 Since K1 is revealed, we can compute $C1 = \text{DES}_{K1}(P)$. As $C2 = \text{DES}_{K2}^{-1}(C1)$ and since C2 is known, we obtain another pair of plaintext/cipher text (C1, C2) for the key K2. Then, a second single DES brute force attack allows us to obtain K2.
- 203 Therefore, this attack allows us to retrieve the full 3DES key.
- 204 The attacker needs to realise the perturbation at the right time and location to cancel the last DES execution and get the correct result from the previous DES execution.
- 205 Other techniques may be required to determine best location and time in order to perform the perturbation like analysing the power consumption to determine when cryptographic computation occurs.

2.3.10.1.2 Characteristic of Attack Path

Perturbation part:

- 206 The real difficulty of this attack is to cancel the last DES execution and get the correct output of the previous DES operations, depending on the countermeasure implemented.

Brute force attack part:

- 207 Several recent examples illustrates the possibility to brute force a DES in few days (less than three) with a reasonable cost for designing the appropriate DES cracker with GPGPU. GPGPU, FPGA and DES cracker rainbow table solutions are included in this example. In addition it is possible to perform two simple DES brute force attacks in less than one week using such specialized equipment.

Rating methodology:

- 208 In the following, we rate such an attack in two different contexts:
- 1) The first one corresponds to the use of Sample with known key (i.e. the attacker knows the key during the Identification phase)
 - 2) The second one assumes that no Sample with known key is used.
- 209 For each of these contexts, we firstly present the attack potential for the two partial attacks (i.e. fault attack and brute force), before rating the full attack path accordingly.
- 210 According to Common Criteria part 1, chapter 1, paragraph f), the assessment of the

⁵ In the case of DES, i.e. with a 56-bit key and blocks of 64 bits, the probability that there is more than one key that encrypts a given plaintext into a fixed cipher text is non-negligible. More precisely, following a statistical analysis based on the assumption that DES behaves like a family of 2^{56} random permutations, we can prove that, given a pair of plaintext/cipher text, the expected number of keys found by a brute force attack is about 1,004. This is related to other probabilities that can be easily evaluated; the probability to obtain only one key, the correct one, is 99.6% and the probability to find more than three keys is about 10^{-8} . Therefore, the DES cracker should find one key in most of the cases, maybe two keys but never more in practice. Nevertheless, whatever the number of found keys by the DES Cracker may be, the complexity of the brute force attack remains the same. Then verifying which key is the good one, if needed, is not considered as a critical part of the attack as it should be neither time consuming nor difficult.

cryptographic strength of an algorithm is out of the scope and not covered by the CC. The certification scheme under which the CC is applied must make provision for such an assessment. At present no harmonized approach of the certification schemes exists for the assessment of the remaining strengths of a crypto algorithm after an attack. Hence the (brute force) attack potential assessment mentioned below is a rating example and might not be agreed by all CC schemes.

- 211 Moreover, we assume in the rest of this section that the attacks are performed on an implementation without any countermeasure.

2.3.10.2 Example rating with Samples with known secret

- 212 In this section we assume that the attacker has access to ‘restricted’ Sample with known key.

2.3.10.2.1 Attack part Perturbation

Factor	Comment	Ident’n	Exploit’n
Elapsed Time	We assume deprocessing and initial testing to identify the failure take less than 1 week, and that demonstrating the exploitation takes less than one further day (assuming the restricted command information is available). We assume exploitation based on a written description of the attack and the commands to use.	< 1 week (2)	< 1 day (3)
Expertise	Finding the attack path and tuning the parameters require an expert level. Once defined and described, a proficient level is, in most cases, sufficient to reproduce the attack.	Expert (5)	Proficient (2)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	We assume that Samples with known secret delivery are managed appropriately and that no Sample is available on the open market (i.e. Specific decision and action is required to release the secret key information). A known key is used in the identification phase to check that the last DES has been skipped successfully.	Restricted (2)	NA
Equipment	Minimum requirement is equipment to de-process the chip, and then to generate and analyse the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)

Sub Total		14	9
Total		23	

- 213 Depending on the complexity of the countermeasure implemented in the product, this attack potential can be increased with the following parameters:
- Knowledge of the TOE can be “sensitive”, if the knowledge of the type of countermeasure and IC features are required (+2).
- 214 Depending on the 3DES implementation and similar Samples access or availability, the rating of “Open Samples / Known Key” can be adjusted in accordance to section 4.8.7 of [JIL-AP]:
- If the access to the Sample with known key is classified “sensitive”, this attack potential is increased (+2).
 - If similar device with identical vulnerability is available (i.e. Java Card platform buyable on the open market), the lowest rating will apply. “Public” or “restricted” can be considered.

2.3.10.2.2 Attack part DES Cracker

- 215 When using samples with a known secret, brute force attack with a DES Cracker is not performed during the Identification phase. Hence the effort is not rated at Identification level.
- 216 Using specialized equipment the brute force attacks can be done in less than one week and we obtain the following rating.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	It takes less than one week to perform DES brute force operations.	NA(0)	< 1 week (4)
Expertise	The expertise required to perform a brute-force attack on DES has to be considered as "layman" because usage is simple.	NA (0)	Layman (0)
Knowledge of TOE	There is no TOE (only a pair of plaintext and cipher text is required). Therefore, this parameter is not applicable.	NA (0)	NA (0)
Access to TOE	There is no TOE (only a pair of plaintext and cipher text is required). Therefore, this parameter is not applicable.	NA (0)	NA (0)
Open Samples / Known Key	Although this attack part is subsequently to the perturbation part it is evaluated separately. Therefore, the points for samples with known secret are added despite the same samples, as used for the perturbation part, are used. In the final combination for the complete attack it can only be counted once as it is a non-destructive attack.	Restricted (2)	NA
Equipment	The equipment required to perform a brute-force attack on DES can be considered as specialized when using dedicated brute force equipment (rainbow table, FPGA or GPU boards)	NA(0)	Specialized (3)

Sub Total		2	7
Total		9	

2.3.10.2.3 Applying rules from [JIL-AP] document for combination

217 The following example applies the rules from the [JIL-AP] document without any discussion.

218 If we do not consider the DES cracker in the Identification and use specialized services for the DES cracker attack part, we obtain the following JIL rating:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Perturbation	< 1 week (2)	< 1 day (3)
	DES Cracker	NA	< 1 week (4)
	Total (addition⁶)	< 1 week (2)	< 1 week (4)
Expertise	Perturbation	Expert (5)	Proficient (2)
	DES Cracker	NA	Layman (0)
	Total (max)	Expert (5)	Proficient (2)
Knowledge of TOE	Perturbation	Restricted (2)	Public (0)
	DES Cracker	NA (0)	NA (0)
	Total	Restricted (2)	Public (0)
Access to TOE	Perturbation	< 10 (0)	< 10 (0)
	DES Cracker	NA (0)	NA (0)
	Total (addition)	< 10 (0)	< 10 (0)
Open Samples / Known Key	Perturbation	Restricted (2)	NA
	DES Cracker	NA	NA
	Total (addition)	Restricted (2)	NA
Equipment	Perturbation	Specialized (3)	Specialized (4)
	DES Cracker	NA (0)	Specialized (4)
	Total (max)	Specialized (3)	Bespoke (6)
Sub Total		14	12
Total		26	

219 Following [JIL-AP], the rating of the combined attack is leading to bespoke equipment in Exploitation phase:

“If clearly different test benches consisting of specialised equipment are required for distinct steps of an attack this shall be rated as bespoke. Test benches for side-channel and fault attacks are normally considered to be too similar and not different enough.”

220 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

221 With the perturbation attack potential, we have the following flexibility, depending on the complexity of the countermeasures:

- Various countermeasures can be added that will extend the time required for the attack.
- Expertise during Exploitation can be “Expert” if reproducing the attack is complex due to the countermeasures (+2).

⁶ For the „elapsed time“, the days are added, and not the values. For example, “<1 week” + “<1 week” results in “<1 month”. Then we take the value for “<1 month”.

2.3.10.3 Example rating without Samples with known secret

- 222 The main difference compared to the rating with Open Sample is the fact that in such a case, the attacker might perform the brute force at the identification level since the key is unknown.

2.3.10.3.1 Attack part Perturbation

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume deprocessing and initial testing to identify the failure take less than 1 week, and that demonstrating the exploitation takes less than one further day (assuming the restricted command information is available). We assume exploitation based on a written description of the attack and the commands to use.	< 1 week (2)	< 1 day (3)
Expertise	Finding the attack path and tuning the parameters require an expert level. Once defined and described, a proficient level is, in most cases, sufficient to reproduce the attack.	Expert (5)	Proficient (2)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	None	NA	NA
Equipment	Minimum requirement is equipment to de-process the chip, and then to generate and analyze the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		12	9
Total		21	

- 223 Depending on the complexity of the countermeasure implemented in the product, this attack potential can be increased with the following parameters:

- Knowledge of the TOE can be “Sensitive”, if the knowledge of the type of countermeasure and IC features is required (+2).

2.3.10.3.2 Attack part DES Cracker

- 224 When not using Samples with known secret, the brute force attack with a DES Cracker is performed during Identification phase and the corresponding effort is rated.

- 225 We assume in the following that the attacker is able to identify the effectiveness (success) of the fault injection in order to get the right faulted cipher-text C2 value amongst all the faulted values that have been logged. A simple case is that thanks to

side-channel trace observation of the execution the attacker identifies directly on the trace the effectiveness (success) of the fault injection. For instance the TDES part of the execution is not present on the side-channel trace. Such side-channel detection might be automated, for instance thanks to pattern recognition.

226 Using DES cracker specialized service, we obtain the following rating table:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	The attacker can identify success of a fault attack with side-channel trace observation and therewith having useful faulty cipher-text. Therefore DES brute force analysis is performed once in the identification to assert the validity of the attack and in the exploitation for key disclosure.	<1 week (2)	< 1 week (4)
Expertise	The expertise required to perform a brute-force attack on DES is be considered as "Layman"	Layman (0)	Layman (0)
Knowledge of TOE	There is no TOE (only a pair of plaintext and cipher text is required). Therefore, this parameter is not applicable.	NA (0)	NA (0)
Access to TOE	There is no TOE (only a pair of plaintext and cipher text is required). Therefore, this parameter is not applicable.	NA (0)	NA (0)
Open Samples	None	NA	NA
Equipment	The equipment required to perform a brute-force attack on DES is considered as specialized	Specialized (3)	Specialized (4)
Sub Total		5	8
Total		13	

227 Depending on the use of 3DES in the cryptographic protocol, one or two brute force attacks will be necessary to lead to the DES keys disclosure.

- If the protocol allows getting different faults for the same input and key, it is possible to carry out one brute force from the faulty ciphertext C2 to C3 to expose the K1 (as described in Section 2.3.10.1.1).
- If the protocol does not allow identical ciphertext to be computed, therefore two DES brute-force attacks will be required to identify and to exploit the attack.

Following [JIL-AP], the rating of the attack on these two protocols would be equivalent as the brute force attack will take "> one day" and "< one week".

2.3.10.3.3 Applying rules from [JIL-AP] document for combination

228 The following example applies the rules from the [JIL-AP] document without any discussion.

229 Using DES cracker specialized service, we obtain the following rating table:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Perturbation	< 1 week (2)	< 1 day (3)
	DES Cracker	< 1 week (2)	< 1 week (4)

	Total (addition⁷) In total more than one week is need to combine the fault and the brute force at identification level.	< 1 month (3)	< 1 week (4)
Expertise	Perturbation	Expert (5)	Proficient (2)
	DES Cracker	Layman (0)	Layman (0)
	Total (max)	Expert (5)	Proficient (2)
Knowledge of TOE	Perturbation	Restricted (2)	Public (0)
	DES Cracker	NA (0)	NA (0)
	Total	Restricted (2)	Public (0)
Access to TOE	Perturbation	< 10 (0)	< 10 (0)
	DES Cracker	NA (0)	NA (0)
	Total (addition)	< 10 (0)	< 10 (0)
Open Samples / Known Key	Perturbation	NA	NA
	DES Cracker	NA	NA
	Total (addition)	NA	NA
Equipment	Perturbation	Specialized (3)	Specialized (4)
	DES Cracker	Specialized (3)	Specialized (4)
	Total (max)	Bespoke (5)	Bespoke (6)
Sub Total		15	12
Total		27	

230 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

231 With the perturbation attack potential, we have the following flexibility, depending on the complexity of the countermeasures and attacker observations:

- It might be different if on the side-channel trace observation of the execution the attacker were not able to identify directly on the trace the effectiveness (success) of the fault injection. It might then require more brute force and the time at identification and exploitation level might become higher to one month. For instance, if the attacker succeeds in skipping the last single DES execution in 10 shots and that the cryptographic protocol does not allow identical ciphertext to be computed, 2*10 single DES brute forces effort would need to be rated.
- Various countermeasures can be added that will extend the time required for the attack.
- Expertise during Exploitation can be “Expert” if reproducing the attack is complex due to the countermeasures (+2).

2.3.10.4 Required demonstration of attack to justify this rating

232 The attack will be considered successful if the evaluation facility is able to get a pair of plaintext/cipher text, required to realise the brute force attack.

233 To be sure that the attack could be successful, (i.e. that the pair of plaintext/cipher text

⁷ For the „elapsed time“, the days are added, and not the values. For example, “<1 week” + “<1 week” results in “<1 month”. Then we take the value for “<1 month”.

obtained is a correct pair), the use of samples with known keys is required. In such a case, the brute force is not performed and the corresponding effort is not rated.

2.3.11 Example: Register localization and modification

2.3.11.1 Attack path

- 234 The objective is to localize Special Function Registers (CPU registers, Coprocessor registers, RNG registers ...) and control them by modifying their content at bit level precision. For that purpose, a laser with sufficient energy and with a high resolution is required.
- 235 The attack then involves the following steps:
- 1) Loading the register with a known value;
 - 2) Apply a laser perturbation;
 - 3) Read the content of the register, compare it with its initial value and notify if there is a change;
 - 4) Repeat 1, 2, 3, scanning the whole glue logic;
 - 5) Deliver cartography of the IC, with the identification of the effect (bit or register value set to 1 or to 0).
- 236 With a good laser resolution, it is possible to modify each bit of each register whatever its initial value.
- 237 Following this characterization, different attack paths can be built depending on the application (Safe-error, DFA, RNG “frozen” to 0 or 1, part of the DES key registers stuck to 0 or to 1 to facilitate the brute force attack, 3DES reduction ...).

2.3.11.2 Example rating 1 without countermeasure

- 238 Countermeasures of the IC (taken from the countermeasure list):
- Randomised location of components
 - Light sensors
 - Hardware ECC for memories
 - Randomised internal execution
- 239 The attack potential calculation is given in the table below, and is provided on the example of freezing an RNG:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Sample preparation, trace recording and analysis, sensitive area exploration, then freezing the RNG output.	< 1 month (3)	< 1 week (4)
Expertise	Building the automated test bench requires an expert level. Reproducing the attack when the RNG register coordinates are known requires also an expert level, due to the difficulty of taking into account all the countermeasures.	Expert (5)	Expert (4)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples	It is assumed that open samples are managed accordingly.	Sensitive (4)	Public (0)
Equipment	Minimum requirement is: Sample preparation	Specialized	Specialized

	equipment (to perform a non-destructive package opening for access to the die surface). A laser, optical microscope and a motorized stage system with software enabling to generate an automated cartography (today, this constitutes a typical laser test bench).	(3)	(4)
Sub Total		17	12
Total		29	

240 This TOE would be resistant for this attack to an attacker with moderate potential (VAN.4).

2.3.11.2.1 Required demonstration of attack to justify this rating:

241 To evidence the IC sensitivity, it has to be demonstrated that the RNG, or more generally the value of a bit of register can be controlled by laser (value forced to 1 or 0).
 242 For a full attack on a real product, it is as usual. For example, in the key reduction, the key reduction has to be successful without detection. Then the brute force part has to be combined, as explained in the example 2.3.9 (Perturbation / cryptographic attack on 3DES).

2.3.12 Example: Double Perturbation attack on AES engine

2.3.12.1 Attack path

243 This attack requires a sophisticated laser manipulation bench set-up with the following features:

- A laser [1] source
- An advanced triggering on power consumption
- A specific module to prevent write actions in non-volatile memory.

244 Let us define a card product implementing an AES calculation command. The card will calculate the correct AES result by using of HW-AES accelerator. The AES operations are clearly visible in the power consumption profile of the card and can be used as reference for the triggering of the perturbation.

245 A lot of software countermeasure against perturbation attack are implemented:

- The AES computation is computed a second time to check that no error has been induced
- In case of fault attacks a perturbation flag will be set leading to a NVM write which will subsequently block the card functionalities.
- Different countermeasure to single attack [2]

246 During the command including the AES execution two laser perturbations are possible targeting on the same way both AES computation. If both faulty outcomes are identical, the subsequent comparison will not detect the fault and the faulty cipher text will be released by the card. This could be exploited by the mean of a DFA attack which could lead to a partial or a full secret key disclosure.

247 However, the second perturbation having a small success rate any discrepancy between both computations is likely to be detected.

- 248 In this case the card tries to write the perturbation flag to non-volatile memory. This action is identified by an advanced trigger feature of the digital oscilloscope. This trigger will initiate a power cut to the card in a very short time, preventing the completion of the write action and hence the card from blocking irreversibly.
- 249 For this attack, a disturbance has to be located inside of HW-AES accelerator chip layout and an additional disturbance has to be located on the same point but inducing the same error on a second AES execution. In this case one laser source is required with a relatively high frequency to allow multiple shot distant of a couple of microseconds. All these attacks can be repeated over and over again. The attack is successful if a DFA with the wrong results of AES calculation is possible.
- 250 The attack involves the following steps:
- Prepare the sample by providing laser access to the silicon. Depending on the characteristics of the light source (laser, flashgun, wavelength, power) and on the IC characteristics (protection, sensitivity, light sensors) this step could be very simple (removing packaging) or more complex, which could include following actions: remove the IC from its packaging, de-processing down to the passivation layer (this may use chemical etch) or lay open the substrate of the IC (this may be done by still milling the protecting material or even thinning the substrate). The laser light can be applied from the top, the lateral or the back of the IC.
 - Connect the IC to a test harness that repeatedly sends AES commands to the IC.
 - Monitor the power traces during the execution of AES commands. The strategy is then to apply the light perturbation when the first AES calculation is repeatedly being executed, i.e. among the latest AES rounds.
 - Because a fault will be detected by the DFA verification, the attacker has to find the right setting to trig and avoid the blocking mechanism of being fully executed. The success rate must remain very high to avoid too many cards being lost during the attack.
 - Monitor and identify the second AES execution with the aim to disturb it in the same way than the 1st one. If both AES outcome are identical it will be possible to get faulty cipher texts.
 - In a latest stage, the 1st perturbation targeting the AES execution must be fine-tuned in order to generate exploitable faulty AES cipher texts for a further DFA.

2.3.12.2 Example Rating with countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Randomised location of components (hardware)
 - Light sensors (hardware)
 - Randomised internal clock execution (hardware)
 - Blocking card mechanism (software)
- 251 The hardware countermeasures “randomised location of components”, “Light Sensors” and “Randomized internal clock execution”, are only delaying the identification of the attack.

252 **1. Double Fault Attack on the HW-AES accelerator**

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Sample preparation, trace recording and analysis, sensitive area exploration and AES implementation and countermeasures analysis	< 1 month (3)	< 1 week (4)
Expertise	Finding the attack path and tuning the parameters probably require an expert level. Depending on the complexity of the combination of equipment, an Expert may be needed to reproduce the attack.	Expert (5)	Expert (4)
Knowledge of TOE	The attack does not require particular information about the OS, as the card behaviour can be deduced from the observation. However this has an impact in the elapsed attack time. The attack is assumed to be scripted in the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	Because of the Irreversible card termination countermeasure, more than 10 samples will be necessary, especially during Identification.	<30 samples (1)	<10 samples (0)
Open Samples / Known Key	Not required in this example	NA	NA
Equipment	This attack requires the equipment to prepare the chip; one laser , an optical microscope and digital oscilloscopes are needed. Additionally dedicated software needs to be developed to automate the attack and advanced triggering options are needed to trigger the laser and simultaneously prevent the setting of the perturbation flag during the attack. In both the identification phase and the exploitation phase the same equipment is needed.	Specialized (3)	Specialized (4)
Sub Total		14	12
Total		26	

2.3.12.2.1 Required demonstration of attack to justify this rating

253 The attack has to be successfully demonstrated by retrieving the AES key by DFA.

[1] Multiple laser are laser sources (e.g. laser diode, laser cutter) which can be placed on different place in chip design and can generate disturbances by minimum of two different times.

[2] Single attacks are attacks which are occurs only one time in AES calculation command

Countermeasures of the IC (taken from the countermeasure list):

- Randomised location of components (hardware)
- Light sensors (hardware)
- Randomised internal clock execution (hardware)
- **Randomised internal execution (software)**
- Secure branching against execution perturbation and fault attacks (software)
- Blocking card mechanism (software)

The hardware countermeasures “randomised location of components”, “Light Sensors” and “Randomized internal clock execution”, are only delaying the identification of the attack.

254 **1. Double Fault Attack on the HW-AES accelerator**

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Sample preparation, trace recording and analysis, sensitive area exploration and AES implementation and countermeasures analysis	< 1 month (3)	< 1 week (4)
Expertise	Finding the attack path and tuning the parameters probably require an expert level. Depending on the complexity of the combination of equipment, an Expert may be needed to reproduce the attack.	Expert (5)	Expert (4)
Knowledge of TOE	The attack does not require particular information about the OS, as the card behaviour can be deduced from the observation. However this has an impact in the elapsed attack time. The attack is assumed to be scripted in the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	Because of the Irreversible card termination countermeasure, more than 10 samples will be necessary, especially during Identification.	<30 samples (1)	<10 samples (0)
Open Samples / Known Key	Not required in this example	NA	NA
Equipment	This attack requires the equipment to prepare the chip; one laser , an optical microscope and digital oscilloscopes are needed. Additionally dedicated software needs to be developed to automate the attack and advanced triggering options are needed to trigger the laser and simultaneously prevent the setting of the perturbation flag during the attack.	Specialized (3)	Specialized (4)

	In both identification and exploitation phases the same equipment is needed.		
Sub Total		14	12
Total		26	

2.3.12.2.2 Required demonstration of attack to justify this rating

255 The attack has to be successfully demonstrated by retrieving the AES key by DFA.

256 It is noted that the elapsed time in exploitation may turn out to be more than 1 week, depending on the level of time required for setting up the different steps of the attack. On the other hand, it is worth noting that additional or more complex countermeasures may increase the efforts required of the attack and lead to a product resistant to a high attack potential.

[1] Multiple laser are laser sources (e.g. laser diode, laser cutter) which can be placed on different place in chip design and can generate disturbances by minimum of two different times.

[2] Single attacks are attacks which are occurs only one time in AES calculation command

2.3.13 Example: Double Perturbation attack on AES engine and CPU

2.3.13.1 Attack path

257 This attack requires a sophisticated laser manipulation bench set-up with the following features:

- A multiple laser [1] sources which can be placed on different locations in chip design
- An advanced triggering on power consumption
- A specific module to prevent write actions in non-volatile memory.

258 Let us define a card product implementing an AES calculation command. The card will calculate the correct AES result by using of HW-AES accelerator. The AES operations are clearly visible in the power consumption profile of the card and can be used as reference for the triggering of the perturbation.

259 A lot of software countermeasure against perturbation attack are implemented:

- checking of results by multiple calculations
- In case of fault attacks a perturbation flag will be set leading to a NVM write which will subsequently block the card functionalities.
- Different countermeasure to single attack [2]

260 During the command including the AES execution two kinds of laser perturbation are possible:

- **The AES calculation by firing on the chip area where the HW-AES accelerator is located can be disturbed** and lead to exploitable faulty results.
- **The CPU processing can be disturbed** by the mean of laser manipulation in the chip area corresponding to the CPU or where code instructions are fetched. This could be

exploited by skipping the DFA verification of the previous AES computation.

- 261 However, the second perturbation having a small success rate a fault induced during the AES calculation or the DFA verification will likely be detected.
- 262 In this case the card tries to write the perturbation flag to non-volatile memory. This action is identified by an advanced trigger feature of the digital oscilloscope. This trigger will initiate a power cut to the card in a very short time, preventing the completion of the write action and hence the card from blocking irreversibly.
- 263 **Attack the HW-AES accelerator and the CPU** at different times. One disturbance has to be located inside of HW-AES accelerator chip layout and an additional disturbance has to be located inside of CPU chip layout. In this case two laser sources which are located at different locations in chip layout and time are required. After reset, the manipulation of one AES calculation inside of HW-AES accelerator can be disturbed by a first disturbance and generate a faulty result. A second disturbance, which is located inside of CPU chip layout, attacks the software countermeasure which detects a faulty calculation in the HW-AES accelerator. All these attacks can be repeated over and over again. The attack is successful if a DFA with the wrong results of AES calculation is possible.
- 264 The attack involves the following steps:
- Prepare the sample by providing laser access to the silicon. Depending on the characteristics of the light source (laser, flashgun, wavelength, power) and on the IC characteristics (protection, sensitivity, light sensors) this step could be very simple (removing packaging) or more complex, which could include following actions: remove the IC from its packaging, de-processing down to the passivation layer (this may use chemical etch) or lay open the substrate of the IC (this may be done by still milling the protecting material or even thinning the substrate). The laser light can be applied from the top, the lateral or the back of the IC.
 - Connect the IC to a test harness that repeatedly sends AES commands to the IC.
 - Monitor the power traces during the execution of AES commands. The strategy is then to apply the light perturbation when the AES calculation is repeatedly being executed, i.e. among the latest AES rounds.
 - Because a fault will be detected by the DFA verification, the attacker has to find the right setting to trig and avoid the blocking mechanism of being fully executed. The success rate must remain very high to avoid too many cards being lost during the attack.
 - Different laser parameters must be tested to disturb the DFA verification with the aim to force the verification to be true. The 2nd perturbation is subject to a successful 1st perturbation generating a faulty AES outcome. For this reason the identification may be time consuming. The difficulty of synchronising the second laser pulse depends on the possibility to be well synchronised. This may need some additional synchronisation capability in case of random executions.
 - In a latest stage, the 1st perturbation targeting the AES execution must be fine-tuned in order to generate exploitable faulty AES cipher texts for a further DFA.

2.3.13.2 Example Rating with a first level of countermeasures

- Randomised location of components (hardware)
- Light sensors (hardware)
- Blocking card mechanism (software)

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	• Sample preparation, • Trace recording and analysis, • Sensitive area exploration of AES, observation, implementation analysis, attack tuning for , • Sensitive area exploration of blocking mechanism Identification: > 1month, due to the difficulty to characterize multiple sensitive areas on the chip	> 1 month (5)	< 1 week (4)
Expertise	Finding the attack path and tuning the parameters probably require an expert. Because of the complexity of the combination of equipment, an Expert is needed to reproduce the attack.	Expert (5)	Expert (4)
Knowledge of TOE	The attack does not require particular information about the OS, as the card behaviour can be deduced from the observation. However this has an impact in the elapsed attack time. The attack is assumed to be scripted in the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	Because of the Irreversible card termination countermeasure, more than 10 samples will be necessary, especially during Identification.	<30 samples (1)	<10 samples (0)
Open Samples / Known Key	Not required in this example	NA	NA
Equipment	This attack requires the equipment to prepare the chip; two lasers , an optical microscope and digital oscilloscopes are needed. Additionally dedicated software needs to be developed to automate the attack and advanced triggering options are needed to trigger the laser and simultaneously prevent the setting of the perturbation flag during the attack. In both identification and exploitation phases the same equipment is needed.	Specialized (3+1)	Specialized (4+1)
Sub Total		17	13

Total		30
--------------	--	-----------

2.3.13.2.1 Required demonstration of attack to justify this rating

265 The attack has to be successfully demonstrated by retrieving the AES key by DFA.

[1] Multiple laser are laser sources (e.g. laser diode, laser cutter) which can be placed on different place in chip design and can generate disturbances by minimum of two different times.

[2] Single attacks are attacks which are occurs only one time in AES calculation command

2.3.13.3 Example Rating with a second level of countermeasures

- Randomised location of components (hardware)
- Light sensors (hardware)
- Randomised internal execution (hardware)
- **Randomised internal execution (software)**
- Blocking card mechanism (software)

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	• Sample preparation, • Trace recording and analysis, • Sensitive area exploration of blocking mechanism • Sensitive area exploration for disrupting AES computations • Tuning of the fault injection to induce a faulty cipher text in spite of multiple execution and random processing. • Observation, AES implementation analysis, attack tuning, collection, characterisation and processing of valid faulty cipher texts More than 1 month for the identification and more than 1 week for the exploitation, justified by the difficulty to tune the attack requiring multiple steps.	> 1 month (5)	< 1 week (4)
Expertise	Finding the attack path and tuning the parameters probably require an expert. Because of the complexity of the combination of equipment, an Expert is needed to reproduce the attack in exploitation.	Expert (5)	Expert (4)
Knowledge of TOE	The attack does not require particular information about the OS, as the card	Restricted (2)	Public (0)

	behaviour can be deduced from the observation. However this has an impact in the elapsed attack time. The attack is assumed to be scripted in the exploitation phase (hence Public).		
Access to TOE	Because of the Irreversible card termination countermeasure, more than 10 samples will be necessary, especially during Identification.	<30 samples (1)	<10 samples (0)
Open Samples / Known Key	Not required in this example	NA	NA
Equipment	This attack requires the equipment to prepare the chip; two lasers , an optical microscope and digital oscilloscopes are needed. Additionally dedicated software needs to be developed to automate the attack and advanced triggering options are needed to trigger the laser and simultaneously prevent the setting of the perturbation flag during the attack. In both identification and exploitation phases the same equipment is needed.	Specialized (3+1)	Specialized (4+1)
Sub Total		17	13
Total		30	

2.3.13.3.1 Required demonstration of attack to justify this rating

266 The attack has to be successfully demonstrated by retrieving the AES key by DFA.

267 It is noted that the elapsed time in exploitation may turn out to be more than 1 week, depending on the level of time required for setting up the different steps of the attack. On the other hand, it is worth noting that additional or more complex countermeasures may increase the efforts required of the attack and lead to a product resistant to a high attack potential.

[1] Multiple laser are laser sources (e.g. laser diode, laser cutter) which can be placed on different place in chip design and can generate disturbances by minimum of two different times.

[2] Single attacks are attacks which are occurs only one time in AES calculation command

2.4 Retrieving keys with FA

268 This type of attack aims at retrieving secret keys from the TOE. Its principle consists in disturbing the TOE during a cryptographic computation, then collecting and analysing the corresponding faulty outputs. Three examples are described; two concerning two classical DFA attacks affecting the DES and the RSA algorithms, and another concerning a perturbation attack on 3DES combined with a brute force cryptographic attack on 3DES.

2.4.1 Description of Attack

269 DFA is an acronym for Differential Fault Analysis. By using DFA, an attacker intends to obtain information about a secret key by analysing the difference between a correct and a faulty cryptographic output, both computations using the same input.

270 DFA requires analysing faulty outputs. Such faulty output could be obtained by inducing a physical perturbation on the device during the corresponded cryptographic computation, or eventually during the algorithm parameters manipulation. Such perturbation can be created by either non-invasive (power glitching for instance) or semi invasive (laser typically) techniques.

271 According to the theory behind this attack, the fault injected during the device processing should fulfil specific requirements to lead to an exploitable output. These requirements are mainly based on both a precise synchronisation and the expected value as a consequence of the perturbation. A lack of accuracy in these requirements can render the analysis to recover the key much more complex.

272 From a practical point of view, the process to mount such an attack can then be divided into the following stages:

- Searching for a suitable fault injection method
- Depending on the cryptographic algorithm to attack, setting up a more or less accurate synchronisation technique.
- Inducing fault(s) during the device's execution and then collecting the corresponding faulty cipher texts
- Analysing the differences between the faulty cipher texts with the correct cipher text (or eventually the plain text).

2.4.2 Effect of Attack

273 The direct effect of such attack is the disclosure of a part or of the whole secret key. A partial key disclosure can eventually be completed by a brute force attack. According to the number of remaining bits to retrieve, the elapsed time to complete the attack will be more or less affected.

274 If recovering the remaining key bits represents a too big effort in term of time processing, a second attack targeting another part of the key could eventually be setup.

- 275 In any way, even partial, the secret key disclosure affects directly the security of the TOE.

2.4.3 Impact on TOE

- 276 This attack can be carried out with more or less invasive manner. The non-invasive method (power-glitching) avoids any direct physical damages. On the other hand the use of laser technique requires to physically preparing the TOE allowing the silicon illumination.
- 277 DFA can threaten most of the cryptographic keys used by the TOE, leading potentially to the disclosure of DES, 3DES or RSA keys for example.
- 278 As for any perturbation attack, the TOE can lose fully or partially its functionality, due to either a too strong physical perturbation or a software reaction to an anomaly detected.

2.4.4 Characteristics of the Attack

- 279 For the first part faulty cipher texts need to be generated. For techniques and hurdles how to achieve such faulty cipher texts we refer to section 2.3. Next, the faulty cipher texts need to be analysed requiring specific software tools. These software tools can implement various statistical treatments, specific for each algorithm.
- 280 The software tools may be standard or specialized, and the expertise level of the attacker as well, depending on how the attack theory is known by the cryptographic community or how sophisticated the software is.
- 281 The knowledge of the keys, by using for instance samples with known secrets, can strongly improve the attack efficiency while no brute force attack is required to check the validity of the attack. The time spent to complete the attack can then be affected in accordance.
- 282 The countermeasures associated could also improve the complexity of the attack and then strongly affect all these ratings. For example hardware sensors or filters may prevent or avoid against physical perturbation in first place, by detecting or removing them. Circumventing such sensors, if they are well designed and implemented, may not be an easy task and can increase substantially the complexity of an attack or even renders any error induction impossible (see section 2.2).
- 283 A classical logical countermeasure consists in computing twice a part or the full algorithm, and then to compare the coherence between results from both calculation. If an error is detected, the faulty cipher text is not returned and then the attack typically becomes useless.
- 284 Some other countermeasures are specific to algorithms, and consist for instance in using their mathematical structure.
- 285 While most of the DFA attacks need several faulty cipher texts generating from the same input, it is reasonable to claim that the use of global keys, i.e. a key of which the value is the same on a set of cards, allows performing an attack using several samples and then ease the attacker investigations.
- 286 In case no countermeasure is implemented against fault injection devices risk facing to a straightforward attack even without any TOE knowledge.

2.4.5 Example: Perturbation attack combined with brute force

287 This section gives a first example for an attack path that combines two attack steps, where both steps are completely different in terms of means, expertise and approach as required for a DFA.

2.4.5.1 Attack path

288 The objective of the attack is to perturb the execution of a cryptographic algorithm or protocol to obtain a faulted encryption of a known plaintext. Then we consider the brute force attack. The brute force attack is independent from the attack of getting a pair of plaintext/cipher text.

289 This attack can break cryptographic key systems, allowing retrieving the full keys.

290 The attacker needs to realise the perturbation at the right time and location to get an exploitable faulted result from the cryptographic computation.

291 Next, so after a perturbed cipher text is retrieved, the key can be revealed by brute force.

292 A number of different approaches can be used for the brute force attack. Several solutions are presented here for the DES and 3DES.

- DES cracker⁸, hardware solution, with a cost of 250 k\$ in 1998. Record 56h. This solution won the « DES challenge II⁹»,
- GPGPU based DES cracker, allowing retrieval of the cryptographic key in a number of days.
- Network solution¹⁰: software + network of 1000 PCs. Estimation: for 1000 PCs running at 3GHz, it takes 20 days to break a DES.
- An intermediate solution is to use reconfigurable devices. Jean-Jacques Quisquater and Francois-Xavier Standaert have done the latest estimation during SHARCS¹¹ ECRYPT¹² workshop in their paper¹³ “Exhaustive Key Search of the DES: Updates and Refinements”. Their conclusion is:

“These predictions already suggest that the use of recent (low cost) reconfigurable devices can be relevant for exhaustive key search attacks against block ciphers. As a practical estimation, a US\$ 12 000 machine could break DES in about 3 days, presently or in the near future. Although these estimations do not take the development costs into account and must therefore be relativized, they clearly underline that the cost of a DES cracker is presently reachable for most organizations, including universities.”

- As of 2008, another recent FPGA based cryptographic cipher cracking project is COPACOBANA (Professor Paar of the Ruhr University of Bochum). Their system is available and costs \$ 9.000 to build, with the average time to crack a 56 bits DES key being 6 days.

293 Note that according to Common Criteria part 1, chapter 1, paragraph f), the assessment

⁸ See http://www.eff.org/Privacy/Crypto/Crypto_misc/DESCracker

⁹ <http://www.rsasecurity.com/rsalabs/>

¹⁰ see <http://www.distributed.net> for instance

¹¹ <http://www.ruhr-uni-bochum.de/itsc/tanja/SHARCS>

¹² <http://www.ecrypt.eu.org>

¹³ http://www.ruhr-uni-bochum.de/itsc/tanja/SHARCS/talks/JJQ_FXS.pdf

of the cryptographic strength of an algorithm is out of the scope and not covered by the CC. The certification scheme under which the CC is applied must make provision for such an assessment. At present no harmonized approach of the certification schemes exists for the assessment of the remaining strengths of a crypto algorithm after an attack. Hence the (brute force) attack potential rating should be discussed with the CC schemes.

2.4.5.2 Example: Perturbation attack and brute force on 3DES

- 294 The attack is described in section 2.3.10 Example: perturbation / cryptographic attack on 3DES. Different brute force solutions have been considered in this attack rating. Following [JIL-AP] document, only the GPGPU based DES cracker attack rating is reported since it is leading to the lowest attack potential.
- 295 Example ratings of the attack part with a GPGPU based DES cracker are given in sections 2.3.10.2.2 and 2.3.10.3.2 considering two cases:
- 1) Brute force with samples with known secret
 - 2) Brute force without samples with known secret.

2.4.6 Example: DFA attack on (3)DES / AES

2.4.6.1 Attack Path

- 296 The attack on (3)DES described in the following was first published by Biham and Shamir in 1996 in [1].
- 297 To apply a DFA on DES, the attacker needs to collect couples of a correct and a faulty cipher text generated with the same input data. From this, it comes obvious that the attack described in [1] is not applicable to every cryptographic protocol, e.g. in case only partial results are available, or the inputs cannot be chosen. However, note that the attack works for DES as well as 3DES.
- 298 Assuming an error is induced during the last rounds (typically the last half, see [6]), some parts of the last round key can be retrieved. To do so, the attacker compares the cipher text of the correct calculation with the one issuing from a faulty calculation. Every useful error allows the attacker to obtain information on the last round key. Commonly more than one useful couple cipher text/correct cipher text is required to find any part of the sub key.
- 299 When the entire last round key has been retrieved, 8 bits are still unknown. The attacker can easily brute-force these remaining bits. In the case of a triple DES (using either 16 or 24 byte keys), the brute force attack is not realistic regarding the time required to find so many remaining bits. The same kind of attack should be applied to retrieve the penultimate round key, and then retrieve some additional remaining bits. This iteration can be repeated until it is reasonable to find the remaining bits by brute force attack. Such iteration can dramatically increase the attack complexity.
- 300 Only a couple faulty cipher texts may be sufficient, typically from 2 to several dozens, to disclose a significant part of the secret key. Attacks using a single faulty cipher texts can also be found in literature, however as of now they are not applicable on 3DES, but only single DES (see for example [9]¹⁴). In case the attacker cannot inject the error in distinct rounds of the DES computation, i.e. in case precision of the fault is difficult, more faulty cipher text might be required, or the attack even becomes infeasible.

¹⁴ S. Guo, T. Wang, Z. Shi, et. al., Improved Algebraic Fault Analysis: A Case Study on Piccolo and Applications to Other Lightweight Block Ciphers, COSADE 2013

- 301 The DFA attack can also be applied to the first round key in exactly the same way. It only requires the attacker is able to decrypt the faulty ciphertexts by using the TOE. The same analysis is then applied to the corresponding decrypted faulty cipher texts. The knowledge of the key in that context can be exploited to forge a clone for instance.
- 302 Note that (3)DES is by far not the only symmetric cryptographic primitive that is vulnerable to DFA. As an example, AES is also known to be vulnerable against DFA if insufficient countermeasures are implemented, see e.g. [10]¹⁵ or [11]¹⁶.
- 303 Summarized, DFA on (3)DES or AES is two-folded: first a perturbation attack needs to be performed to get pairs of correct and fault cipher texts. For details, refer to the techniques introduced in section 2.3 of this document. Second, the pairs need to be analysed, e.g. as described above.

2.4.6.2 Example rating 1 without countermeasures

- 304 For the following example, we assume that the TOE does not implement any specific countermeasures to prevent a DFA on the (3)DES or AES implementation.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume initial testing to identify suitable test parameters take less than 1 week, and that demonstrating the exploitation takes less than one further day (assuming the restricted command information is available). We assume exploitation based on a written description of the attack and the commands to use.	< 1 week (2)	< 1 day (3)
Expertise	Generally speaking, finding the attack path and tuning the parameters requires a proficient level only. However, cryptanalysis of the faulty cipher texts and tailoring the attack parameter for such attack requires an Expert during identification. A proficient level is, in most cases, sufficient to reproduce the attack.	Expert (5)	Proficient (2)
Knowledge of TOE	By assumption, the attack requires Restricted information in this example to identify exploitable commands. The command is then considered Public as it is scripted for the exploitation phase.	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required in this example	NA	NA
Equipment	Minimum requirement is equipment to de-process the chip, bond out the pads, and then to generate and analyze the required commands to drive the IC. A laser and optical	Specialized (3)	Specialized (4)

¹⁵ P. Dusart, G. Letourneux, O. Vivolo, Differential Fault Analysis on AES

¹⁶ Sk Subidh Ali and Debdeep Mukhopadhyay, Differential Fault Analysis of AES-128 Key Schedule using a Single Multi-Byte Fault Authors, CARDIS 2011

	microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing. In both the identification phase and the exploitation phase the same equipment is needed.		
Sub Total		12	9
Total		21	

305 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3).

2.4.6.2.1 Required demonstration of attack to justify this rating

306 It has to be demonstrated that the DES or AES key can be retrieved by means of DFA. For 3DES it has to be demonstrated that the last key can be retrieved.

2.4.6.3 Example rating 2 with countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Redundancy.
- Randomised location of components (hardware)
- Light sensors (hardware)
- Randomised internal clock execution (hardware)
- Blocking card mechanism (software)

307 In this example rating we consider a TOE implementing above mentioned countermeasures to protect against DFA on (3)DES or AES. In more detail, let us define a card product implementing a (3)DES / an AES calculation command. The card will calculate the correct result by using of HW-AES accelerator. The (3)DES/AES operations are clearly visible in the power consumption profile of the card and can be used as reference for the triggering of the perturbation.

308 A lot of software countermeasure against perturbation attack are implemented:

- The (3)DES / AES computation is computed a second time to check that no error has been induced.
- In case of fault attacks a perturbation flag will be set leading to a NVM write which will subsequently block the card functionalities.
- Different countermeasure to single attack such as light sensors.

309 During the command including the (3)DES/AES execution two laser perturbations are possible targeting on the same way both (3)DES/AES computations. If both faulty outcomes are identical, the subsequent comparison will not detect the fault and the faulty cipher text will be released by the card. This could be exploited by the mean of a DFA attack which could lead to a partial or a full secret key disclosure.

310 However, the second perturbation having a small success rate any discrepancy between both computations is likely to be detected. In this case the card tries to write the perturbation flag to non-volatile memory. This action is identified by an advanced trigger feature of the digital oscilloscope. This trigger will initiate a power cut to the card in a very short time, preventing the completion of the write action and hence the

card from blocking irreversibly.

311 For this attack, a disturbance has to be located inside of HW-AES accelerator chip layout and an additional disturbance has to be located on the same point but inducing the same error on a second AES execution. In this case one laser source is required with a relatively high frequency to allow multiple shot distant of a couple of microseconds. It has to be assumed that a state-of-the-art test bench is capable of triggering the laser source in such short order.

312 Summarized, the attack involves the following steps:

- 1) Preparing the samples and connecting to test bench.
- 2) Monitor the power traces during the execution of (3)DES / AES commands. The strategy is then to apply the light perturbation when the first (3)DES / AES calculation is repeatedly being executed, i.e. among the latest (3)DES / AES rounds.
- 3) Because a fault will be detected by the DFA verification, the attacker has to find the right setting to avoid the blocking mechanism of being fully executed. The success rate must remain very high to avoid too many cards being lost during the attack.
- 4) Monitor and identify the second (3)DES / AES execution with the aim to disturb it in the same way than the 1st one. If both (3)DES / AES outcomes are identical it will be possible to get faulty cipher texts.
- 5) In a latest stage, the 1st perturbation targeting the (3)DES / AES execution must be fine-tuned in order to generate exploitable faulty (3)DES / AES cipher texts for a further DFA.

313 The attack potential calculation is then as is shown in the table below:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume initial testing to do sample preparation, trace recording and analysis, identify suitable test parameters and countermeasures analysis take less than 1 month. In contrast to the previous example without countermeasures, the exploitation takes less than a week due to the presents of countermeasures.	< 1 month (3)	< 1 week (4)
Expertise	Generally speaking, finding the attack path and tuning the parameters requires a proficient level only. However, cryptanalysis of the faulty cipher texts and tailoring the attack parameter for such attack requires an Expert during identification. In contrast to the previous example without countermeasures, an Expert is also needed for Exploitation due to the implemented countermeasures, especially the card blocking countermeasure.	Expert (5)	Expert (4)

Knowledge of TOE	By assumption, the attack requires Restricted information in this example to identify exploitable commands. The command is then considered Public as it is scripted for the exploitation phase.	Restricted (2)	Public (0)
Access to TOE	Because of the Irreversible card termination countermeasure, more than 10 samples will be necessary, especially during Identification.	<30 samples (1)	<10 samples (0)
Open Samples / Known Key	Not required in this example	NA	NA
Equipment	Similar to the previous example without countermeasures, minimum requirement is equipment to de-process the chip, bond out the pads, and then to generate and analyze the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing. Additionally, software needs to be developed to prevent the setting of the perturbation flag during the attack. In both the identification phase and the exploitation phase the same equipment is needed.	Specialized (3)	Specialized (4)
Sub Total		14	12
Total		26	

314 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4) with CC v3.

315 It is noted that the elapsed time in exploitation may turn out to be more than 1 week, depending on the level of time required for setting up the different steps of the attack. On the other hand, it is worth noting that additional or more complex countermeasures may increase the efforts required of the attack and lead to a product resistant to a high attack potential.

2.4.6.3.1 Required demonstration of attack to justify this rating

316 It has to be demonstrated that the DES or AES key can be retrieved by means of DFA. For 3DES it has to be demonstrated that the last key can be retrieved.

2.4.6.4 Example rating 3 with countermeasures

Countermeasures of the IC (taken from the countermeasure list):

- Redundancy.
- Randomised location of components (hardware)
- Light sensors (hardware)
- Randomised internal clock execution (hardware), Randomised internal execution (software)

- Blocking card mechanism (software)

317 The preconditions for this example as well as the implemented countermeasures are identical to those listed in the previous example (see section 2.4.6.3). However, in section 2.4.6.3 we considered firing on the same chip area where the HW accelerator is located twice in short order. Here, we consider laser manipulation in two distinct chip areas: first in the area where the HW-accelerator is located, and second in the chip area corresponding to the CPU or where code instructions are fetched. While the first perturbation attempts to compute a faulty cipher text in first place, the latter perturbation attempts skipping the DFA verification step.

318 Therefore, this attack requires a sophisticated laser manipulation bench allowing attacking several features of the IC, at the same or at different points in time, to eventually mount a successful attack. Such test benches are typically no longer considered Specialized, but not yet bespoke.

319 Summarized, the attack involves the following steps:

- 1) Preparing the samples and connecting to test bench.
- 2) Monitor the power traces during the execution of (3)DES / AES commands. The strategy is then to apply the light perturbation when the AES calculation is repeatedly being executed, i.e. among the latest (3)DES / AES rounds.
- 3) Because a fault will be detected by the DFA verification, the attacker has to find the right setting to avoid the blocking mechanism of being fully executed. The success rate must remain very high to avoid too many cards being lost during the attack.
- 4) Different laser parameters must be tested to disturb the DFA verification with the aim to force the verification to be true. The 2nd perturbation is subject to a successful 1st perturbation generating a faulty (3)DES / AES outcome. For this reason the identification may be time consuming. The difficulty of synchronising the second laser pulse depends on the possibility to be well synchronised. This may need some additional synchronisation capability in case of random executions.
- 5) In a latest stage, the 1st perturbation targeting the (3)DES / AES execution must be fine-tuned in order to generate exploitable faulty (3)DES / AES cipher texts for a further DFA.

320 The attack potential calculation is then as is shown in the table below:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	In contrast to the previous example (attacking the same area twice) identification take more than 1 month, while exploitation takes less than a week.	> 1 month (5)	< 1 week (4)
Expertise	Generally speaking, finding the attack path and tuning the parameters requires a proficient level only. However, cryptanalysis of the faulty cipher texts and tailoring the attack parameter for such attack requires an Expert during identification. In contrast to the previous	Expert (5)	Expert (4)

	example without countermeasures, an Expert is also needed for Exploitation due to the implemented countermeasures, especially the card blocking countermeasure.		
Knowledge of TOE	By assumption, the attack requires Restricted information in this example to identify exploitable commands. The command is then considered Public as it is scripted for the exploitation phase.	Restricted (2)	Public (0)
Access to TOE	Because of the Irreversible card termination countermeasure, more than 10 samples will be necessary, especially during Identification.	<30 samples (1)	<10 samples (0)
Open Samples / Known Key	Not required in this example	NA	NA
Equipment	In contrast to the previous example (attacking the same area twice) this attack requires an even more sophisticated test bench including two lasers . In both identification and exploitation phases the same equipment is needed.	Specialized (3+1)	Specialized (4+1)
Sub Total		17	13
Total		30	

321 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4) with CC v3.

322 It is noted that the elapsed time in exploitation may turn out to be more than 1 week, depending on the level of time required for setting up the different steps of the attack. On the other hand, it is worth noting that additional or more complex countermeasures may increase the efforts required of the attack and lead to a product resistant to a high attack potential.

2.4.6.4.1 Required demonstration of attack to justify this rating

323 It has to be demonstrated that the DES or AES key can be retrieved by means of DFA. For 3DES it has to be demonstrated that the last key can be retrieved.

2.4.7 Example: DFA attack on RSA

2.4.7.1 Attack Path

324 DFA attacks can potentially affect specific asymmetric cryptographic primitives such as ECDSA or RSA operations. They consist in exploiting one or several faulty signatures to disclose a part or the entire private key. In this example we will discuss some DFA attacks on RSA.

325 For instance, a “safe error attack” concerns the exponentiations implemented with a straightforward square and multiply always algorithm. The principle consists in inducing a consistent fault during a given multiply operation and then to simply compare the signature obtained with the expected one. Depending on the signature is

correct or not, the attacker can then deduce if the corresponding multiply is fake or not and then retrieve the corresponding key bit. It requires applying as many perturbations as the number of key bits with a relatively fine synchronisation and a good level of perturbation reproducibility. The attack efficiency can be improved in the particular case of standard RSA using short exponents, by exploiting the property described hereafter. Short exponent values are commonly used to ease the computation. However according to [2] some parts of the private key can be retrieved in that context among a limited number of possible value, by simply computing $(kN+1)/e$, k denoting a number such as $0 < k < e$, N the modulus and e the public exponent. The case public exponent equal to 3 is particularly strong, since half of the private key can be disclosed using $k=2$.

- 326 A second example concerns the perturbation of the public parameters, particularly the modulus N , which may lead to the disclosure of the private key.
- 327 The most famous DFA attack was published in 1996 (see [3]), and is known as the “Bellcore attack”.
- 328 To apply a DFA attack on a CRT RSA, the attacker launches the computation of a correct signature S and manages perturbing a wrong signature S' of the same message. One single useful faulty signature S' is enough to retrieve the secret key. Indeed the generation of a useful faulty signature S' consists in creating a fault only on S_p or S_q . To obtain one of the prime factors of the modulus p or q , the attacker computes the great common divider (gcd) of $S - S'$ and the modulus N . The knowledge of p or q leads obviously to the entire secret key.
- 329 A variant of this attack consists in computing the gcd of $S'^e - M$, M being the message and e the public exponent, and the modulus N . This attack has the advantage to not require any multiple computations.
- 330 Finally, some improvements have been introduced in [4] and [5], showing the latter can be extended using several faulty signatures even when the complete message is only partially known. This technique has shown its efficiency on ISO 9796-2 signature scheme used among others in EMV standard.

2.4.7.2 Example rating 1 without countermeasures

- 331 In this example we discuss a “Bellcore attack” on a RSA CRT implementation without dedicated countermeasures.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume initial testing to identify suitable test parameters take less than 1 week, and that demonstrating the exploitation takes less than one further day (assuming the restricted command information is available). We assume exploitation based on a written description of the attack and the commands to use.	< 1 week (2)	< 1 day (3)
Expertise	Generally speaking, finding the attack path and tuning the parameters requires a proficient level only. As cryptanalysis of the faulty signatures is requires computation of a GCD	Proficient (2)	Proficient (2)

	only, Expert level is not required in any phase.		
Knowledge of TOE	By assumption, the attack requires Restricted information in this example to identify exploitable commands. The command is then considered Public as it is scripted for the exploitation phase.	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Minimum requirement is equipment to de-process the chip, bond out the pads, and then to generate and analyze the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		9	9
Total		18	

332 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3) with CC v3.

2.4.7.2.1 Required demonstration of attack to justify this rating

333 It has to be demonstrated that the RSA key can be retrieved by means of DFA.

2.4.7.3 Example rating 2 without countermeasures

334 In this example we discuss a Safe-Error attack on a RSA implementation without dedicated countermeasures.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume initial testing to identify suitable test parameters take less than 1 week, and that demonstrating the exploitation takes less than one further day (assuming the restricted command information is available). We assume exploitation based on a written description of the attack and the commands to use. However, in contrast to the Bellcore attack, exploitation time scales with the key length. Typically for a 1024 bit-long key disclosure more than one day is needed.	< 1 week (2)	< 1 week (4)
Expertise	Generally speaking, finding the attack path and tuning the parameters requires a proficient level only. In practice, this attack requires also a certain delicacy which justifies the need of an expert for exploitation.	Expert (5)	Proficient (2)
Knowledge of TOE	By assumption, the attack requires Restricted information in this example to identify exploitable commands. The command is then	Restricted (2)	Public (0)

	considered Public as it is scripted for the exploitation phase.		
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Minimum requirement is equipment to de-process the chip, bond out the pads, and then to generate and analyze the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		12	10
Total		22	

335 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3) with CC v3.

2.4.7.3.1 Required demonstration of attack to justify this rating

336 It has to be demonstrated that the RSA key can be retrieved by means of DFA.

2.4.8 Further References

[1] E. Biham & A. Shamir. “*A New Cryptanalytic Attack on DES*”. Manuscript, 18 October 1996.

[2] Boney, “*Twenty years of Attack on the RSA Cryptosystem*”, Notice of the AMS volume 6.

[3] D. Boneh, R. DeMillo & R. Lipton. “*New Threat Model Breaks Crypto Codes*”. Bellcore Press Release, September 1996.

[4] J.-S. Coron, A. Joux, I. Kizhvatov, D. Naccache and P. Paillier. “*Fault Attacks on RSA Signatures with Partially Unknown Messages*”, Proceedings of CHES 2009, Incs, vol. 5747, Springer-Verlag, 2009, pp. 444-456. Full version: eprint.iacr.org/2009/309.

[5] J.-S. Coron, D. Naccache and M. Tibouchi. “*Fault Attacks Against EMV Signatures*”, eprint.iacr.org/2009/503.

[6] M. Rivain. “*Differential Fault Analysis on DES Middle Rounds*”. Proceedings of CHES 2009, Incs, vol. 5747, Springer-Verlag, 2009, pp. 457-469.

[7] Boneh, DeMillo & R. J. Lipton, “On the Importance of Checking Cryptographic Protocols for Faults”, proceedings of Eurocrypt 97

[9] S. Guo, T. Wang, Z. Shi, et. al., “Improved Algebraic Fault Analysis: A Case Study on Piccolo and Applications to Other Lightweight Block Ciphers”, COSADE 2013

[10] P. Dusart, G. Letourneux, O. Vivolo, “Differential Fault Analysis on AES”

- [11] S. Ali and D. Mukhopadhyay, “Differential Fault Analysis of AES-128 Key Schedule using a Single Multi-Byte Fault Authors”, CARDIS 2011
- [12] P. Loubet-Moundi, D. Vigilant and F. Oliver, “Static Fault Attack on Hardware DES Registers”, PHISIC 2011: International Workshop on Practical Hardware Innovations in Security Implementation and Characterisation, 2011
- [13] G. Piret and J.-J. Quisquater, “A Differential Fault Attack Technique against SPN Structures with Application to the AES and KHAZAD”
- [14] Henri Cohen, Gerhard Frey (Eds.), “Handbook of Elliptic and Hyperelliptic Curve Cryptography”, Chapter 29, Chapman & Hall/CRC, 2005
- [15] I. Biehl, B. Meyer, and V. Muller, “Differential fault attacks on elliptic curve cryptosystems”, In: Mihir Bellare (Ed.), CRYPTO, vol. 1880 of Lecture Notes in Computer Science, pp. 131–146. Springer, Berlin / Heidelberg, 2000
- [16] M. Ciet and M. Joye, “Elliptic curve cryptosystems in the presence of permanent and transient faults”, Des. Codes Cryptography, 36(1):33–43, 2005
- [17] *Fault Attack on Elliptic Curve with Montgomery Ladder Implementation*, Pierre-Alain Fouque, Reynald Lercier, Denis Real and Frederic Valette, in L. Breveglieri, S. Gueron, I. Koren, D. Naccache, and J.-P. Seifert (eds.), Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC), pp. 92–98. IEEE Computer Society, 2008
- [18] J. Blomer, M. Otto, and J.-P. Seifert, “Sign change fault attacks on elliptic curve cryptosystems”, In Luca Breveglieri, Israel Koren, David Naccache, and Jean-Pierre Seifert, editors, FDTC, Vol. 4236 of Lecture Notes in Computer Science, pp. 36–52. Springer, Berlin / Heidelberg, 2006
- [19] M. Joye, S. Yen, “The Montgomery Powering Ladder”, in: B.S. Kaliski Jr., Ç. K. Koç and C. Paar (Eds.), Cryptographic Hardware and Embedded Systems – CHES 2002, vol. 2523 of Lecture Notes in Computer Science, pp. 291–302, Springer, Berlin / Heidelberg, 2003
- [20] Fouque, Guillermin, et. al., “Attacking RSA-CRT Signatures with Faults on Montgomery Multiplication”, <http://eprint.iacr.org/2012/172.pdf>, 2013

2.5 Side-channel Attacks – Non-invasive retrieving of secret data

2.5.1 Description of Attack

- 337 Side-channel attacks target secret information leaked through unintentional channels in a concrete, i.e. physical, implementation of an algorithm. These channels are linked to physical effects such as timing characteristics, power consumption, or electromagnetic radiation.
- 338 SPA and DPA stand for ‘Simple’ and ‘Differential Power Analysis’, respectively, and aim at exploiting the information leaked through characteristic variations in the power

consumption of electronic components, usually without damaging the TOE. Although various levels of sophistication exist, the power consumption of a device can in essence be simply measured using a digital sampling oscilloscope and a resistor placed in series with the device.

- 339 When an IC is operating, each individual element will emit electromagnetic radiation in the same way as any other conductor with an electrical current flowing through it. Thus, as this current varies with the data being processed, so does the electromagnetic radiation emitted by the TOE. Electromagnetic Analysis (EMA) attacks target this variant of information leakage. These attacks are sometimes referred to as SEMA (Simple Electromagnetic Analysis), or DEMA (Differential Electromagnetic Analysis). They may use emissions from the whole IC (chip-EMA), or may focus on the emissions from particular areas of the die, where critical components are located (local-EMA).
- 340 Experimental evidence shows that electromagnetic data (particularly from localised areas of a die) can be rather different from power trace data, and ICs that are protected against power analysis may therefore be vulnerable to EMA.
- 341 For the sake of unity in what follows SPA and DPA will denote not only attacks based on measurements of the power consumption, but are understood to cover their “cousins” in electromagnetic attacks as well, unless stated otherwise. Other side channels, such as timing differences from cache hit/miss arising from speculative/out of order execution (for example Spectre [16] and Meltdown [17]), can also provide measurements useful for this type of attack.
- 342 Implementations that include countermeasures like Boolean masking that resist first order DPA may be vulnerable to higher-order DPA. This attack requires that the attacker is able to correlate more than one data point per TOE computation using hypotheses on intermediate states that depend on secret key parts.
- 343 The combined statistical analysis for higher-order DPA may be based on aligned measurements of the same side channel at different times or on aligned simultaneous measurements of different channels such as power consumption and electromagnetic radiation of the device during the computation.
- 344 The outcome of a side-channel attack may be as simple as finding a characteristic trigger point for launching other attacks (such as DFA), or as complete as the secret key used in a cryptographic operation. It can also aim at recovering other secret data such as PINs, or random numbers generated for use as secrets, or even the opcode of the code being executed on the TOE. Depending on the goal of the attack it may involve a wide range of methods from direct interpretation of the recorded signal to a complex analysis of the signal with statistical methods. In the latter case the initial filtering used for signal analysis will generally depend on the type of the measurement (i.e., power consumption or electromagnetic radiation), but the mathematics for retrieving the secret information eventually is largely the same.

2.5.2 Effect of Attack

- 345 At the heart of this attack lies the observation that – at least in the absence of any countermeasures – electronic devices consume different amounts of energy depending on their internal state and their activity. Typically, a read operation from EEPROM requires much less power than the corresponding write operation, and a co-processor consumes more power during a cryptographic calculation than in idle mode. Moreover, the power consumption is typically dependent on the instructions being performed and the data being operated upon. Consequently, each activity of an electronic device leaves a characteristic – although possibly faint – footprint in its power and electromagnetic

radiation signatures, which in the following it referred to as a “trace”. The art of side-channel attacks lies in devising physical, statistical, as well as mathematical models that recover the leaked information.

346 Both SPA and DPA attacks require detailed knowledge of the cryptographic algorithm to be analysed, and benefit strongly when implementation knowledge is available as well. SPA and DPA differ in that SPA is typically concerned with the analysis of characteristic patterns found in the traces, often involving more or less complex signal processing steps for filtering out the signal sought. SPA may or may not require multiple measurements, depending on the quality of the signal and the precise nature of the attack. Some countermeasures (such as, e.g., exponent blinding in RSA) only work when many traces need to be averaged over, and completely fail when the information leaked in a single trace is already enough to reveal the secret. Template attacks – which for the sake of this document are considered a special, more advanced variant of an SPA attack based on a statistical model – require an initial characterisation or profiling stage, after which the actual attack will be performed in a second stage. Ideally, the profiling stage will have to be performed only once on a single TOE. DPA, on the other hand, goes further in that it employs a statistical analysis to correlate the traces with the cryptographic algorithm to be attacked. Typically, this involves repeated measurements for random but known input or output data, and the verification or falsification of a suitably chosen hypothesis regarding a secret entity located somewhere towards the beginning or the end of the cryptographic algorithm. In some cases it is also possible for the attacker to choose the input or – with quite some difficulties and only indirectly so – the output data. In such a case a wider range of potential points of attack need to be considered in the algorithm. Often – and particularly so in the presence of countermeasures – considerable pre-processing SPA steps are required before the actual DPA attack can be launched.

347 A major advantage of an SPA or DPA attack lies in the fact that it may be conducted using a normal smart card interface without any physical modifications – except possibly for having to expose the die surface to allow for better EMA measurements. There is no way for the TOE to detect this attack, and thus it has to rely entirely on its built-in countermeasures that reduce the information leakage.

348 All side-channel measurements benefit from being carried out in an environment that reduces electrical noise (such as a Faraday cage).

349 Although power measurements can be as simple as adding a small resistor in line with the TOE, it is often advantageous to create a small test harness that decouples the electrical measurement from the operation of the device. In particular, it is beneficial to keep the voltage over the TOE constant, independent of the power drawn by the TOE (if only for avoiding voltage sensors to kick in).

350 Chip-EMA is typically carried out by placing an antenna near the surface of the TOE (either top or bottom side), and collecting the electromagnetic variations detected by the antenna. A special type of this analysis is used with contact-less smart cards, where variations in the power flow are measured, when the IC is in the electromagnetic field of the interface (i.e. reader) device.

351 Local-EMA is usually performed on an IC with packaging removed to expose the die surface. A small antenna coil is then stepped over the die area, just above the surface, with the aim of recording variations in the electromagnetic emissions from very small areas of the TOE. In principle, this might be used to collect the emissions from the targeted cryptographic coprocessor whilst avoiding, for example, the emissions from the CPU. Compared to collecting power traces, this may give a cleaner signal (i.e., with less noise from other parts of the TOE operation), or may minimise the effects of

- countermeasures such as deliberate noise generation. However, in practice there are limitations to the potential gain achievable with this approach, since the antenna coil cannot be made small enough.
- 352 Over the past few years technology has been steadily advancing in this field, with new antenna designs and the usage of multiple antennae.
- 353 Even when standard DPA fails, there might still be some information leakage present, due to, e.g., more or less subtle algorithmic dependencies of data at different points (and hence times) in the cryptographic algorithm. The method of choice to retrieve this information is higher-order DPA, which correlates information at two or more different points in time. This becomes particularly important when analysing countermeasures such as Boolean and arithmetic masking which may be effective against first-order DPA, but not necessarily against higher-order DPA. In order for it to be successful, it is of even higher importance to know – or rightly guess – implementation details (such as which type of blinding is applied, and where precisely) when performing a higher-order DPA attack.
- 354 Finally, it should be noted that in some cases physical modifications of the TOE (e.g., disabling of hardware countermeasures) may be used to improve the efficiency of a side-channel attack.

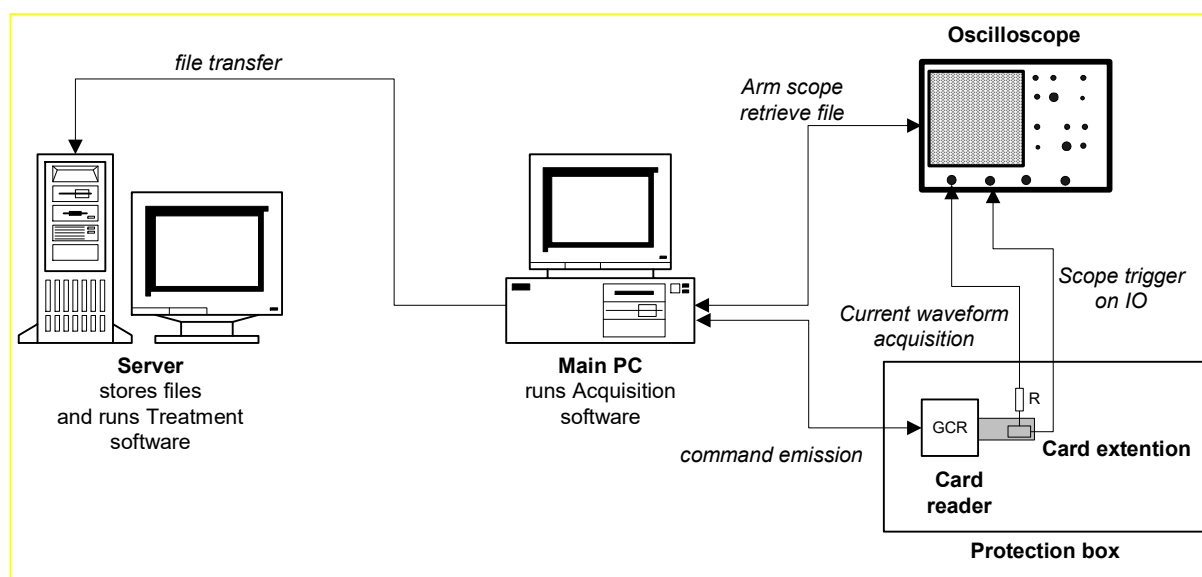
2.5.3 Impact on TOE

- 355 It lies in the very nature of SPA and (higher-order) DPA attacks that they may in principle be applied to any cryptographic algorithm – either stand-alone, e.g., for retrieving secret keys or PINs, or as part of a composite attack. Additionally, SPA may serve as a stepping stone for launching further attacks. For instance, SPA may be employed to detect a critical write operation to the EEPROM that needs to be intercepted. An SPA analysis may also be performed as part of a timing attack (e.g., in the square-and-multiply algorithm of RSA), or for deducing which branch of a conditional jump has been taken by the program flow. Or it could simply be used as a first step for identifying countermeasures to side-channel attacks that need to be overcome. Finally, an SPA attack could be employed to determine the proper trigger point for a subsequent glitch or light attack, or as an aid for localising a suitable time window for a physical probing attack.
- 356 A DPA (or template) attack does not need to be entirely successful for it to become dangerous. Given a suitable key search strategy that takes into account imperfect DPA results as discussed further below, it may be enough to retrieve only part of the secret key by DPA, and obtain the rest by brute-force methods.
- 357 Implementations that resist DPA attacks may still be vulnerable to higher-order DPA attacks since that type of attack is tapping additional information not considered in a standard attack. Of course, algorithms that are vulnerable to first-order DPA are vulnerable to higher-order DPA, too. It appears that higher-order DPA is particularly suited to deal with Boolean and arithmetic masking / blinding of symmetric algorithms. On the other hand, the extension of higher-order DPA to public key (asymmetric) algorithms seems to be very difficult, because of the widely applied blinding countermeasures that make use of algebraic transformations during the calculation that are completely different from ordinary masking.
- 358 Power analysis as well as EMA attacks may be carried out for a hardware TOE (an IC), or a software/composite TOE (an OS or application running on an IC). Some countermeasures may already exist in the hardware TOE, whilst others are added later in software. Thus, the way in which software uses the IC functions may make a critical

difference to its vulnerability to this type of attack.

2.5.4 Characteristics of the Attack

- 359 Both, SPA and DPA attacks require detailed knowledge of the cryptographic algorithm
to be analysed, and benefit strongly when implementation knowledge is available as
well.
- 360 The tasks involved with an SPA attack vary greatly depending on the nature of the
attack. Sometimes a simple low-pass filter may be enough for reducing the noise level
when all that is needed is a simple trigger flank for a subsequent DFA attack, whereas in
other cases very advanced signal-processing techniques and various profiling stages
may be required before any useful signal can be retrieved.
- 361 DPA attacks always need to be tailored to the cryptographic algorithm at hand. This is a
relatively straight-forward procedure for block ciphers such as DES and AES, where the
analysis of the secret key bits can be done in parallel as the S-boxes are independent of
each other, but could become more cumbersome for algorithms such as RSA and ECC,
where a sequential processing of the secret-key bits becomes necessary. For a DPA
attack to work, either the input to or the output from a cryptographic operation needs to
be known by the attacker. Moreover, these messages need to vary. It is not possible to
launch a DPA attack when the input (or output, as case may be) of a cryptographic
operation is always the same. Similarly, the attack does not work when the secret key is
being changed every so often. DPA attacks become more severe when the attacker has
the ability to choose the input data (chosen-plaintext attack) as then, for instance, inner
rounds of the DES algorithm may be attacked, or some mathematical blinding
countermeasures may be circumvented. In principle, it is also possible to perform a
chosen-ciphertext attack, i.e., where the output data is chosen, but in practice this
usually means the attacker has to vary the input data hoping to generate suitable output
data.
- 362 A DPA attack typically requires a large number of traces to be collected and analysed
(ranging from thousands to millions). The following scheme gives an example
description of a power trace acquisition and how to monitor the power consumption of a
smart card – the extension to measurements of the electromagnetic radiation is in
principle straight forward:



363 The 'Main PC' is used to run the acquisition software, and to send the command sequences to the smart card. The 'Server' is used to store measurement files and to run signal-processing software as well as other analysis tools.

364 The acquisition procedure consists of running the cryptographic algorithm n times, acquiring n associated power consumption traces during the process. Usually, the input messages vary randomly for each measurement, and may possibly even be more or less freely chosen by the attacker. It is, however, also possible to perform a DPA attack when only the output messages are known (and again are sufficiently random). In either case the input and / or output messages are stored in a database so that they are linked to the corresponding measurement.

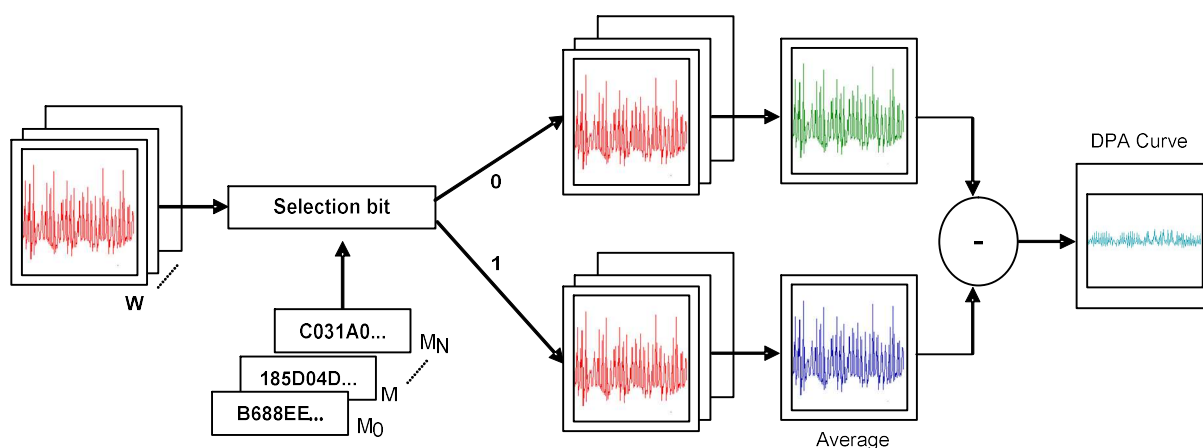
365 Before proceeding any further it may be necessary to compress the vast amount of data collected. Thereafter, a more or less substantial realignment and cleaning up of the power consumption traces is usually required in order to reduce the noise level. This may involve advanced signal-processing techniques, depending on the countermeasures being in place. The analysis of the average power consumption trace and its standard deviation can be used to get a first idea of the quality of this step, and where precisely within the power consumption traces the target might be found. In this context, it is rather helpful to analyse the correlation with respect to the known input (and / or output) messages to further localise the target.

366 At the heart of a DPA attack lies a so-called select function, which in essence is a correlation function between the measurement (represented by the tuples {plaintext, cipher text, power consumption and/or electromagnetic trace}) and a critical point (the so-called target bit) in the cryptographic algorithm under attack. This target bit is linked to a (preferably small) part of the secret key – which for the sake of clarity we will call secret sub-key – as well as either to the plaintext or the cipher text. The secret sub-key that enters the select function is obviously not known and needs to be guessed, and the number of guesses required for each secret sub-key determines the effectiveness of such an attack. The correct guess will have the strongest correlation signal should the DPA attack be successful. It should be noted that by construction of the attack, there is usually more than one secret sub-key to be found, and hence that in the final stage the entire secret key will need to be reconstructed from the set of secret sub-keys guessed.

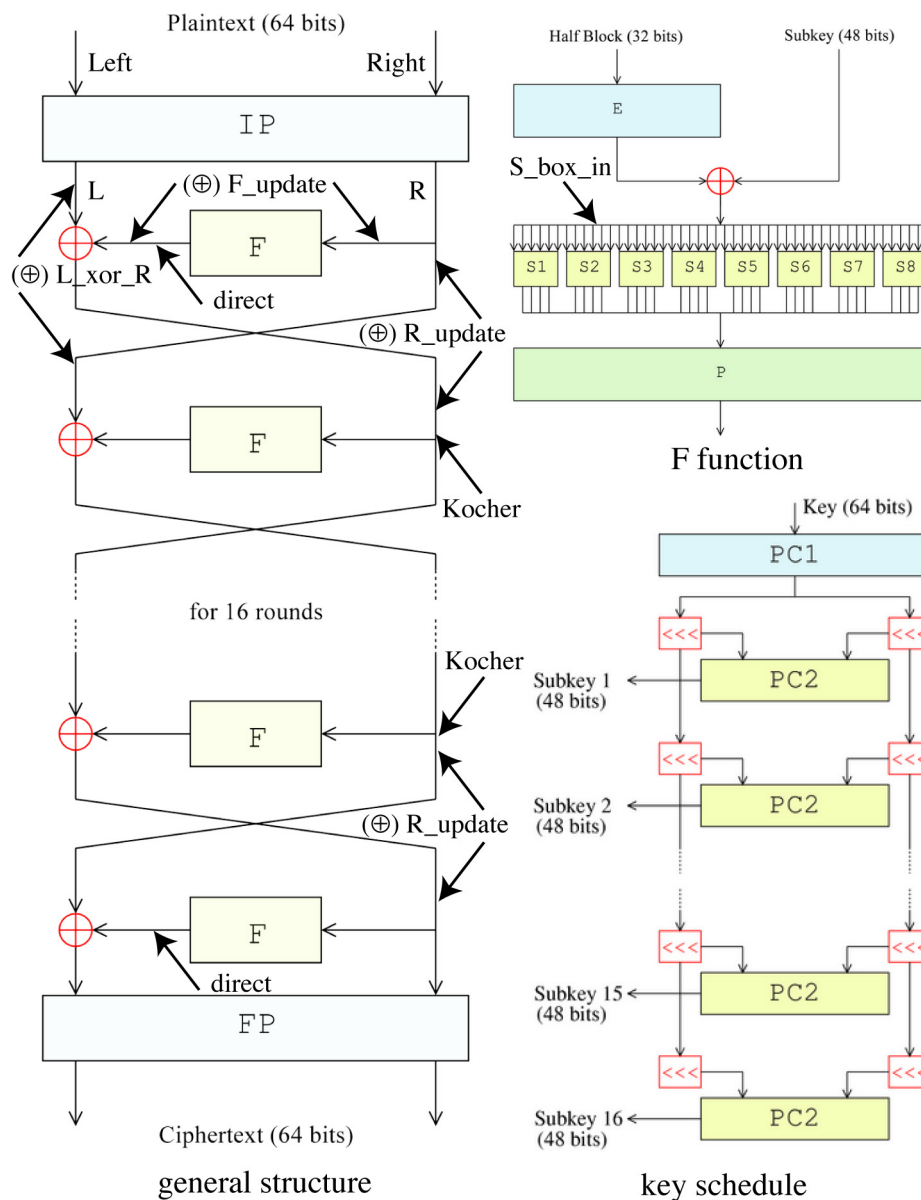
367 Various select functions exist, based e.g., on Hamming-Weight or Hamming-Weight-Distance models and more details can be found in the literature. It must be stressed,

however, that each and every cryptographic algorithm requires its own analysis in this respect – and a good choice for the select function is crucial for success. Generally speaking, the smaller the secret sub-keys are, and hence the smaller the number of possible guesses is for a target bit, the more susceptible a cryptographic algorithm tends to be to these types of attacks. Some standard select functions useful to attack the DES and AES algorithms are indicated in the Figs. below. Although most of these target critical points in either the first or last round of the respective algorithm, extensions thereof can be constructed when the plaintext (and sometimes the ciphertext) is controlled by the attacker. In that case even rounds 2, 3 and 4 (and likewise from the back) may be targeted.

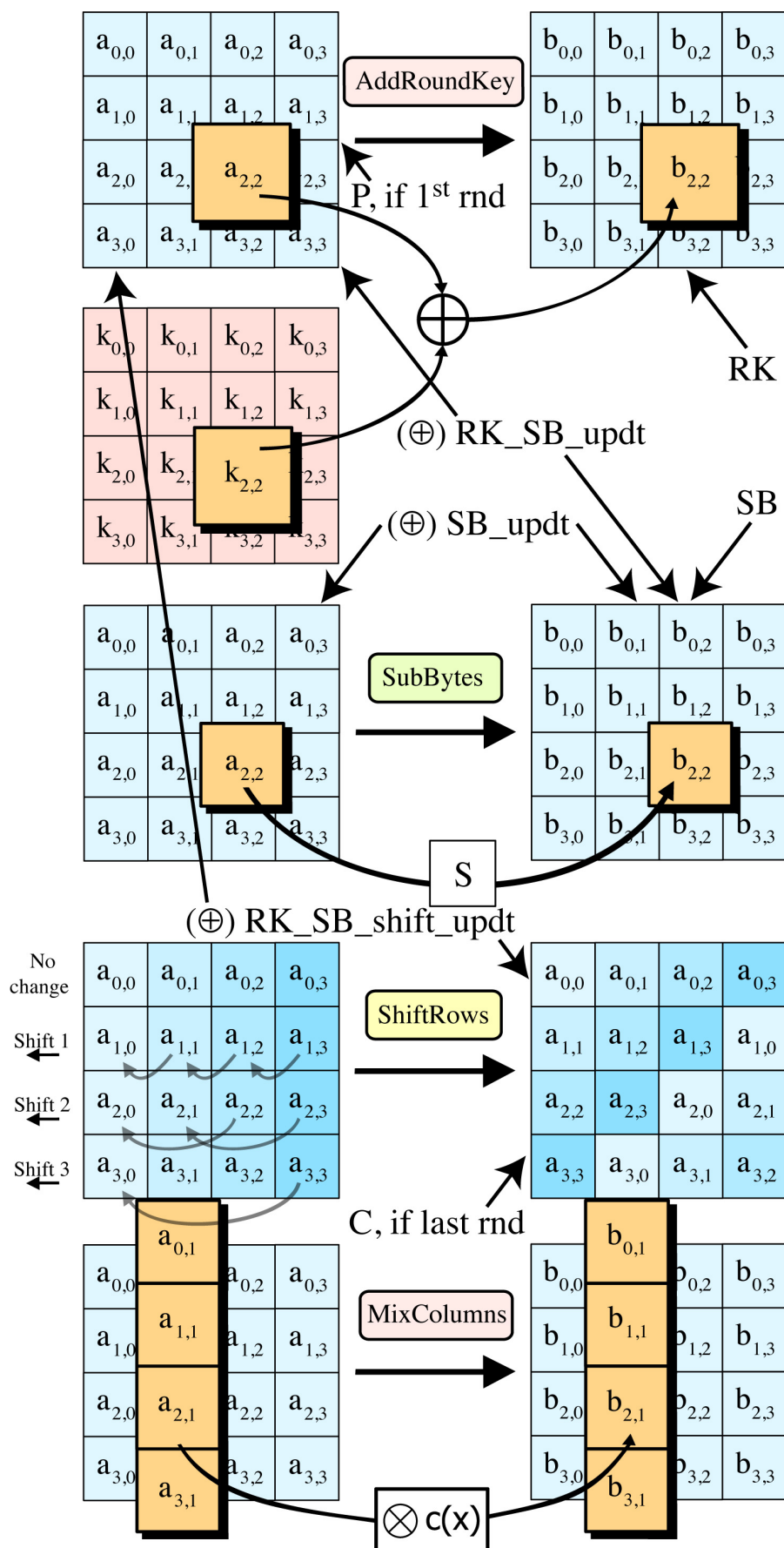
368 It is not uncommon for countermeasures – particularly those based on mathematical properties – to be effective only against one or two select functions, but not against others. Consequently, it is always required to verify a given countermeasure against *all* available select functions.¹⁷



¹⁷ So, for DES this would mean the following select functions: Kocher, direct, R_update, and S_box_in, whilst for AES this means: RK, SB, SB_updt, and RK_SB_updt.



Standard select functions for the DES algorithm: ‘Left’, ‘Right’, ‘L’, ‘R’, and ‘L_xor_R’ do not target the secret key as such, but are instead used to localise the DES operation within a trace. Out of these the ‘L_xor_R’ select function has the advantage of being specific to the DES operation itself and agnostic of the plaintext and ciphertext messages as such.

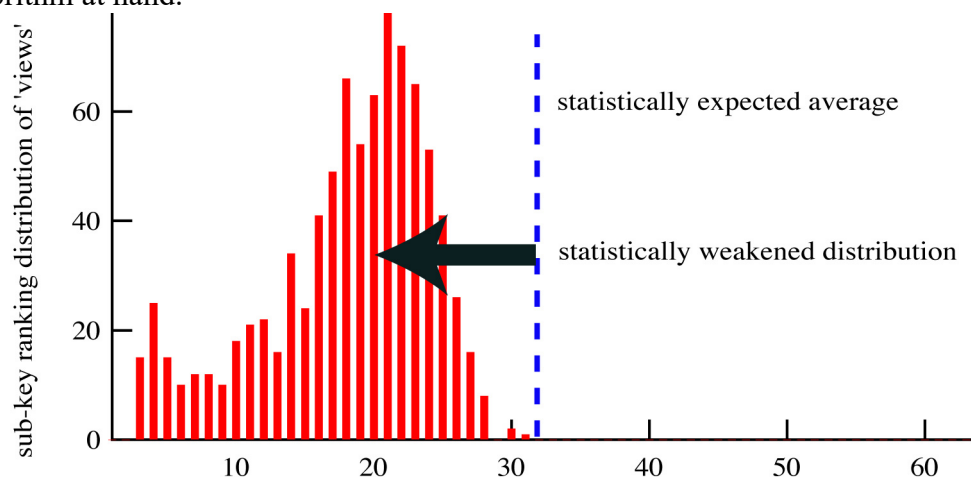


370 Standard select functions for the AES algorithm. ‘P’ and ‘C’ do not target the secret key as such, but are instead used to localise the AES operation within a trace.

- 371 The outcome of side-channel attacks (e.g. DPA or template attack providing probability tables) is a ranking list of all the possible guesses for each sub-key, for instance according to the strength of the correlation signals found. At the top of this ranking list is the guess for the sub-key part with the highest probability of being the correct one. Obviously, for a side-channel attack to be successful, the correct guess should be at the top for each sub-key in the ranking list. However, even if this is not the case, a successful attack could be possible within considerable time, if the combination of different sub-keys is tested according to their ranking in the list achieved by the side-channel attack. An attacker can take advantage of the statistical bias by using smart key-search strategies and combine the side channel attack with a biased brute-force attack. This must be considered for the overall time estimated for an attack. Typically, this smart key-search strategy can reduce the overall time of the combined attack compared to a side channel only attack path. Furthermore, it is possible to reduce the number of measured traces while still being successful with the attack, which has an impact in case of a restricted number of key usages.
- 372 The key verification is the time determining factor in the process of key enumeration and verification. This process is objective as it does not take advantage of the knowledge of the secret to stop the searching. If the evaluator is in a known key setting, he/she can perform key rank estimation to determine the leakage of the Device Under Test (DUT).
- 373 The rank estimation provides a result for the reduction of the key search space to the analysis performed. However the results are probabilities that can vary between two measurements even if the key and the measurement setup are the same. Therefore, the result of a rank estimation is not conclusive to perform an assessment based on a fixed limit of remaining entropy. The ranking difference between two measurements with same parameters and same test setup can be up to 25 bits for the same number of traces (empirical result reported by [RankDist] and confirmed by JHAS expert group). This means that for a rank measurement of 55 bits it could be concluded that an entropy of 80 bits security remains at maximum.
- 374 To support comparable evaluation results, an algorithm that provides both rank estimation and key enumeration, for example [Knapsack] or [SKEA], is recommended to be used.
- Other search strategies may be devised as well – as long as they are all objective in the sense described above – it is perfectly legitimate to follow the strategy which leads to the best attack results
- 375 Great care should be taken to make use of all the information leaked. For instance, taking the AES algorithm as an example, it would not be enough to simply select the first bit of each S-box as the target point for the select function. The other remaining 7 output bits are likely to leak similar amounts of information as the first one. On the other hand, it is also not enough to take the Hamming weight of all 8 bits only. In fact, it has been proven useful to take all possible ‘views’ of target bits and generate a statistics over all these views. In the case of the AES, 8 different single target bits can be selected for each S-box, 28 different combinations of 2 target bits, 56 different combinations of 3 target bits, and so on. In total, 255 different views can be generated like this. In the next step one can improve the statistics by averaging the amplitudes over, say, all 56 3-bit select functions. Experience shows that this can yield substantial improvements. Yet, there is more information contained in the traces. Since the smallness of the AES S-box (and even more so in the case of the DES S-box) introduces correlations inside the S-box, one finds that even wrong guesses for the sub-key yield correlation peaks –

although usually smaller ones than for the correct sub-key. Thus, even in the limit of infinitely many traces collected, wrong guesses will yield – so-called – ghost peaks. Thus, rather than looking at one correlation peak in isolation, it is also possible to study the distribution of these ghost peaks as a function of the guessed sub-key. This immediately leads to another 255 different views. And yet more views may be defined. Clearly, these so far 510 views are not statistically independent, but it is still valuable to study the associated sub-key ranking statistics for any trends as the number of acquired traces increases. A good reference point for declaring the AES to be broken is when the first of the 510 views retrieves the entire key.

376 Clearly, such statistical strategies will always need to be tailored to the cryptographic algorithm at hand.



sub-key ranking for DES

377 Distribution of sub-key rankings for a number of ‘views’ for the DES algorithm.
378

379 The above approach is largely the same for EMA attacks. However, a major difficulty with local-EMA is the number of steps at which data are collected and then analysed. With present technology, it is likely that the attack will begin by attempting to identify gross features of the emissions that are visible within a single example (e.g. areas that have some distinct pattern of emission when a coprocessor is operating). Areas of interest can then be investigated in more detail (i.e. by moving the antenna in smaller steps) in order to find the best location to collect and analyse large numbers of examples.

380 Typical issues that need to be solved when performing side-channel attacks are how to determine the best collection conditions such as sampling window within the data collection and sampling rate (as well as other conditions which may depend on the design of the data collection apparatus), the choice of select function, and an appropriate and efficient signal pre-processing.

381 For higher-order DPA attacks the approach is essentially the same as described above, except that in this case correlation functions between different points in the algorithm will need to be analysed. In general, this requires a detailed model that the attacker needs to create and validate. Such a model for higher-order DPA takes into account that, e.g., the masked data and the mask value can be expected at different points in time. To complicate things, the relative distance between these points is usually not known to the attacker and needs to be found out experimentally. In general, k -th order DPA considers simultaneously k samples that correspond to k different intermediate values within the same power consumption trace at different times or within several channels at the same time. Several statistical methods can be applied for analysing the information leakage

- and overcoming the countermeasures. Generally, these methods need a greater amount of acquisition traces than standard DPA – provided vulnerability exists for standard DPA in the first place.
- 382 Other DPA attacks on, e.g. RSA and ECC will be discussed by way of examples further below.
- 383 Typically, a broad range of ‘Expert’ skills and techniques are required to launch side-channel attacks – and particularly so higher-order attacks: HF-savvy measurement techniques for collecting high-quality power or electromagnetic traces, advanced signal-processing techniques, and a deep but practical knowledge of cryptography and in particular the secure implementation of cryptographic algorithms. It is acknowledged that these skills may vary strongly between evaluation labs and are difficult to assess.
- 384 In terms of equipment a high-end digital sampling oscilloscope, preferably with active probes, and a high-end PC are enough. In the case of a power analysis, custom-made card readers will greatly improve the quality of the measurement, as will an electrically screened measurement environment for all of these measurements. A number of very decent “Specialized” EMA probes are readily available from commercial outlets and thus “Bespoke” is not a rating usually justified any longer, even for probes hand-crafted by the evaluator. For local-EMA automated x-y positioning tables will be beneficial. The generally accepted JIL rating here is ‘Specialised’, in particular for DPA attacks. However, the rating could also be ‘Standard’ only, when low-end oscilloscopes are enough to carry out the attack (e.g., when SPA is only used to define a trigger point for a subsequent DFA attack).
- 385 The Knowledge of the TOE used is often Public (based on just taking measurements. However, the effort for an SPA/DPA attack depends strongly on the countermeasures that are implemented in the TOE, which may be software and / or hardware related. Depending on the nature of these countermeasures, ‘Sensitive’ or even ‘Critical’ information may be required by the attacker in order to be successful, and it should be carefully considered whether availability of such information would reduce the overall attack potential required in specific cases. For instance, local-EMA can be made more efficient, e.g., when knowing the precise location of a cryptographic coprocessor, but it has to be verified that this information cannot be obtained by simply scanning the TOE surface. Knowledge of the TOE may make an attack more efficient (reducing the attack potential required) or may disclose the possibility of different attacks (e.g. measuring the operation of design features specific to a particular TOE).
- 386 The higher-order DPA attack requires at least the same effort as for a standard DPA attack with respect to expertise, knowledge of the TOE, and equipment. Depending on the countermeasures and the implementation the effort increases at least for the identification of the attack.
- 387 In the case of a successful correlation attack, consideration is needed whether (or how easily) the attack could have been carried out by an attacker without knowledge of the secret data¹⁸. It is also relevant to consider whether the TOE is available to the attacker in a suitable form for characterisation experiments, i.e. for determining the underlying characteristics independent of any particular software that may be the part of the TOE.
- 388 Some standard publicly available references are:
- P. Kocher, “*Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems*,” in Proceedings of Advances in Cryptology–CRYPTO’96, Springer-Verlag, 1996, pp. 104-113;

¹⁸ Some correlations are relatively easy to identify with knowledge of the key, since the benefit of this knowledge can be applied to guide the search for best correlations. But it may be hard to find without this knowledge.

- P. Kocher, J. Jaffe, and B. Jun, “*Introduction to Differential Power Analysis and Related Attacks*,” <http://www.cryptography.com/dpa/technical>, 1998;
- Th. S. Messerges, “*Power Analysis Attacks and Countermeasures for Cryptographic Algorithms*” Thesis, University of Illinois at Chicago, 2000.
- S. Mangard, E. Oswald, and T. Popp, “*Power Analysis Attacks: Revealing the Secrets of Smart Cards*”, Advances in Information Security. Springer, 2007.
- F. Amiel, B. Feix, and K. Villegas, “*Power Analysis for Secret Recovering and Reverse Engineering of Public Key Algorithms*”, SAC 2007, LNCS 4876, p 110-125. Springer, 2007.
- *ElectroMagnetic Analysis (EMA): measures and countermeasures for smart cards*, J.-J. Quisquater and D. Samyde, in I. Attali and T. Jensen editor, Proceedings of the International Conference on Research in Smart Cards E-Smart, Lecture Notes in Computer Science, Springer-Verlag vol 2140, pp. 200 - 210, September 2001
- *Electromagnetic Attacks: concrete results*, K. Gandolfi, C. Mourtel, and F. Olivier, in D. Naccache editor, Proceedings of the Cryptographic Hardware and Embedded Systems CHES 2001, Lecture Notes in Computer Science, Springer-Verlag vol 2162, pp. 251 - 261, May 2001
- *The EM side-channel(s)*, D. Agrawal, B. Archambeault, J.R. Rao, and P. Rohatgi, in B. Kaliski editor, Proceedings of the Cryptographic Hardware and Embedded Systems CHES 2002, Lecture Notes in Computer Science, Springer-Verlag vol 2523, pp. 29 - 45, August 2002

389 In summary, applications for SPA and DPA attacks are manifold. ‘Obvious attacks’ may be considered to be:

- SPA attack to recognise – in real time – a write operation to EEPROM that handles critical information such as a PIN verification count-down counter
- SPA attack for launching a subsequent DFA attack, again in real time
- SPA attack for timing analysis or code-branching analysis purposes
- SPA attack on the unsecured square-and-multiply stage in RSA or the double-and-add stage in ECC
- SPA attack of the key loading stage of cryptographic algorithms
- DPA attack on DES and AES
- DPA attack on RSA and ECC
- DPA attack on elementary operations such as $k * m$, where k is the secret key and m a known but varying value.
- SPA or DPA attacks using local-EMA to localize a cryptographic component on the die in order to launch a subsequent attack (i.e., reverse engineering or fault attacks).
- Combinations of power and electromagnetic measurements.

390 The power analysis method itself has been described in many publications and is by now well understood – at least in principle. Hence, it can be considered as being in the

public domain. However, depending on the effectiveness of the countermeasures being used in the TOE, the effort required for launching a successful attack may vary greatly, leading to substantially different ratings. For example, for a very basic and straight-forward implementation of the cryptographic algorithm at hand, a proficient attacker might be able to perform a successful attack in less than a day, using simple equipment and standard software tools only. In such a case not even a “low” rating could be achieved. On the other hand, when facing a hardened implementation of the algorithm, even an expert using specialised equipment like an expensive oscilloscope and custom-made analysis software may require far more than a month before the attack is showing any signs of a possible success. This would be rated as “high” attack potential.

391 Given the large uncertainty in the assessment of a “partially successful” attack, which is always open to debate and controversial discussions, it is of paramount importance to provide some guidance here. In essence, it is always a good idea to establish objective criteria based on, e.g., the key-search strategies or the ranking distributions discussed above. For instance, if it can be shown that there is a clear trend of the number of broken bits (obtained with such a search strategy) as a function of the number of power consumption traces collected, then it is only reasonable to argue that by increasing the number of power consumption traces the attack will ultimately be successful. In contrast, it is generally discouraged to argue on the basis of vague statements like “the correct sub-key is in the top third of the ranking list” and the like, as this is often highly subjective, if a detailed statistical analysis is not available. This can be particularly misleading when the focus is not on the entire secret key but on a single sub-key only. It is, in fact, not recommended to restrict the analysis to a single sub-key (i.e. S-box in the case of the DES algorithm) only, as experience has shown that the ranking results may differ widely and not reproducibly so from one sub-key to the next, simply due to statistical effects. Such a narrow approach is only acceptable if a particular target sub-key has been agreed upon beforehand to avoid a biased assessment or, alternatively, if an extensive statistical analysis has been supplied as well.

2.5.5 Note about Profiled Side-Channel Attacks

2.5.5.1 Generalities

392 Profiled Side-Channel Attacks are very similar to non-profiled Side-Channel Attacks and the whole process is the same. The only difference is:

- In non-profiled SCA (SPA, DPA, CPA, ...), the adversary chooses *a priori* several attack parameters (target intermediate value, leakage model, ...), using his experience and his knowledge of the target (e.g. classical leakage models are Hamming weight and Hamming distance). Then he uses the chosen leakage model for the *attack phase*.
- In profiled SCA (Template Attack, Stochastic Attack, ...), the adversary chooses *a priori* several attack parameters (target intermediate value, ...), but he builds the leakage model *a posteriori*, during the first step of the attack, called the *profiling phase*. Thus he requires a copy of the target where it is possible to use chosen or at least known secrets, in order to perform measurements to compute the leakage model. Then he uses the built leakage model for the *attack phase*.

393 We recall that the *attack phase* of both non-profiled and profiled SCA is the same (for both statistical distinguishers and algorithmic approaches can differ depending on the context).

2.5.5.2 Profiled Side-Channel Attacks Scenarios

(a) First case: same device copy and same SCA measurement bench for the two phases

394 In this case the same device copy is used for the two phases. Moreover, the adversary performs the two phases using the same SCA measurement bench.

395 Example 1: An open platform has an AES hardware co-processor, and an application that uses the co-processor with a secret AES key. We consider an adversary who has the rights to install new applications. Thus the adversary can create another application enabling the AES hardware co-processor to be loaded with different chosen AES keys to perform the *profiling phase*. Then, he uses the same device and processes the target application to perform the *attack phase*.

396 Example 2: An application provides the cryptographic functionalities SHA-256 for hash computation (e.g. during signature verification done in the TOE) and HMAC-SHA-256 for keyed hash calculation. The adversary can perform the *profiling phase* with chosen input for SHA-256 and the *attack phase* with HMAC-SHA-256 on the corresponding key if the implementation of SHA-256 is the same in both cases.

397 Example 3: A smartcard is compliant with PKCS#11 interface, and it is possible to generate a RSA private key off chip, and then to inject this key inside the smartcard. Then, the adversary can use this key to perform RSA signatures in a chosen key context, and then perform the profiling phase.

2

(b) Second case

398 Different device copies and same SCA measurement bench for the two phases.

399 In this case the adversary uses a first device copy for the *profiling phase*, and another device copy from the field for the *attack phase*. In both phases he uses the same SCA measurement bench.

400 Example 4: A banking application runs on a Java Card platform as a closed platform. The same Java Card platform is also deployed through an open platform product. The adversary manages to get the loading keys of the open platform. Thus he targets the triple-DES co-processor, and builds the profiles on the open platform by loading its own application, enabling the triple-DES co-processor to be used with chosen secrets to perform the *profiling phase*. Then he performs the *attack phase* on the banking product to retrieve the triple-DES key.

(c) Third case different device copies and different SCA measurement benches for the two phases

401 In this case a first adversary uses a first device copy with a first SCA measurement bench to perform the *profiling phase*. Then, a second adversary gets from the first adversary the *profiles* for the considered type of target (for instance through Internet), and uses another device copy from the field and another SCA measurement bench to perform the *attack phase*.

(d) Miscellaneous

402 We do not consider the case where the adversary uses the same device copy for the *profiling phase* and the *attack phase* with two different SCA measurement benches. Indeed, if the same adversary uses the same device copy for the two phases, he will obviously use the same SCA measurement bench for the two phases (less costly, more efficient).

2.5.5.3 Evaluation process considerations

403 In a security evaluation process, the scenario a) should be considered by the evaluator as the default one (same device copy and same SCA measurement bench for the two Profiled Side-Channel Attack phases), since the best case for the adversary (and then the worst case for the developer) should be always preferred.

2.5.5.4 Example of Targets

- input data: all kind of data that the (cryptographic) operation takes as input, e.g. keys, messages;
- sensitive data: confidential data used or generated during the (cryptographic) operation. Sensitive data can be input data but can also be temporary data generated during operation.

404 The following list gives examples of Profiled side-Channel Attack targets:

- cryptographic key, PIN code or other kinds of secrets during their transfer from memory / (co-)processor to memory / (co-)processor;
- cryptographic key, PIN code or other kinds of secrets if they are handled for integrity checking;
- sensitive data if it is handled by a cryptographic implementation (cryptographic key, message that will be encrypted, result of a decryption, sensitive intermediate data linked to the input and the cryptographic key, temporary sensitive data during operation like Elliptic Curve point in PACE ...);
- reference PIN code when it is handled for comparison with the submitted one.

2.5.6 **Example: SPA/SEMA to retrieve DES key**

2.5.6.1 Attack Path

405 In this example an attacker uses simple power analysis to analyse the key-loading stage prior to a DES operation. It is assumed that the TOE has one “playground” application where the key for a (simple) DES operation can be freely chosen by the attacker, and another target application where the secret key is to be obtained.

406 The aim of a first profiling stage of the template attack is to characterise the power consumption profiles during the loading stage on the basis of known keys, using the playground application. The outcome is a database, say, for each possible value of each key byte. This database of known power consumption profiles is then employed in a second stage of the attack to identify the secret key bytes used in the target application. Since absolute (as opposed to differential) power consumption profiles tend to be very delicate and vary greatly from one TOE to the next, it is generally not expected that the profiling information gathered for one TOE can be taken advantage of for another one. Therefore, the “playground” application and the target application usually need to reside on the same TOE. Also, for the same reason, such a profiling step will need to be repeated for every TOE, even in the Exploitation phase.

407

2.5.6.2 Example rating 1 without countermeasure

408 It is assumed that there are no further countermeasures other than a “natural” timing-jitter in the power traces due to imperfections of the TOE’s internal clock.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Pure data collection of a minimum of 30,000 traces takes less than 1 day for the profiling stage (e.g., 20,000 known and 10,000 unknown keys). Some realignment sessions are likely to be needed, which can easily take a few days for 30000 traces. This applies for both, Identification and Exploitation. (The elapsed time scales roughly linearly with the number of traces taken.)	< 1 week (2)	< 1 week (4)
Expertise	For Identification, the attacker must be an Expert for setting up the measurement and specifying the algorithms for further signal processing. Only a Proficient attacker is required when the attack is repeated in the Exploitation phase.	Expert (5)	Proficient (2)
Knowledge of TOE	In general, we assume that command details are taken from the FSP for the TOE and hence, following [JIL-AP], are Restricted. However, in some cases, where the command specification is freely available (e.g. Global Platform), the knowledge may be Public only.	Restricted (2)	Public (0)
Access to TOE	Only a single TOE is required	< 10 Samples (0)	< 10 Samples (0)
Open Samples / Known Key	This attack assumes that a “playground” application exists on the device where the DES keys can be chosen freely.	Public (0)	Public (0)
Equipment	The equipment includes dedicated equipment built to collect the power traces, a high-end digital oscilloscope, and non-standard, home-grown SPA analysis software.	Specialized (3)	Specialized (4)
Sub Total		12	10
Total		22	

409 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3).

2.5.6.2.1 *Required demonstration of attack to justify this rating*

410 If the secret key of the target application is known in advance, it is possible to set up a key-search strategy and assess the remaining size of the key space that is left for a brute-force search. The attack is deemed to be successful, if a trend analysis is positive and a significant¹⁹ number of bits have been found.

411 Countermeasures that may make this attack more difficult by reducing the signal-to-noise level include Current Balancing, Current Smoothing, Randomised Clock, Randomized Loading of Key Bytes, and Blinding. Some of these can be compensated for – at least partially – by advanced signal processing techniques, but this will cost

¹⁹ For this example, significant means to obtain one third of the bits. In general it depends on the search strategy, the algorithm and the strength of the trend.

processing time. Note also that processing time is not the mere computing time but, more importantly, includes the human time as well for analysis and for deciding upon the next steps.

412

2.5.6.2.2 Remark on key schedule

413 In single-trace attack scenarios many software countermeasures are ineffective but very little information is available for an attacker. Thus, an attacker wants to combine all information given by a single trace, e.g. by overlaying parts of the trace and combining results of different analysis of the same trace. For instance, in case there exists strong Hamming-distance leakage of subsequent subkeys in the DES key schedule, then there exists the risk that combining leakage points is useful for single trace template attacks because pairs of key bits contribute to the Hamming-distance leakage in several rounds of the DES key schedule. Furthermore, overlapping groups of key bits can be chosen separately as target for template attacks such that consistency arguments for the results can potentially be used to improve the remaining brute force attack.

2.5.7 Example: DPA to retrieve DES key during DES calculation

2.5.7.1 Attack Path

414 In this example an attacker uses differential power analysis to extract the secret key from power consumption traces taken during the execution of the DES algorithm itself. It is assumed that the TOE contains an application where the message for a (simple) DES operation can be freely chosen by the attacker.

2.5.7.2 Example rating 1 without countermeasure

415 It is also assumed that the DES algorithm is performed by a HW coprocessor and that no further countermeasures exist other than a “natural” timing-jitter in the power traces due to imperfections of the TOE’s internal clock. For SW implementations the number of traces required will be substantially reduced.

Factor	Comment	Ident’n	Exploit’n
Elapsed Time	Pure data collection of a minimum of 100,000 traces takes less than 1 day. Signal analysis and running the DPA analysis software will take less than a week for the Identification and less than a day for the Exploitation phase. In the Exploitation phase less traces will be needed, say 50,000, since it is expected that the attacker then knows precisely where to ‘zoom in’ for maximising the leakage signal. (The elapsed time scales roughly linearly with the number of traces taken.)	< 1 week (2)	< day (3)
Expertise	For Identification, the attacker must be an Expert for setting up the measurement and specifying the algorithms for further signal processing. Only a Proficient attacker is required when the attack is repeated in the	Expert (5)	Proficient (2)

	Exploitation phase.		
Knowledge of TOE	In general, we assume that command details are taken from the FSP for the TOE and hence, following [JIL-AP], are Restricted. However, in some cases, where the command specification is freely available (e.g. Global Platform) the knowledge may be Public only.	Restricted (2)	Public (0)
Open Samples / Known Key	It is expected that performing a Known Key attack will only be of substantial benefit in the presence of countermeasures. When open samples are managed appropriately, a 'Sensitive' rating would then apply.	Public (0)	Public (0)
Access to TOE	Only a single TOE is required.	< 10 Samples (0)	< 10 Samples (0)
Equipment	The equipment includes dedicated equipment built to collect the power traces, a high-end digital oscilloscope, and non-standard, home-grown DPA analysis software.	Specialized (3)	Specialized (4)
Sub Total		12	9
Total		21	

416 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3).

2.5.7.2.1 Required demonstration of attack to justify this rating:

417 If the secret key of the target application is known in advance, it is possible to set up a key-search strategy and assess the remaining size of the key space that is left for a brute-force search. The attack is deemed to be successful, if a trend analysis is positive and a significant²⁰ number of bits have been found. If the key is not known in advance, the entire key will need to be compromised. Alternatively, a trend analysis based on a statistical analysis of 'views' as a function of traces acquired – as explained above – can be performed.

418 Countermeasures that may make this attack more difficult by reducing the signal-to-noise level include Current Balancing, Current Smoothing, Randomised Clock, 1:N Randomization²¹ and Blinding. Some of these can be compensated for – at least partially – by advanced signal processing techniques, but this will cost processing time. Note also that processing time is not the mere computing time but, more importantly, includes the human time as well for analysis and for deciding upon the next steps.

419 Depending on the nature of the countermeasures the evaluation time may be drastically reduced using Known Keys.

²⁰ For this example, 'significant' means to obtain one third of the bits. In general it depends on the search strategy, the algorithm and the strength of the trend.

²¹ This countermeasure requires the DES calculation to be repeated N times, with N-1 randomly chosen wrong keys and once the correct key (and always the same input for all N calculations). The position of the correct key within the N calculations needs to be randomly chosen. One can show that as long as SPA on single traces does not leak any information, this countermeasure increases the number of traces required for a successful attack at most by a factor N. Note also that the countermeasure can be improved somewhat by choosing fixed (wrong) N-1 keys rather than random ones. This will increase the attack resistance for attacks on the first round, but not for attacks on the last round.

2.5.8 Example: higher-order DPA to retrieve DES-key

2.5.8.1 Attack Path

420 In this example, an attacker uses higher-order differential power analysis to break an implementation of DES with well implemented countermeasures (masking). The attacker sends arbitrary encrypted messages that require the TOE to decrypt each message for validation. Yet, a possible vulnerability was identified based on the analysis of the design. The rating takes into account that the straightforward adjustment of the higher-order analysis according to the identified possible vulnerability is sufficient. The power consumption traces are sufficient to apply the attack. The statistical analysis was adapted using design information with respect to the combination of the appropriate time intervals of the traces. The attack was successful using 150,000 power consumption traces. One measurement is expected to take 0.5 sec.

421 Attack Steps:

- 1) The attacker sends repeated messages (generated by a test harness developed by the attacker for this type of attack) to the TOE in its normal condition (i.e. no physical modification or de-processing is necessary), and collects the power traces using a digital oscilloscope.
- 2) The data is analysed using a variety of different pre-processing steps. Further statistical steps are applied to combine the results at different time slots indicating a key leakage.

422 After Identification, it is assumed that the Exploitation phase requires the attack to be repeated but with knowledge of the appropriate analysis technique (detailed setup of the higher-order analysis, etc.).

2.5.8.2 Example rating 1 without specific countermeasures against higher-order dpa

423 A sound implementation resistant to DPA is assumed.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Pure data collection of the minimum 150,000 examples takes less than 1 day (assuming 24 hours/day collection with 0.5 seconds per example). In reality, such data collection sessions are likely to need to be repeated as part of the development of the attack. Using sufficient pre-processing and the adjustment of the higher order analysis also require planning, execution and analysis time. Also the exploitation requires the same amount of data. Even when the vulnerability has already been identified with 'Sensitive' knowledge, the amount of data processing of higher-order DPA is still substantially higher than for standard DPA.	< 1 week (2)	< 1 week (4)
Expertise	For identification, the attacker must be an Expert and capable of deriving new attacks, since the higher order analysis must be	Expert (5)	Proficient (2)

	adjusted based on the implementation and the countermeasures. Only a Proficient attacker is required when the attack is repeated in the Exploitation phase.		
Knowledge of TOE	Since the possible vulnerability is identified using sensitive design knowledge is assumed for the identification phase. Since the script used for the exploitation "includes" all information no further knowledge is required for the exploitation.	Sensitive (4)	Public (0)
Access to TOE	Only a single card is required.	< 10 Samples (0)	< 10 Samples (0)
Open Sample / Known Key	In the Identification phase the key needs to be known to the attacker	Sensitive (4)	NA
Equipment	The equipment includes equipment built to collect the power data, a digital oscilloscope, and analysis software based on a 'typical' home-developed DPA tool.	Specialized (3)	Specialized (4)
Sub Total		18	10
Total		28	

424 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

2.5.8.2.1 Required demonstration of attack to justify this rating:

425 The higher order DPA requires normally a great amount of data. The adjustment of the analysis is usually supported by the knowledge of the design. The post-calculation time is also more significant for this attack than for a standard DPA attack. It may be possible to identify a possible vulnerability based on special values used for the first part of the analysis. The attack is deemed to be successful, if a trend analysis is positive and a significant²² number of bits have been found.

2.5.9 Example: SPA on RSA

Attack Path

426 In this example an attacker uses simple power analysis to identify the sequential exponent bits of an RSA implementation using binary exponentiation. The attack can also be applied on implementations using Montgomery or CRT optimization. Once the attacker has the exponent bits he can mathematically derive all relevant crypto parameters if the public key is available. In the identification phase the attacker will distinguish the process steps for the individual exponent bits, and use filtering to recognize the value of the processed key bit. If the S/N ratio is low the attacker may want to use multiple traces and use an averaging method. In that case it is required that the exponent bits do not vary from one trace to the next and are always processed in the

²² For this example 'significant' means to obtain one third of the bits. In general it depends on the search strategy, the algorithm and the strength of the trend.

same order. In the exploitation phase the key bit recognition process will be repeated for all exponent bits.

2.5.9.1 Example rating 1 without countermeasure

427 It is assumed that no further countermeasures exist other than timing-jitter in the power traces due to instability of the TOE's internal clock, and some noise superimposed on the power signal.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Data collection of about 1,000 traces takes at a few hours. Extensive alignment and filtering sessions are likely to be needed, which may take several days for Exploitation.	< 1 week (2)	< 1 day (3)
Expertise	For Identification, the attacker must be an Expert for setting up the measurement and specifying the algorithms for further signal processing. Only a Proficient attacker is required when the attack is repeated in the Exploitation phase.	Expert (5)	Proficient (2)
Knowledge of TOE	The attacker will only need protocol specifications, which are freely available (e.g. Global Platform).	Public (0)	Public (0)
Access to TOE	Only a single TOE is required	< 10 Samples (0)	< 10 Samples (0)
Open Samples / Known Key	This attack assumes that a "playground" application exists on the device where the RSA keys can be chosen freely. This is for figuring out whether the algorithm works left-to-right, right-to-left,	Public (0)	Public (0)
Equipment	The equipment includes dedicated equipment built to collect the power traces, a high-end digital oscilloscope, and SPA analysis software.	Specialized (3)	Specialized (4)
Sub Total		10	9
Total		19	

428 This TOE would be resistant for this attack to an attacker with basic attack potential (VAN.2).

2.5.9.1.1 Required demonstration of attack to justify this rating

429 If the private key of the target application is known in advance, it is possible to tune a filter to distinguish multiplication from square operations. For practical RSA private keys the first and last exponent bits are always set to '1', this may be helpful in the design of the filtering. The attack is deemed to be successful, if at least 8 exponent bits have been found.

430 Countermeasures that may make this attack more difficult by reducing the signal-to-noise level include hiding measures such as Current Balancing, Current Smoothing, Current Scrambling, Randomised Clock and Random Delays. These countermeasures

can be compensated for – at least partially – by advanced signal processing techniques, but this will cost processing time. Further security improvement is possible with algorithmic countermeasures, such as multiply-always, (randomized) n-ary exponentiation, and exponent blinding.

2.5.10 Example: DPA on RSA (Cross-Correlation Attack)

2.5.10.1 Attack Path

- 431 In this example an attacker uses differential power analysis to extract the private key from power consumption traces taken during the exponentiation operation with the RSA algorithm in Straight-Forward mode. The attack does not require any plain text or cipher text to be recorded. It is assumed that the algorithm has been implemented using the single-bit “Always Square and Multiply” method. The target of the attack is derived from the observation that the exponentiation has two similar but still distinct mathematical operations (with intermediate value S , message M , modulus N),
- Square: $S' := S * S \bmod N$
 - Multiply: $S' := S * M \bmod N$
- 432 Subsequent square operations usually do not share operands, whilst multiply operations do share a common operand (M). Operand sharing may be observed if the order of square and multiply operations is identical for repetitive encryptions. This can be visualised in a matrix.
- 433 It should be noted that a number of classical countermeasures are ineffective against this attack.

2.5.10.2 Example rating 1 including countermeasure

- 434 It is assumed that the countermeasures include misalignments due to clock-jitter and various random delays. Furthermore the S/N ratio is low due to severe noise. The mathematical countermeasure of message blinding is also activated.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Data collection of 1000,000 traces takes several days. Signal analysis and running the DPA analysis software will take less than a month for the Identification and less than a week for the Exploitation phase. In the Exploitation phase less and shorter traces will be needed since it is expected that the attacker then knows precisely where to ‘zoom in’ and what filtering to apply for maximising the leakage signal.	< 1 month (3)	< 1 week (4)
Expertise	For Identification, the attacker must be an Expert for setting up the measurement and specifying the algorithms for further signal processing. Only a Proficient attacker is required when the attack is repeated in the Exploitation phase.	Expert (5)	Proficient (2)
Knowledge of TOE	We assume the command specification is freely available (e.g. Global Platform, EMV)	Public (0)	Public (0)

Open Samples / Known Key	No open samples needed.	Public (0)	Public (0)
Access to TOE	Only a single TOE is required.	< 10 Samples (0)	< 10 Samples (0)
Equipment	The equipment includes dedicated equipment built to collect the power traces, a high-end digital oscilloscope, and DPA analysis software.	Specialized (3)	Specialized (4)
Sub Total		11	10
Total		21	

435 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3).

2.5.10.2.1 Required demonstration of attack to justify this rating:

436 The attack is deemed to be successful, if at least 8 exponent bits have been found.

437 Countermeasures that mitigate this attack would use more complex forms of mathematical blinding and/or more complex ways of parsing the exponent during the exponentiation.

2.5.11 Example: DPA on RSA-CRT

2.5.11.1 Attack Path

438 In this example an attacker uses differential power analysis to extract the private key from power consumption traces taken during a signing operation with the RSA algorithm in CRT mode. The attack targets the recombination phase of the algorithm. The attacker should be able to sign many different (possibly unknown) messages and record the signature.

2.5.11.2 Example rating 1 without countermeasure

439 It is assumed that the countermeasures include misalignments due to clock-jitter and various random delays. Furthermore the S/N ratio is low due to severe noise.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Data collection of 1000,000 traces takes several days. Signal analysis and running the DPA analysis software will take less than a month for the Identification and less than a week for the Exploitation phase. In the Exploitation phase less and shorter traces will be needed since it is expected that the attacker then knows precisely where to 'zoom in' and what filtering to apply for maximising the leakage signal.	< 1 month (3)	< 1 week (4)
Expertise	For Identification, the attacker must be an Expert for setting up the measurement and	Expert (5)	Proficient (2)

	specifying the algorithms for further signal processing. Only a Proficient attacker is required when the attack is repeated in the Exploitation phase.		
Knowledge of TOE	We assume the command specification is freely available (e.g. Global Platform, EMV)	Public (0)	Public (0)
Open Samples / Known Key	No open samples needed.	Public (0)	Public (0)
Access to TOE	Only a single TOE is required.	< 10 Samples (0)	< 10 Samples (0)
Equipment	The equipment includes dedicated equipment built to collect the power traces, a high-end digital oscilloscope, and DPA analysis software.	Specialized (3)	Specialized (4)
Sub Total		11	10
Total		21	

440 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3).

2.5.11.2.1 Required demonstration of attack to justify this rating:

441 The attack targets the multiplication of a secret prime with an intermediate value. The RSA signature, the result of the recombination formula, consists of a significant term containing this product, and a small term unknown to the attacker. By ignoring the small term the attacker can use DPA to subsequently derive most prime bytes. The final prime bytes can easily be found with a small brute-force search, involving the RSA modulus (part of the public key). The attack is deemed feasible if several of the most significant prime bits can be retrieved. Finding the remaining bits would just take more time.

442 Countermeasures that mitigate this attack would use a mathematical blinding countermeasure to avoid the direct and repeated use of the secret prime.

2.5.12 Profiled Side-Channel Attack on AES

2.5.12.1 Profiled Side-Channel Attack on AES co-processor of an IC with a masking countermeasure that cannot be deactivated

443 This example consists in an IC evaluation. Here the AES co-processor is evaluated against Profiled side-channel Attacks. As in this case the evaluator cannot deactivate the masking countermeasure, no point for the use of an evaluation sample is given. Nevertheless, two (2) points have to be given for the profile construction effort.

Factor	Comments	Identification	Exploitation
Elapsed time	The identification phase will take less than one month. The exploitation phase will take less than one week	< one month (3)	< one week (4)

Expertise	For identification phase, an expert is required. Only a proficient is required for exploitation phase.	Expert (5)	Proficient (2)
Knowledge of the TOE	For identification phase, the knowledge of the AES interface is required	Restricted (2)	Public (0)
Open samples / known key	For identification phase, the Profile construction effort has to be taken into account	2	NA
Access to the TOE	One sample is enough to mount the attack	< 10 samples (0)	< 10 samples (0)
Equipment	A side-channel bench is required for both phases	Specialized (3)	Specialized (4)
Sub Total		15	10
Total		<u>Profiled Side-Channel Attack on AES co-processor of an IC with a masking countermeasure that cannot be deactivated</u>	

2.5.12.2 Profiled Side-Channel Attack on AES co-processor of an IC with a masking countermeasure that can be deactivated

- 444 This example consists in an IC evaluation. Here the AES co-processor is evaluated against Profiled side-channel Attacks. In this case the evaluator uses the ability the switch off/on the masking countermeasure to speed-up the evaluation. Thus he has to rate by 4 points (sensitive) the use of an evaluation sample allowing to switch off/on the masking countermeasure. Furthermore, 2 points have to be given for the profile construction effort. So the maximum between these two scores is reported in the “Open samples / known key” factor in identification phase.

Factor	Comments	Identification	Exploitation
Elapsed time	The identification phase will take less than one month. The exploitation phase will take less than one week	< one month (3)	< one week (4)
Expertise	For identification phase, an expert is required. Only a proficient is required for exploitation phase.	Expert (5)	Proficient (2)
Knowledge of the TOE	For identification phase, the knowledge of the AES interface is required	Restricted (2)	Public (0)

Open samples / known key	For identification phase, an evaluation sample is required to be able to switch off/on the countermeasure	4	NA
Access to the TOE	One sample is enough to mount the attack	< 10 samples (0)	< 10 samples (0)
Equipment	A side-channel bench is required for both phases	Specialized (3)	Specialized (4)
Sub Total		17	10
Total		27	

2.5.13 Profiled Side-Channel Attack on RSA

2.5.13.1 Profiled Side-Channel Attack on RSA implementation provided by an IC cryptographic library

445 This example consists in an IC evaluation. Here the RSA implementation provided by the IC cryptographic library is evaluated against Profiled side-channel Attacks. The considered RSA implementation has message and modulus randomization as countermeasures.

Factor	Comments	Identification	Exploitation
Elapsed time	The identification phase will take less than one month. The exploitation phase will take less than one week	< one month (3)	< one week (4)
Expertise	For identification phase, an expert is required. Only a proficient is required for exploitation phase.	Expert (5)	Proficient (2)
Knowledge of the TOE	As commands for banking products are public, no particular knowledge is required.	Public (0)	Public (0)
Open samples / known key	For identification phase, the Profile construction effort has to be taken into account	2	NA
Access to the TOE	One sample is enough to mount the attack	< 10 samples (0)	< 10 samples (0)
Equipment	A side-channel bench is required for both phases	Specialized (3)	Specialized (4)

Sub Total		13	10
Total		23	

2.5.13.2 Profiled Side-Channel Attack on RSA implementation provided by an open platform

- 446 This example consists in an open platform evaluation. Here the RSA implementation provided by the open platform (and thus developed by the smartcard vendor who has also developed the platform) is evaluated against Profiled side-channel Attacks. The considered RSA implementation has exponent and message randomization as countermeasures.
- 447 The evaluator uses the ability to choose special values for the exponent and the modulus such that it renders the exponent randomization partially ineffective. Thus he is able to know some of the bits of the randomized exponent, and can create profiles for known exponent bits.

Factor	Comments	Identification	Exploitation
Elapsed time	The identification phase will take less than one month. The exploitation phase will take less than one week	< one month (3)	< one week (4)
Expertise	For identification phase, an expert is required. Only a proficient is required for exploitation phase.	Expert (5)	Proficient (2)
Knowledge of the TOE	As commands for banking products are public, no particular knowledge is required.	Public (0)	Public (0)
Open samples / known key	For identification phase, the Profile construction effort has to be taken into account	2	NA
Access to the TOE	One sample is enough to mount the attack	< 10 samples (0)	< 10 samples (0)
Equipment	A side-channel bench is required for both phases	Specialized (3)	Specialized (4)
Sub Total		13	10
Total		23	

2.5.13.3 Profiled Side-Channel Attack on RSA implementation of a banking product with exponentiation randomization

- 448 This example consists of an RSA implementation with an exponentiation randomization

technique. Thus, even in the evaluation context where the evaluator can know the RSA exponent secret, the randomized exponent secret is not known by the evaluator as the randomization procedure is performed inside the RSA implementation using for instance the on-chip RNG.

- 449 If this RSA implementation is only used in AVA_VAN5 projects and if the evaluator has checked that the exponentiation technique is effective and extensive then the profile building phase cannot be conducted on this implementation and the Profiled Side-Channel attack has to be ranked as **not practicable**.

2.5.14 Example: combined Side-channel and brute force attack on ISO 9797-1 (MAC Algo3)

2.5.14.1 Attack Path

- 450 In this example an attacker uses differential power analysis to extract an intermediate single DES ciphertext value from power consumption traces taken during a MAC operation with the DES algorithm in ISO 9797-1 (MAC Algo3) mode. The attack targets a DES ciphertext when xored with a plaintext of the CBC chain. It also relevant to target the loading of this xored value during the next DES operation. The attacker performs a first brute force DES Cracker operation to recover the single DES key K_1 from the recovered ciphertext $C = \text{DES}(K_1, P)$ and known plaintext P (input value). The attacker performs a second brute force to recover the second key K_2 from the input plaintexts and the returned MAC value. At this stage the full TDES key is recovered.

2.5.14.2 Example rating 1 without countermeasure

- 451 This attack rating is similar to the one described previously Example: perturbation / cryptographic attack on 3DES.
- 452 We give an example in the case where the attacker has access to an open samples and known secret. The other cases can be deduced from the ratings given in section Example: perturbation / cryptographic attack on 3DES.
- 453 When not considering the DES cracker in the Identification and using specialized services for DES cracker attack part we obtain the following JIL rating:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Sample preparation, bench setup and traces acquisitions Signal processing of traces and statistical analysis.	< 1 week (2)	< 1 day (3)
	DES Cracker	NA	< 1 week (4)
	Total (addition²³)	< 1 week (2)	< 1 week (4)
Expertise	Side-channel analysis	Expert (5)	Proficient (2)
	DES Cracker	NA	Layman (0)
	Total (max)	Expert (5)	Proficient (2)

²³ For the „elapsed time“, the days are added, and not the values. For example, “<1 week” + “<1 week” results in “<1 month”. Then we take the value for “<1 month”.

Knowledge of TOE	Side-channel analysis	Restricted (2)	Public (0)
	DES Cracker	NA (0)	NA (0)
	Total	Restricted (2)	Public (0)
Access to TOE	Side-channel analysis	< 10 (0)	< 10 (0)
	DES Cracker	NA (0)	NA (0)
	Total (addition)	< 10 (0)	< 10 (0)
Open Samples / Known Key	Side-channel analysis (in accordance with paragraph 214)	Restricted (2)	NA
	DES Cracker	NA	NA
	Total (addition)	Restricted (2)	NA
Equipment	Perturbation	Specialized (3)	Specialized (4)
	DES Cracker	NA (0)	Specialized (4)
	Total (max)	Specialized (3)	Bespoke (6)
Sub Total		14	12
Total		26	

This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

2.5.15 Machine Learning as alternative for Profiled Side Channel Attacks

454 Machine learning is a subfield of computer science which includes all techniques aiming to give computers the ability to learn without being explicitly programmed and that gives rise to a lot of problems and methodologies. Several problems are similar to those coming from side channel context and many methods coming from machine learning field can be adapted, or even directly used, in such a context.

455 Mainly, Machine learning techniques are divided into three categories depending on the learning approach:

- Supervised machine learning
- Unsupervised machine learning
- Semi-supervised machine learning

456 For the sake of simplicity, this document is focused on supervised and unsupervised machine learning techniques.

457 Supervised learning is used to solve many different problems. Among them, the classification problem is the most widely studied since it is the central building block for all other problems (detection, regression, analysis...). It consists in taking an input, for example a side channel trace, and assigning to it a label, for example the value of the target variable handled during its acquisition. In the typical setting for the supervised classification problem, a training dataset is available. It is a set of data already assigned with the correct label (the profiling set in a side channel context). Supervised learning methods consist of at least 2 phases: a training phase and a matching phase. The training phase is performed on a labelled dataset while the matching phase is performed on unlabelled datasets which need to be classified. If the parameter settings of the used

learning method can be varied then a third phase has to be considered to guarantee the statistical independence of the unlabelled datasets of the matching phase. In this case the supervised learning methods consist of a training phase, an optimization phase and the matching phase. The optimization phase is also performed on unlabelled datasets, but may be repeated unlimited times to agree on the used parameter settings. If the settings are fixed then the matching phase with an independent dataset should be conducted only one time.

458 The supervised machine learning includes a lot of different methods such as Random Forest, Support Vector Machine, Kernel estimators or neural networks. [1, 2, 3, 4, 5, 6, 7].

459 The unsupervised machine learning problems consist in finding patterns or structures from unlabelled datasets. This category comprises a lot of methods such as k-means clustering [8, 9], unsupervised neural networks, method of moments, principal component analysis [10].

460 Almost all these methods can be used in a side channel context. “Usual” profiled attacks, such as template attacks [11], can be considered as supervised machine learning methods and CPA/DPA [12, 13] can be considered as unsupervised machine learning ones. In this context, the dataset consists of the acquired side channel traces and the labels are the usual selection function / classes. These methods can provide advantages compared to classical side channel attacks while they are easy to use (many open sources library are available). For this, they must be taken into account.

461 Recent machine learning based attacks exploit the same discriminating criteria (i.e. the dependence between the sensitive data and some statistical moments of the leakage) as a template attack. However, two major differences between these attacks exist:

- 1) The template attack approximates the data distribution by a multivariate Gaussian distribution (aka Gaussian leakage assumption) whose parameters (i.e. the mean vector and the covariance matrix) are estimated during the profiling phase. This implies that leakage distributions which don’t fit this model make the attack sub-optimal or even ineffective in some contexts.

- 2) The Machine learning-based attacks make no assumption on the data distribution and build classifications directly from the raw data sets.

462 A recent line of work has investigated these profiled side-channel attacks based on Machine Learning techniques to defeat both unprotected and protected cryptographic implementations. These contributions focus on the following techniques: Support Vector Machine (SVM), Random Forest (RF), Neural Network [1, 2, 3, 4, 5, 6, 7] and on Deep learning attacks [14]. Deep learning is a branch of machine learning which relies on sets of algorithms that attempt to model high-level abstractions in data by using model architectures with multiple processing layers, composed of a sequence of scalar products and non-linear transformations called activation functions. Deep learning techniques include Multilayer Perceptron, Convolutional Neural Networks, Stacked Auto-Encoders and more.

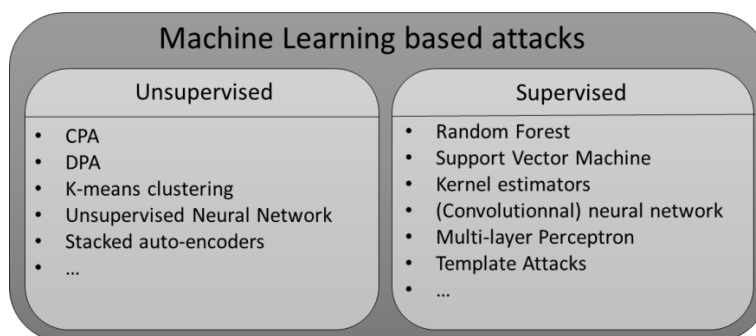


Figure 1: Machine Learning Based Attacks - Classification

463 However, lots of other techniques exist in the machine learning field and could
464 potentially give good results in a side channel context.

2.5.15.1 Rating

464 Compared to “classical” Side Channel Attacks, the rating is very similar. The
465 equipment required for the acquisition phase is exactly the same.

466 Some adjustments could be done for the attack phase.

466 Time:

The required time to mount the attack may be slightly different. In some cases, the computation time may be larger for machine learning techniques while in other cases the attack would need less traces to succeed, so the time may be reduced. This must be taken into account according to the studied case. Also sometimes, time can be saved on trace processing and POI selection when using advanced machine learning techniques (such as neural networks). Generally speaking the time effort will shift from human time to machine time.

467 Expertise:

The expertise remains unchanged compared to classical side channel attacks. Indeed, most knowledge is publicly available and several tools are open source. However, the choice of parameters (methodology, optimizer, meta-parameters ...), may require some expertise during the Identification phase. The expertise criterion should thus stay “Expert”. For the exploitation phase, the required expertise is typical not high and this is expected to remain similar.

468 Knowledge of the TOE:

remains unchanged

469 Access to the TOE:

remains unchanged

470 Equipment:

remains unchanged - except if very large scale computation means are used: in this case the equipment may become “Bespoke”-. There is a trade-off between time and equipment. The cost of machine processing is strongly correlated to its speed. An evaluator should consider the effect of less time with the classification of equipment.

471 Open samples:

Remains unchanged

2.5.15.2 Example of Rating 1

472 Example of a Template Attack targeting an AES implementation handling one byte at a time with a jitter clock as countermeasure:

Factor	Comments	Identification	Exploitation
Elapsed time	The identification will take less than one month. The exploitation phase will take less than one week	< one month (3)	< one week (4)
Expertise	For identification phase, an expert is required. Only a proficient is required for exploitation phase	Expert (5)	Proficient (2)
Knowledge of the TOE	The datasheet is required for the identification phase. Nothing is required for the exploitation phase	Restricted (2)	Public (0)
Open samples / known key	An open sample is required for the identification phase	2	0
Access to the TOE	One sample is enough to mount the attack	< 10 samples (0)	<10 samples (0)
Equipment	A side-channel setup is required for both phases	Specialized (3)	Specialized (4)
Sub Total		17	10
Total		27	

2.5.15.3 Example of Rating 2

- 473 Example of CNN-based attack on a straightforward exponentiation on RSA. The exponentiation is protected using randomization. However a “one trace” matching methodology is applied. First in the identification phase, a CNN is trained on modular blocks in order to distinguish when the exponentiation processes a bit equals to 1 from 0. Then, the exploitation phase consists in distinguishing each bit individually.

Factor	Comments	Identification	Exploitation
Elapsed time	The identification will take less than one week. The exploitation phase will take less than one day	< one week (2)	< one week (3)
Expertise	For identification phase, an expert is required. Only a proficient is required for exploitation phase	Expert (5)	Proficient (2)
Knowledge of the TOE	The datasheet is required for the identification phase. Nothing is required for the exploitation phase	Restricted (2)	Public (0)
Open samples / known key	An open sample (or known secret) is required for the identification phase	2	0
Access to the TOE	One sample is enough to mount the attack	< 10 samples (0)	<10 samples (0)
Equipment	A side-channel setup is required for both phases	Specialized (3)	Specialized (4)

Sub Total		14	9
Total		23	

474 In this case, the proposed rating is exactly the same for a CNN-based attack as for a classical template attack.

2.5.16 Further References

- [1] T. Bartkewitz and K. Lemke-Rust. Efficient Template Attacks Based on Probabilistic Multi-class Support Vector Machines, pages 263{276. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013.
- [2] A. Heuser and M. Zohner. Intelligent Machine Homicide - Breaking Cryptographic Devices Using Support Vector Machines. In W. Schindler and S. A. Huss, editors, COSADE, volume 7275 of LNCS, pages 249{264. Springer, 2012.
- [3] G. Hospodar, B. Gierlichs, E. De Mulder, I. Verbauwhede, and J. Vandewalle. Machine learning in side-channel analysis: a _rst study. Journal of Cryptographic Engineering, 1(4):293{302, 2011.
- [4] L. Lerman, G. Bontempi, and O. Markowitch. Power analysis attack: an approach based on machine learning. International Journal of Applied Cryptography, 3(2):97{115, 2014.
- [5] L. Lerman, R. Poussier, G. Bontempi, O. Markowitch, and F. Standaert. Template attacks vs. machine learning revisited (and the curse of dimensionality in side-channel analysis). In S. Mangard and A. Y. Poschmann, editors, Constructive Side-Channel Analysis and Secure Design - 6th International Workshop, COSADE 2015, Berlin, Germany, April 13-14, 2015. Revised Selected Papers, volume 9064 of Lecture Notes in Computer Science, pages 20{33. Springer, 2015.
- [6] L. Lerman, S. F. Medeiros, G. Bontempi, and O. Markowitch. A machine learning approach against a masked AES. In A. Francillon and P. Rohatgi, editors, Smart Card Research and Advanced Applications - 12th International Conference, CARDIS 2013, Berlin, Germany, November 27-29, 2013. Revised Selected Papers, volume 8419 of Lecture Notes in Computer Science, pages 61-75. Springer, 2013.
- [7] R. Gilmore, N. Hanley, and M. O'Neill. Neural network based attack on a masked implementation of AES. In Hardware Oriented Security and Trust (HOST), 2015 IEEE International Symposium on, pages 106-111, May 2015.
- [8] R. O. Duda, P. E. Hart, and D. G. Stork. Pattern Classi_cation (2nd Edition). Wiley-Interscience, 2000.
- [9] J. Heyszl, A. Ibing, S. Mangard, F. D. Santis, and G. Sigl. Clustering Algorithms for Non-Pro_led Single-Execution Attacks on Exponentiations. IACR Cryptology ePrint Archive, 2013:438, 2013.

- [10] C. Archambeau, E. Peeters, F.-X. Standaert, and J.-J. Quisquater. Template Attacks in Principal Subspaces. In CHES, volume 4249 of LNCS, pages 1-14. Springer, October 10-13 2006. Yokohama, Japan.
- [11] S. Chari, J. R. Rao, and P. Rohatgi. Template Attacks. In CHES, volume 2523 of LNCS, pages 13-28. Springer, August 2002. San Francisco Bay (Redwood City), USA.
- [12] P. C. Kocher, J. Jaeger, and B. Jun. Differential Power Analysis. In CRYPTO, volume 1666 of LNCS, pages 388-397. Springer, 1999.
- [13] E. Brier, C. Clavier, and F. Olivier. Correlation Power Analysis with a Leakage Model. In CHES, volume 3156 of LNCS, pages 16-29. Springer, August 11-13 2004. Cambridge, MA, USA.
- [14] H. Maghrebi, T. Portigliatti and E. Prouff. Breaking Cryptographic Implementations Using Deep Learning Techniques. ePrint Archive, Report 2016/921, <http://eprint.iacr.org/2016/921>.
- [15] E. Cagli, C. Dumas, E. Prouff. Convolutional Neural Networks with Data Augmentation against Jitter-based Countermeasures – Profiling Attacks without Pre-Processing. In CHES, volume 10529, pages 45-68. Springer, September 25-28, 2017. Taipei, Taiwan.
- [16] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz and Yuval Yarom. Spectre Attacks: Exploiting Speculative Execution. In 40th IEEE Symposium on Security and Privacy (S&P'19), 2019.
- [17] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom and Mike Hamburg. Meltdown: Reading Kernel Memory from User Space. In 27th USENIX Security Symposium (USENIX Security 18), 2018.

2.6 Exploitation of Test Features

2.6.1 Description of Attack

- 475 The attack path aims to enter the IC test mode to provide a basis for further attacks.
- 476 These further attacks might lead for example to disclosure or corruption of memory content, a change in the lifecycle state, or deactivation of security features. But as this depends on the possibilities of the test mode, the details about those further attacks are not considered here.

2.6.2 Effect of Attack

- 477 The attacker is able to circumvent the protection of the test features and can use the test interface and test functions as appropriate for the intended attacks.

2.6.3 Impact on TOE

- 478 As result of successful access to the IC test mode , the attacker might be able to:
- Read out the content of the non-volatile memory using test functions. The implementation of the test functions may have an impact on the usability of the retrieved user data.
 - Re-configure lifecycle data or error counters using a test function. Thereby an attacker is able to extend his analysis on the same device, even when a lifecycle status change would otherwise have stopped him.

2.6.4 Characteristics of the Attack

479 The mechanisms used to protect the test features are different for every IC manufacturer. The protection comprises a set of different mechanisms that are not described in the user documentation (Data Sheet). Therefore, reverse engineering and testing are required to analyse the mechanisms.

480 The usage of the test mode by an attacker may require him to:

- physically manipulate the IC or
- break an authentication mechanism or
- apply a perturbation.

481 For state-of-the-art smart cards a combination of these attacks will generally be required. In addition, the functions of the test interface provided depend on the test concept and need further analysis before they can be used for exploitation.

2.6.5 Example: Attack against test mode with physical fuse

2.6.5.1 Attack path

482 In this example the test mode is protected by a physical fuse in the scribe line between the chips on the wafer, which is destroyed during wafer sawing. The attacker reconnects the physical fuse to gain access to the test mode.

483 The attack then involves the following steps:

- 1) Reconnecting the physical fuse.

2.6.5.2 Example rating without countermeasures

484 In this example it is assumed that the fuse can be reconnected without using a FIB. After reconnecting this fuse the IC is in test mode again. No additional countermeasures (like logical access control) are implemented.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the identification of the fuse and the reconnection takes more than one hour but less than 1 day. After the fuse is identified the exploitation can be done in less than an hour.	< 1 day (1)	< 1 hour (0)

Expertise	A Proficient rating is required for identification and exploitation, because knowledge about principles of test mode protection are needed.	Proficient (2)	Proficient (2)
Knowledge of TOE	The attack requires restricted information regarding the fact of fuse protection. For the exploitation, the information is assumed to be published and therefore public.	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Only Standard equipment is required to perform this attack. The required equipment for identification and exploitation is the same.	Standard (1)	Standard (2)
Sub Total		6	4
Total		10	

485 This TOE would receive no rating for this attack with CC v3.

2.6.5.2.1 Required demonstration of attack to justify this rating

486 The attack is judged to be successful when the Test Mode has been accessed, as demonstrated by successful execution of test commands.

2.6.6 Example: Attack against test mode with lifecycle control

2.6.6.1 Attack path

487 In this example, an attacker applies a laser pulse to a TOE while the lifecycle information check is carried out during the start-up phase of the TOE. The attacker continuously resets the TOE until access to test mode is gained. Details of the implementation (e.g., of the exact timing) of the lifecycle information check is not required since the laser can be applied at different times during the check.

488 The attack then involves the following steps:

- 1) Remove the IC from its packaging, de-processing down to the passivation layer (this may need chemical etching). Connect the IC to a test setup that continuously resets the TOE in order to cause the lifecycle information check to be carried out and check if test mode access is gained.
- 2) Repeatedly apply laser pulses while lifecycle information check is performed and monitor the results. It is assumed that the attacker cannot immediately identify individual components of the logic design to attack (although certain targets such as memory blocks and their interfaces will inevitably be visible). Therefore the position of the laser attack has to be scanned over the entire IC surface.

489 For this example, knowledge of the TOE, and in particular of the implementation of the TOE software, does not make a significant difference to the time taken to find a suitable

point at which to apply the perturbation. The identification part of attack potential calculations using [JIL-AP] assumes only knowledge about the fact that the test mode is protected by a lifecycle information check.

490 The rating of the attack is very similar to the rating of the authentication bypass attack in the Perturbation Attacks section.

2.6.6.2 Example rating 1 without countermeasures

491 An attack against a simple test mode protection (e.g. by irreversibly set lifecycle control information) without any integrity control.

Assumptions on the IC:

- The test interface uses the standard IC pads, i.e. no special preparation of tests pads is required.
- No restrictions of the test interface exist, e.g., test commands require the same protocol as commands during normal operation and attacks to test programs are assumed to lead to similar results as those of standard perturbation attacks.

Countermeasures of the IC:

- Light sensors

492 It is assumed that obtaining a perfect result (i.e. continued operation of the TOE as access to test mode is gained) does not occur on each attempt, and hence the exploitation may not be immediate. However, in most cases repeating the actual attack, once the location and timing for the perturbation has been identified, will require substantially less than a day.

493 The HW countermeasure “light sensors” is required as otherwise a simple flash light may be used for mounting the attack.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the de-processing and initial testing with demonstrating the vulnerability takes more than 1 day but less than one week to find the right combination of trigger and location. We assume exploitation based on a written description of the attack and the timing and location to use.	< 1 week (2)	< 1 day (3)
Expertise	A Proficient rating is required for identification and exploitation, because de-processing techniques and equipment operation are required.	Proficient (2)	Proficient (2)
Knowledge of TOE	The attack requires Restricted information to analyse the way of test mode protection.. For the exploitation, the information is assumed to be published and therefore public.	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required	NA	NA

Equipment	Minimum requirement is equipment to deprocess the chip. All equipment used for the perturbation (laser, delay generator, etc) as well as optical microscope and digital oscilloscope for the analyses are specialised equipment. The required equipment for identification and exploitation is the same.	Specialized (3)	Specialized (4)
Sub Total		9	9
Total		18	

494 This TOE would be resistant for this attack to an attacker with and basic attack potential (VAN.2) with CC v3.

2.6.6.2.1 Required demonstration of attack to justify this rating

495 The attack is judged to be successful when the Test Mode has been accessed, as demonstrated by successful execution of test commands.

2.6.6.3 Example rating 2 with countermeasures

496 An attack against a simple test mode protection (e.g. by irreversibly set lifecycle control information) without any integrity control.

Countermeasures of the IC:

- Light sensors
- Dedicated test interface (destroyed by sawing the wafer into individual ICs)
- Proprietary test protocol

497 It is assumed that obtaining a perfect result (i.e. continued operation of the TOE as access to test mode is gained) does not occur on each attempt, and hence the exploitation may not be immediate. However, in most cases repeating the actual attack, once the location and timing for the perturbation has been identified, will require substantially less than a day.

498 The HW countermeasure “light sensors” is required as otherwise a simple flash light may be used for mounting the attack.

499 The preparation of test pads requires additional time and equipment (FIB).

500 The ‘proprietary test protocol requires additional knowledge of the TOE (not available in the user guidance).

Factor	Comment	Ident’n	Exploit’n
Elapsed Time	We assume the de-processing and preparation of tests pads to the test interface combined with demonstrating the vulnerability takes more than 1 day but less than one week to find the right combination of trigger and location. We assume exploitation based on a written description of the attack and the timing and location to use. Nevertheless the pads for the test interface have to be prepared.	< 1 week (2)	< 1 day (3)

Expertise	An Expert rating is required for identification and exploitation because an understanding of the proprietary protocol and chip preparation techniques is required.	Expert (5)	Expert (4)
Knowledge of TOE	The attack requires Restricted information to analyse the way of test mode protection but Sensitive information about the proprietary protocol. For the exploitation, the information is assumed to be published and therefore public.	Sensitive (4)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Minimum requirement is equipment to deprocess the chip. All equipment used for the perturbation (laser, frequency generator, etc) as well as optical microscope and digital oscilloscope for the analyses are specialised equipment. For preparation of the test pads an FIB is required. This is rated as bespoke. The required equipment for identification and exploitation is the same.	Bespoke (5)	Bespoke (6)
Sub Total		16	13
Total		29	

501 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4) with CC v3.

2.6.6.3.1 Required demonstration of attack to justify this rating

502 The attack is judged to be successful when the Test Mode has been accessed, as demonstrated by successful execution of test commands.

2.6.7 Example: Enter test mode protected by access code with error counter

2.6.7.1 Attack path

503 The assumption is that the access code is so short that it could be broken by exhaustive search. To prevent an exhaustive search the number of attempts is limited by an error counter. It is also assumed that the error counter is increased prior to a access code comparison and only reset after a successful comparison. The access code is assumed to be identical for all instances of this type of device.

504 The attacker uses laser pulses to perturb the TOE while the error counter is tested for its maximum value. Then the attacker forces a decision that allows him to continue to the access code comparison. This allows him to repeat the procedure until he has ascertained the correct access code. Now he can use this access code to access test mode on any device of this type.

505 The attack then involves the following steps:

- 1) Remove the IC from its packaging, de-processing down to the passivation layer (this may need chemical etching) or lay open the substrate of the IC (the laser light can be applied from the top or the back of the IC). Connect the IC to a test harness that repeatedly sends access code verification commands to the IC.
 - 2) Monitor the power traces during the execution of the command and identify distinct sequences of the command. Use characteristics in the power traces as a trigger for the laser pulse. Knowledge of the TOE, and in particular of the implementation of the TOE software, may make a significant difference to defining the right trigger; alternatively expert knowledge for the interpretation of the power traces and longer time for trial and error has to be calculated.
 - 3) Repeatedly apply laser pulses at different locations while operating the IC and monitor the results. It is assumed that the TOE uses randomised logic and hence that the attacker cannot immediately identify individual components of the logic design to attack (although certain targets such as memory blocks and their interfaces will inevitably be visible). If the light sensors are well implemented, the area covered by the laser will have to be very small (of the order of 10~50 μm square). However, in most cases, a light sensor reset will be caused. Therefore, the time to discover the areas where the TOE can be forced to make errors will be considerable.
 - 4) Laser pulses at combinations of trigger and location have to be applied and the reaction has to be sampled to determine whether test mode has been activated. Due to the randomised internal code execution, the trigger cannot be defined at a fixed position during the execution time but has to be derived from a pattern recognition algorithm (or trial and error).
 - 5) Once test mode has been entered, a search for useful commands has to be conducted.
- 506 For this example, knowledge of the TOE, and in particular of the implementation of the TOE software, may make a significant difference to the time needed for finding a suitable point at which to apply the perturbation. In this example, the identification part of the attack potential calculations using [JIL-AP] assumes only knowledge sufficient to operate the TOE in ways that the attacker can repeat the same processing in order to develop the attack in terms of where and when to apply the perturbation.
- 507 It is assumed that obtaining a perfect result (i.e. allowing an access code verification although the error counter has reached its limit) does not occur on every attempt, and hence the exploitation may not be immediate. However, in most cases repeating the attack, once the location and timing for the perturbation has been identified, will require less than a month but is strongly dependent on the PIN length.

2.6.7.2 Example with countermeasures

Countermeasures of the IC:

- Randomised location of components
- Light sensors
- Hardware ECC for memories

- Randomised internal execution

508 The attack potential calculation is then as is shown in the table below:

Note that these values are given for what is considered an average product. For especially good or bad implementations, the rating might be different. Nonetheless, the rating gives a baseline.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the de-processing and initial testing to identify the failure combined with demonstrating the vulnerability takes more than 1 month. To find the right combination between trigger and location might be performed within on day but the extensive brute force search for the correct access code takes a long time. We assume exploitation based on a written description of the attack, location to use, and known access code. Due to the countermeasure "Randomized internal execution" an exact definition of the timing cannot be defined.	>1 month (5)	< 1 week (4)
Expertise	To find the right combination of trigger and location the attacker needs expert knowledge. He has to analyse the power traces and define the pattern recognition. A Proficient rating is required for exploitation, because of the de-processing techniques and equipment operation required.	Expert (5)	Proficient (2)
Knowledge of TOE	The attack requires Restricted information to identify exploitable commands, but the command (including the access code) is assumed to be scripted for the exploitation phase (hence Public).	Restricted (2)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 Samples (0)	< 10 Samples (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Minimum requirement is equipment to de-process the chip and then to generate and analyse the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		15	10
Total		25	

509 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4) with CC v3.

2.6.7.2.1 Required demonstration of attack to justify this rating

510 It is not enough for this attack to show that a perturbation can be caused. The test mode has to be accessed and test mode commands executed.

2.7 Attacks on RNG

2.7.1 Description of Attacks

511 Attacks on RNGs aim in general to get the ability to predict the output of the RNG (e.g. reducing the output entropy) which can comprise:

- past values of the RNG output (with respect to the given and possibly known current values),
- future values of the RNG output (with respect to the possibly known past and current values),
- forcing the output to a specific behaviour, which leads to:
 - known values (therefore also allowing for the prediction of the output),
 - unknown, but fixed values (reducing the entropy to 0 at the limit),
 - repetition of unknown values either for different runs of one RNG or for runs of two or more RNGs (cloning) .

512 An RNG considered here can be one of the following types²⁴:

- true RNGs (TRNG), the output of which is generated by any kind of sampling inherently random physical processes,
- pseudo RNG (PRNG) the output of which is generated by any kind of algorithmic processing (the algorithm is in general state based, with the initial state (seed) generated by a TRNG),
- hybrid RNG (HRNG), which consists of a TRNG and a PRNG with a variety of state update schemes.

513 The applicable attack methods vary according to the type of RNG:

514 A true RNG may be attacked by²⁵:

- permanent or transient influence of the operating conditions (e.g. voltage, frequency, temperature, light)
- non-invasive exploitation of signal leakage (e.g. signal on external electrical interfaces)
- physical manipulation of the circuitry (stop the operation, force the line level, modify and/or clone the behaviour, disconnect entropy source)
- wire-tapping internal signals (compromise internal states).

515 A pseudo RNG may be attacked by:

- direct (cryptographic) attack on the deterministic state transition and output function (e.g. based on known previous outputs of the RNG)

²⁴ In the context of smart cards the RNG based on some measurements of environment are not considered to be relevant.

²⁵ It is here assumed that the direct attack on a true RNG (i.e. guessing the value) is not feasible for any attacker.

- indirect attack on the state transition computation process by employing some side channel information (i.e. leakage on external electrical interfaces)
- attack on the execution path of the processing (modification of the results)
- attack on the seed (prevent reseeding, force the seed to fixed known or unknown (but reproducible) value, compromise the seed value)
- exceed the limit of RNG output volume (e.g. forcing the RNG to repeat values or to produce enough output to enable the attacker to solve equations and based on the solution to predict the output)

516 The attacks on hybrid RNGs will be in general a combination of attacks on TRNGs and PRNGs.

517 All RNG designs can be expected to incorporate test procedures to counter attacks like those listed above. The analysis above does not take attacks on test procedures into account, as such attacks will be covered sufficiently by the more general attack scenarios on software. Note that test procedures may be subject to attacks such as SPA/DFA to reveal the RNG output values.

2.7.2 Effect of Attacks

518 The output of the RNG is random by design. The deviation of the behaviour of the RNG from the specified behaviour may or may not be observable directly depending on the usage of the RNG. Reduced entropy of the RNG output used to generate secret keys may not be detectable by any practical means when, for example, only an access to a limited amount of encrypted data (with the above keys) is available. Depending on the strength of the post-processing of the TRNG output (also relevant in case of a hybrid RNG) it can also be infeasible to detect and to misuse the total failure of the TRNG. On the other side the successful forcing of the RNG output to some fixed value can become easily observable if this output is used for example as a challenge in an authentication protocol. Furthermore a known bias of the RNG may make a key search easier, depending on the algorithm used. There are no universal criteria for how to recognise directly when an attack on an RNG has succeeded.

2.7.3 Impact on the TOE

519 A successful attack on the RNG will result in breaching the security mechanisms of the chip, which rely on the randomness of the RNG. The mechanisms may be DPA/SPA countermeasures, sensor testing, integrity checking of active shield, bus and/or memory encryption and scrambling. The application software is affected by such attacks indirectly, e.g. if sensors and related tests are disabled by an attacker then this will generate further attack possibilities.

520 The software developer can rely on the capabilities of the hardware platform for testing the RNG and make use of them or implement and perform additional tests based on such capabilities. The software developer may also implement tests for repetition of RNG output, but the coverage and feasibility of such tests may depend on the implementation and may be a problem. The cloning attack for RNG output on different

instances of an RNG cannot be countered by tests, so other mechanisms must be designed as appropriate.

- 521 In the case of TRNGs, sufficient tests should be performed either by the chip platform itself or by the software developer. [AIS31] is an example of a methodology for assessing the effectiveness of the testing mechanisms. In the case of a PRNG, special effort for protecting the seed and the algorithm in terms of integrity and confidentiality is required. This effort relates to general software and data protection aspects and will be not discussed further in this section.

2.7.4 Characteristics of the Attack

- 522 The basic attack step on a TRNG, which aims to change the statistical properties of the generated TRNG output, may require only standard equipment like power supplies, signal generators and light sources. As a result it is possible that the statistical properties are such that the RNG output may be easily predicted (i.e. in a case that the RNG output is forced to a constant value), or that they are changed only slightly, i.e. the entropy is lowered by small amount or spurious correlations are induced. It is expected that in the latter case the attacker must perform extensive identification and/or exploitation effort to be successful with the attack. Note that the self-test of the RNG may be effective (i.e. the attacker would also have to bypass or be undetectable to the testing procedure in order to be successful).
- 523 Measurement of signal leakage requires standard equipment. The interpretation of measured signals will require further signal processing. The attacker may succeed in disclosing the RNG output or in getting some correlated data. With such knowledge, the attacker may then start attacks on the security functionality that uses the RNG output (e.g. the knowledge of the RNG output signal may be used in DPA type attack on cryptographic functions).
- 524 Physical manipulation requires some reverse engineering effort, which can be performed only with at least basic knowledge on common RNG principles. The amount of such effort depends strongly on the current implementation. Then the identified circuitry can be manipulated by measures, which may range from needles to a laser cutter and FIB. Successful manipulations will at least turn the RNG off, so that the mechanisms relying on the RNG become ineffective. On the very high end of the attack effect, the attacker may succeed in replacing the true RNG output by any sequence of his choice (i.e. repeating of a sequence, inducing a pseudo-random sequence with a known seed, etc.). Such attacks could remain undetected by, for example, (statistical) self-tests performed in software.
- 525 Wire-tapping is comparable in reverse engineering effort to manipulation attack, while the effects and the further effort are similar to that of the leakage attack.
- 526 A direct (cryptographic) attack requires in general (i.e. when the RNG construction is not obviously trivial) a huge expertise and computing power. In addition the attacker must be given access (or the attacker must gain this access to prepare for an attack) to a sufficient amount of RNG output data.
- 527 An indirect attack on the state transition computation by exploitation of side channels will generally require access to design details of the state transition algorithm, and preparation of analysis tools. The measurement of leaking signals may be then performed with standard equipment. The attacker may be then able either to

- compromise the RNG output values or to compromise the state of the RNG. The latter would allow compromising the past and the future values of the RNG output.
- 528 The attack on the execution path of the RNG processing does not differ significantly from any other attack on the execution path of the software and/or hardware. Therefore, the attack potential characteristics are the same. To be successful the attacker is expected to perform reverse engineering on the RNG software. For example, he has to identify the opportunities to prevent the update of the RNG state (e.g. by removing power at the right time).
- 529 The attack on the seed is considered to be related to the more general attack scenarios on memory, register contents and bus transfers. This pertains to both the compromise and modification of the content. The attacker is expected to perform reverse engineering on the RNG processing in order to be able to exploit his knowledge of the seed.
- 530 To overcome the limit of the RNG output based on the particular seed value the attacker will in general perform an attack similar to that already discussed for attacking the state transition flow.
- 531 The following is the agreed list of obvious attacks:
- exploitation of statistical properties of the RNG output depending on environmental conditions,
 - leakage tests,
 - physical manipulations aiming in modifying and/or wire-tapping the RNG circuitry of both TRNG and PRNG,
 - direct (cryptographic) attack of PRNG,
 - attack on the execution path of the PRNG state transition process and/or output amount limitation mechanism,
 - attack on the seed aiming to manipulate and/or compromise the seed of PRNG.

2.7.5 Example: attack on True Random Number Generator

2.7.5.1 Attack path

- 532 A straightforward attack on a true RNG (TRNG) is the destruction of parts or the disconnection of lines within the TRNG. The tools required for identification include microscope, etching tools and micro-probe workstation. For the exploitation, only some of these tools, e.g. the laser cutter, may be required. In this example an attack leads to a constant output of the true random number generator.
- 533 The identification of components of the true random number generator requires reverse engineering including de-layering of the hardware platform. In most cases, at least parts of the true random number generator can be detected. Probing or other measurements can be used to validate the results of the reverse engineering. As a result the output of the TRNG can be fixed or probed.

2.7.5.2 Example Rating 1 without specific Countermeasure

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	The reverse engineering to locate the RNG and disconnect the relevant line requires more than a week but less than a month	< 1 month (3)	1 day (3)
Expertise	Since different implementations are possible that attacker must know basic principles of the RNG implementation to be able to detect them. Since the design is considered to be non-obvious the attacker must be able to adapt his knowledge for the implementation.	Expert (5)	Proficient (2)
Knowledge of TOE	At least the access to the RNG using application specific commands must be known to run the RNG for identification purpose and to validate a successful attack.	Restricted (2)	Public (0)
Access to TOE	Although the reverse engineering requires the destruction of about three samples the complete application of the attack is considered to be successful with less than 10 sample based on the reverse engineering.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	Microscope, etching tools and micro-probe workstation. Exploitation needs a laser cutter.	Specialized (3)	Specialized (4)
Sub Total		13	9
Total		22	

534 This TOE would be resistant for this attack to an attacker with enhanced-basic attack potential (VAN.3) with CC v3.

2.7.5.2.1 Required demonstration of attack to justify this rating

535 The attack is successful when it can be proven that the random numbers obtained do not meet the specified randomness metric.

2.7.5.3 Example Rating 2 with Countermeasures**Countermeasures of the IC (taken from the countermeasure list):**

- Active shield (complex implementation)

536 In the following example, an active shield covers the TRNG requiring an adaptation of the attack and increases the attack potential rating. Before the attack to the TRNG can

be applied, the active shield must be circumvented. It is assumed that a FIB is required to disable parts of the active shield.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	The identification and exploitation depends on the complexity and implementation of the active shield. The exploitation requires more steps.	< 1 month (3)	1 week (4)
Expertise	Since different implementations are possible, the attacker must know basic principle of the RNG implementation to be able to detect them. Since the design is considered to be non-obvious the attacker must be able to adapt his knowledge for the attacked implementation	Expert (5)	Proficient (2)
Knowledge of TOE	At least the access to the RNG using application specific commands must be known to run the RNG for identification purpose and to validate a successful attack.	Restricted (2)	Public (0)
Access to TOE	Although the reverse engineering requires the destruction of about three samples the complete application of the attack is considered to be successful with less than 10 sample based on the reverse engineering.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	The FIB must be aligned with the design to be able to perform the manipulation. It is assumed that complex manipulation is required because of the active shield and the deeper layers involved.	Bespoke (5)	Bespoke (6)
Sub Total		15	12
Total		27	

537 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

538 The attack potential rating may be judged "not practical" if additional countermeasures (e.g. a test of the random numbers) are implemented in the software, provided that these cannot be disabled, bypassed or defeated in other ways for the purposes of the attack.

2.7.5.3.1 Required demonstration of attack to justify this rating

539 The attack is judged successful when it can be proven that the random numbers do not meet the specified randomness metric.

2.7.5.4 Other Attack Scenarios

- 540 The wire-tapping of random numbers is also considered as a possible attack. The rating for this attack may be different from the rating mentioned above. For smart cards used within machines (e.g. payment systems where random numbers are used for challenge response protocols or the generation of session keys) this attack may not be exploitable because of other properties of the protocols used. If smart cards are used to generate private/public key pairs internally then this attack must be considered.
- 541 Manipulation of the random number generator output using perturbations must be considered. The perturbation may influence either the content of registers that are part of the random number generator output or the behaviour of source signals for the random number generator. The perturbation of registers used in a random number generator is covered by the other perturbation attacks since the behaviour is considered to be the same. The influence of the source signals strongly depends on the implementation. Therefore, a general attack description is not possible.

2.7.6 Example: attack on Pseudo Random Number Generator

2.7.6.1 Attack path

- 542 A basic attack on a pseudo random number generator tries to force the input parameters of the pseudo random number generator stored in the memories to known values. For example, it may be possible to force seed values in memory or in an internal register to be all 1's using a laser. This attack is considered as a perturbation attack on a software sequence.

2.7.6.2 Example rating without countermeasure

- 543 This attack relies on using laser perturbation to make a specific change to the seed value for the PRNG. The attack is therefore based on that described in section 2.3 perturbation, but with the different objective here of altering the seed either permanently (e.g. in EEPROM) or temporarily (e.g. in an internal CPU or PRNG register).

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the deprocessing and initial testing to identify the failure take more than 1 week, because of the need to locate the seed value and achieve a specific alteration to a known value. Having achieved this result however, it is assumed that the method can be scripted and that exploitation then takes less than one further week (assuming the restricted command information is available).	< 1 month (3)	< 1 week (4)
Expertise	It is assumed that the attacker can recognise the fault by simple observation of an output message, so a Proficient attacker can do the identification. A Proficient rating is also required for exploitation, because of the de-processing techniques and equipment operation required.	Proficient (2)	Proficient (2)
Knowledge of	The attack requires Restricted information to	Restricted	Public

TOE	identify exploitable commands, but the command is assumed to be scripted for the exploitation phase (hence Public).	(2)	(0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 (0)	< 10 (0)
Open Samples / Known Key	Not required	NA	NA
Equipment	The minimum requirement is equipment to de-process the chip, bond out the pads, and then to generate and analyse the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing.	Specialized (3)	Specialized (4)
Sub Total		10	10
Total		20	

544 This TOE would be resistant for this attack to an attacker with basic attack potential (VAN.2) with CC v3.

2.7.6.2.1 **Required demonstration of attack to justify this rating**

545 The attack is judged successful when it can be proven that the seed has been set to a known value (this might be direct reading of the seed value if possible or perhaps by observation of repeated pseudo-random number values).

2.8 Software Attacks

2.8.1 Introduction

546 Most of the examples of attacks in this document require hardware attack steps for all or
part of the attack. However, it is clear that there are many relevant attacks that can be
made on software alone. This section considers some of these attacks. In many cases
software attacks start with either source code analysis or extensive software testing.
Both are usually combined for more efficiency on the coverage of vulnerabilities
detection.

547 In general, it is important to note that most software attacks arise from:

- errors (bugs) in the TOE, either in design or implementation;
- inconsistency, holes or ambiguity in the specification or standards;
- exploitation of sensitive or critical knowledge obtained on the TOE.

548 In the case of errors (bugs), it will generally result in a failure to meet the requirements
of one (or more) of the ADV families (e.g. ADV_IMP.1.2E: The evaluator shall
determine that the least abstract TSF representation provided is an accurate and
complete instantiation of the TOE security functional requirements). Hence, an error of
this sort will cause the TOE to fail evaluation (or, more usually, will require a
modification to the TOE to correct the error).

549 In the case of issues coming from the specification or standards, the modification of the
design's specification may be insufficient to meet the TOE security objectives: for
example, a protocol specification might itself contain critical vulnerabilities. This would
also cause a TOE to fail the evaluation.

550 In the case of exploitation of knowledge of the TOE, the attacker may have access to
authentication data for example, opening the usage of some features only accessible for
the developers. For example, the attacker may use proprietary administration commands
requiring authentication.

551 This section therefore lists a number of attack steps that may be used to discover
software errors, but few attack potential examples are given, since, in several cases but
not all the time, if any error is discovered then it must be corrected if the TOE is to pass
evaluation.

552 In the text below we consider first an information gathering attack step, which may be
relevant to a number of different types of attack. We introduce five specific attack
techniques that may exploit software vulnerabilities:

- Information gathering on commands
- Direct protocol attacks
- Man-in-the-middle and replay attacks
- Buffer overflow or stack overflow
- Communication interface switching
- Firewall bypass

553 The attacks are of a logical nature, to perform such attacks, it is necessary to have:

- a means to listen to message sequences (reader, traffic analyser)
- a means to create messages (information on external API, pattern generator)

- a means to interrupt messages without detection (protocol dependent)
- a means to analyse the source code with a tool
- a means to create applications
- a means to build and run larger test suites

554 So the test environment may consist of :

- A PC
- a smart card reader
- a test software for test scripts writing and execution and communicating with the smart card
- a source code analysis tool (for white box testing or memory dump analysis)
- a protocol analyser (for reverse engineering of communication protocols)
- a development environment (for development of applications to load on the TOE)

555 Though setting up a test environment and identifying an attack seems simple, as the tools are considered to be standard equipment (some software tools are even available as freeware on the internet) and the commands are often ISO standard and therefore public knowledge, some tools usage and interfacing with them to build test scripts may require some significant set-up time. If the command set is proprietary, the expertise needed is slightly higher because the communication must be interpreted. However, if the security level is based on 'security by obscurity', it would not be considered a valid defence against attack.

556 The expertise of the attacker could be proficient or expert, and may be multiple expert, especially when combining very precise area of expertise (Java Card and cryptography for example).

557 In the rest of the section, different type of software attacks techniques are described and have to be considered as elementary building bricks: usually, a full attack path is a combination of the different techniques.

2.8.2 Software attack detailed objective

558 This type of attack aims for example at getting unauthorised access to data residing on the smartcard respectively at performing operations which do not match the current lifecycle state of processed data objects or of the Operating System. In particular, unauthorised reading or modification of personalisation data stored on the card, or a further (unauthorised) initialisation or personalisation of the product, could be the target of such an attack scenario. This type of attack (which may also be whole program sequences) makes use of weaknesses in software implementation and is performed by a logical or physical attack on the Operating System and its processed data. The attack techniques used are protocol attacks, such as man-in-the-middle, replay, command editing or using commands which are undefined or not allowed in the current lifecycle state of the Operating System. Furthermore, logical and physical attacks manipulating the program flow, status information (such as the lifecycle state of objects and of the Operating System) and access rules for objects processed by the Operating System are other attacks means, detailed in the rest of the document, but not considered in this section.

2.8.2.1 Description of Attack

559 This type of attack aims to get unauthorised access to data residing on the smartcard to perform operations which do not match the current lifecycle state of processed data objects or of the Operating System. As an example, such an attack aims to read or modify personalised data that resides on the card or aims to perform a further (unauthorised) initialisation or personalisation of the product.

560 Getting unauthorised access to data stored on the smartcard can be obtained by various techniques:

- Impersonating the other side of the communication (known as ‘man-in-the-middle’),
- using timing differences (by capturing and replaying commands),
- trying command variations (either editing valid commands or finding undefined commands),
- manipulation of access rules themselves,
- circumvention or manipulation of the request and evaluation of access rules during program execution.

561 Executing commands that are not allowed in the current lifecycle state of the Operating System or of a data object can also be obtained by various techniques:

- manipulation of the current lifecycle state itself,
- circumvention or manipulation of the request and
- evaluation of the current lifecycle state during program execution, and
- trying command variations (either editing valid commands or finding undefined commands).

2.8.2.2 Effect of Attack

562 The effect of the attack is unauthorised access to data residing on the smartcard and the possibility to perform operations which do not match the current lifecycle state of the Operating System or of data objects processed by the Operating System. In particular, such an attack could lead to the disclosure of stored secret data or to a further (unauthorised) initialisation or personalisation of the product. The attacker knows the attack has succeeded by analysing the answers the smartcard gives during subsequent communication.

563 In general, the described attack scenario aims at the manipulation of the intended security structure integrated in the Operating System, in the applications set up on this Operating System and in the (application) data processed by the Operating System. The integrated access control to data objects and commands is affected.

564 Replay attacks have existed for a long time. Years ago, replay attacks were aimed at stealing passwords. Given the encryption strength of passwords these days, the focus of this type of attack has shifted to stealing digital signatures and keys.

565 The command editing attack aims to find commands that are not documented or to use valid commands in a way that breaks the communication mechanisms in the TOE. The attacker may try to find improper bounds checking by sending longer commands than the TOE expects. He may try to send commands with unexpected values, forcing the smartcard to dump memory contents.

566 The manipulation of lifecycle state information and access rules themselves, and the

manipulation of their request and evaluation can be considered as a direct attack on the access control implemented in the Operating System and the applications running on this platform. In particular, the access control is modified or completely switched off in a way that unauthorised access to secured data or the execution of disallowed commands is possible.

2.8.2.3 Characteristics of the Attack

- 567 The manipulations of lifecycle state information and access rules require a logical attack on the smartcard and its Operating System and applications. The circumvention and manipulation of the request and evaluation of lifecycle state information and access rules is based on a manipulation of the intended program flow that may be achieved by logical means (physical means are not considered in this example). An active countermeasure for securing lifecycle state information and access rules and their request and evaluation during program execution could be to attach an integrity attribute and to check this attribute appropriately during program execution.
- 568 The logical attacks such as direct protocol attacks, man-in-the-middle attacks, replay attacks, and buffer overflow, communication interface switching and firewall bypass are considered in detail in the rest of this section.

2.8.3 Information gathering on commands

2.8.3.1 Overview

- 569 By their nature, communication protocols are susceptible to information leakage. This unwanted effect is a consequence of the fact that they are designed to pass information. This type of attack tries to use the protocols in ways that were not intended by the protocol developer, by first gathering information and then changing that communication to obtain secret data or other resources.
- 570 The attack step is usually a non-invasive technique, with the aim of getting information on the communication commands that the smartcard supports or using information from message sequences to enable other attacks. It is noted that the information is assumed to be information not contained in design documents (e.g. undocumented responses to commands). This information may then enable the attacker to modify the interaction or to disclose information (e.g. user data or keys) using weaknesses in the software implementation. This attack step is normally not a full attack path leading to the retrieval of secret data, although it might do in specific cases (exposure of secret data in this way would generally be considered a sufficient vulnerability to cause the TOE to fail evaluation²⁶).
- 571 This attack step results in gathering information on the operation of the TOE. The information gathered is analysed to see whether it can be used to mount an attack to retrieve secret data from the TOE with one of the other mechanisms described in this document. The attacker knows the attack has succeeded by analysing the answers the smartcard gives during the communication.

²⁶ Depending on the scope of the evaluation and the environment, there may be some situations where such information exposure is accepted, e.g. in a protocol for use only in secure personalisation environments.

2.8.3.2 Attack Step Descriptions

2.8.3.2.1 Observing Message Sequences

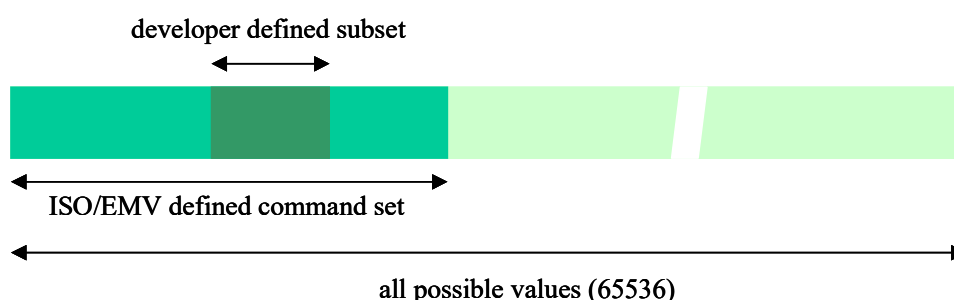
572 Observing message sequences may result in:

- obtaining information on an unknown protocol (e.g. where the interface specification is not public) to prepare an attack
- obtaining information on unknown internal product structures (typically data structures in software) to prepare an attack
- disclosing information, keys, or security attributes during import or export operations
- tracing product activity or user behaviour (e.g. to enable a replay attack).

573 Such observation is only possible when intercepting a valid communication between a smart card and a terminal. If the attacker don't have such possibility, he has to proceed with the next step, "Command searches".

2.8.3.2.2 Command searches

574 The total amount of values that a smartcard can communicate using a typical protocol such as ISO 7816 T=1 is 2^{16} , or 65536 different commands. Of this set, ISO defined a subset as being valid commands. And of this ISO set, a developer defines a subset and documents these commands as being valid commands for this card.



575 A T=1 test plan may contain the following tests:

- A 'brute force' approach in which all values outside the ISO defined set are tried and it is checked whether the card responds (inopportune behaviour).
- A 'brute force' approach in which all values of the ISO defined set, but outside the developer defined set are tried for a response (undocumented command search).
- Trying all developer documented commands and checking the answers.
- Trying all developer documented commands, but with emphasis on limit cases and multiple error cases.
- Influencing the communication by sending commands in different sequences.
- Interrupting message from system or from product

576 Attacks that make use of undocumented commands and editing commands are closely related, but distinctive attacks. Finding undocumented or undefined commands is a straightforward brute-force type of attack, where the attacker runs the ISO defined set of commands to see if the card replies to one or more commands that it should not answer

to.

577 However, if source code is not available, simple command search of valid CLA/INS pair is not sufficient, as especially in the context of identification of existing commands: sometimes all CLA/INS/P1/P2/Lc have to be correct. So it represents 2^{40} or 1 099 511 627 776 different possibilities (see ISO/IEC 7816-4 standard).

578 Though an undocumented command search can be highly standardized and automated, the identification could be brief or very costly in terms of time, or even too costly to be considered as practical. Once all variations of parameters are tried and the answers are recorded, the attacker analyses if there is any interesting attack mount point. Once an interesting answer has been determined the attacker builds a script to discover the behaviour of the identified command and exploit a potential vulnerability. This could also be done by source code checking. Note that finding a single command may not be sufficient, as the attacker may have to look for a specific sequence of commands, sometimes following a proprietary protocol.

579 Whether the undocumented command may present attack points depends on the quality of the software (the separation of execution domains) and the type of command that is discovered.

2.8.3.2.3 *Editing commands*

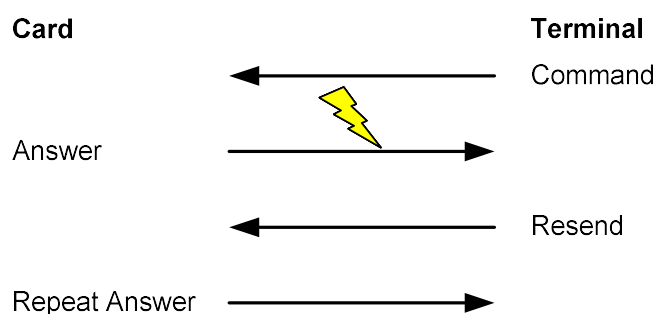
580 Editing commands is an attack step where the attacker tries to modify commands during the communication sequence to see if the card gives an unexpected reply (these commands may be in an interface specification, or they may have been discovered by observing message sequences or a command search as described above). Such editing commands are usually performed at the communication protocol level. These attack steps may enable vulnerabilities to be discovered and exploited (e.g. editing previously observed messages to supply a parameter that is too long may enable a buffer overflow attack). They may also expose timing differences that assist in reverse engineering of the software.

581 According to the security mechanisms associated with the API and the type of message, it may be easy or complex to forge a message (mutual authentication, secure channel, MAC, encryption, session key,...). However, as noted earlier, if an attack of this sort can be found then it will generally cause a TOE to fail evaluation.

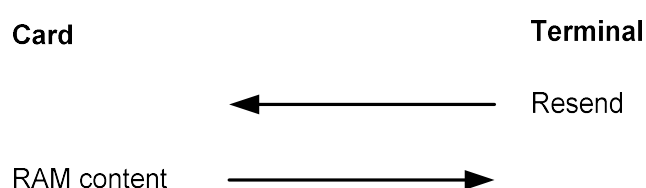
2.8.4 Direct protocol attacks

582 A typical protocol attack is to try to send commands that the smartcard does not expect in its current state. For example: the ISO 7186-3 and 14443 protocols for smartcards contain a command for handling failure in the communication. Instead of starting a genuine communication, by sending this command an attacker may receive an uninitialized buffer, or the last buffer that was written. This example is shown in the following pictures.

T=1 example valid behavior



T=1 example of security risk (inopportune behavior)

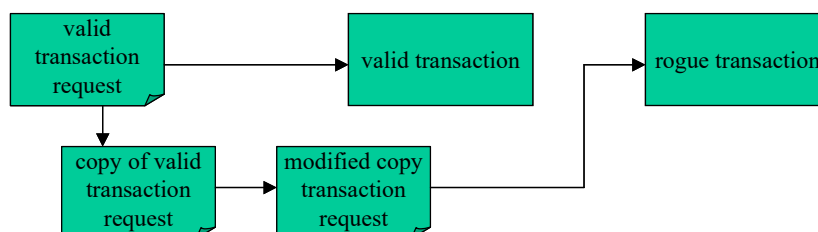


- 583 In this example, whether the TOE actually dumps the memory contents depends on the proper initialisation of I/O buffer pointer and length. The memory shown in the example might contain residual secret data, for example a recently calculated DES session key. Therefore this attack may allow an attacker to retrieve secret data from the TOE.
- 584 Under the category direct protocol attacks, there are also attacks focusing on the state machine of the TOE, where some sensitive operations need a specific order. Such order may ensure that the keys used in the cryptographic calculations are not exposed (such as challenge sending before a signature).

2.8.5 Man-in-the-middle and replay attacks

- 585 In this attack, the attacker hides in the communication path between two entities that are executing a valid communication. The attacker presents himself to either party as the other (valid) party. Some applications of man-in-the-middle attacks in public literature may be found in the following papers:
- An Example of a Man-in-the-middle Attack Against Server Authenticated SSL-sessions, Mattias Eriksson
 - Man-in-the-Middle in Tunnelled Authentication Protocols, N. Asokan, Valteri Niemi, Kaisa Nyberg, Nokia Research Center, Finland
 - Why Cryptosystems Fail, Ross Anderson
- 586 Man in the middle attacks are based on valid command interception either to perform replay attacks or to change some of the parameters to compromise the exchange of data (get access to confidential data exchanged, modify the parameters exchanged).
- 587 Replay attacks are possible when a mechanism does not check that a command is a genuine part of the current message sequence, or that a complete message sequence has not been used before (in general, a secure protocol should prevent this sort of attack by

design²⁷). An attacker uses a protocol analyser to monitor and copy packets as they flow between smartcard and reader or host. The packets are captured, filtered and analysed for interesting information like digital signatures and authentication codes. Once these packets have been extracted, the packets are sent again (replayed), thus giving the attacker the possibility to get unauthorized access to resources.



588 The picture shows a situation where the attacker copies a valid transaction request, modifies it and sends a second request using the same (or slightly modified) versions of the messages. In general this type of attack might allow the attacker to get unauthorised access to a user's assets, for example a bank withdrawal or access to protected system resources.

589 The attack may be a full attack path, such as if a bank account withdrawal succeeds. In the case where system resources are accessed, it might be a partial attack path, depending on the nature of the resources that are accessed (e.g. as a result of the attack the attacker may be able to communicate as an ordinary user and may then try to gain privileged status).

590 The replay attack might be countered by using sequence numbers with appropriate integrity protection, making the use of recorded valid messages much harder.

2.8.6 Buffer overflow or stack overflow

591 This attack is applicable to open platforms.

592 Open platforms are defined in this document as smart card operating systems with the capability of running and downloading multiple applications.

593 Open platforms provide a set of services to applications, in particular services to protect their sensitive data against external applications (unauthorized access and unexpected modification).

594 This attack could be performed through buffer overflow or stack overflow, produced by the execution of a malicious application.

595 Overflow, when not checked by the platform, can have various effects, such as overwriting existing content in the current stack.

596 The expected effect by the attacker here is that the malicious application modifies the current execution context and obtains system privileges. For example, the execution rights of the current context is written in the stack; if the attacker can overwrite these rights and put the administrator rights, all operations performed after this operation will have the administrator rights. Other example, the attacker may overwrite a memory location that contains a pointer in memory. The attacker may then control where an application is getting its data.

597 Gaining such privileges allows this application to execute virtually every operation and then disclose or modify secret data, e.g. modifying or disclosing the PIN of another

²⁷ Even where a protocol is designed to be secure, it may be possible to use a replay attack if a further attack step (such as a perturbation) is used to avoid a check that would otherwise detect and reject the replayed commands.

application.

- 598 Another effect is that internal data or data that were not presumed to be returned by a command are retrieved.

2.8.7 Malicious code execution

- 599 This attack consists of executing ill-formed applications, *i.e.* malicious applications that are made of illegal sequences of code (resp. bytecode) instructions or that do not have valid code (resp. bytecode) parameters.

- 600 This attack is not only applicable to Java Cards since there may be equivalent attacks for other operating systems. If not combined with any other attack such as authentication bypass, this attack has to be applied to open platforms with known loading keys (these could be considered as a privileged attacker as defined in section). In addition, if the card includes an embedded byte-code verifier (few Java Cards), this verifier must be disabled. No other specific configuration is required.

- 601 Ill-formed applications execute a sequence of code (resp. bytecode) that violates the runtime environment rules. Ill-formed applications are usually created from standard applications, in which the code (resp. bytecode) is manually modified.

- 602 It means for Java Cards that such ill-formed applications cannot be the output of a normal CAP file generator. As a consequence, most Java Card platforms do not enforce the rules during the execution of applications.

- 603 Ill-formed applications could be a way to illegally access data or functions directly. But in most cases, the purpose of these ill-formed applications is to perform a kind of reverse engineering, in particular to understand the platform's internal architecture and algorithms.

- 604 In the most successful cases, the attacker can retrieve information (e.g. a dump of memory), execute functions that usually require specific privileges or even switch to a context giving full control over the platform (e.g., Java Card RE context).

- 605 This attack requires the capability to load the ill-formed applications: this attack must be combined with an attack able to bypass the loading authentication mechanism or requires a privileged attacker.

- 606 The attacker must be an expert in the smart card operating system (e.g. Java Card). Once the attacker is able to download applications, the attack is performed in two steps:

- 3) In a first step, the attacker runs a specialized test suite of ill-formed applications, containing a large combination of "bugs".
- 4) In a second step, according to the results returned, the attacker needs to tune this test suite with new developments to interpret the results. He will then build an application combining the vulnerabilities found and producing the desired effect.

2.8.8 Communication interface switching

- 607 This attack is applicable to dual interface cards. The purpose is to exploit the possibility to communicate with a TOE on two different interfaces.

- 608 For example, on a TOE inserted in a mobile phone with a NFC interface, the TOE can be accessed either by the NFC interface or by the applications downloaded in the phone (communication in contact mode). The attacker may wait for a valid mutual authentication between the terminal and the TOE, and then through the contact interface, the attacker could communicate with the TOE to take advantage that a secure

session is opened. This is a way for the attacker to bypass a state machine chaining and to get access to commands with a privileged access.

2.8.9 Firewall bypass

2.8.9.1 Description of Attack

- 609 This attack is applicable to multi-application platforms, usually Java Card platforms but not only. For example, some cards have both native and Java Card applications.
- 610 On multi-application cards, sensitive applications and non-sensitive applications coexist on the same cards: for example, loyalty, e-couponing may be contained in the card along with payments applications.
- 611 A firewall is generally present on the multi-application card to confine each applet and the data owned by that applet to its own execution domain. However, sharing of data and functions is usually a feature offered with a high functional interest, but such mechanism also opens a channel of communication with a sensitive application through a mean which is different from sending APDU commands. Such sharing is therefore a target for the attackers.
- 612 The purpose of the attack is to access through non-sensitive applications to data belonging to sensitive applications, to modify the behaviour of these sensitive applications.
- 613 This could be done either by a malicious application downloaded by the attacker (see for example “Malicious code execution”, Section 2.8.7), but also through a non-sensitive application which has no security in it and is exploited by the attacker aware of this.

2.8.9.2 Effect of attack

- 614 There are several forms of firewall bypass:
- Access to functions or methods belonging to a sensitive application, such as:
 - Executing them from a unauthorized application;
 - Modifying them from a unauthorized application to change their behaviour (for example modifying a retrieving key method by making it copying the key value in a public buffer);
 - Force context switches;
 - Access to data belonging to a sensitive application, such as:
 - Unexpected sharing of data
 - Residual data exploitation (for example data in APDU buffer coming from another application)
 - Usage of shared resources to influence the behaviour of the sensitive application, such as:
 - Shared functions or methods having an output used by both sensitive and non-sensitive application;
 - Shared buffer which content may have an impact on the calculations performed within the sensitive application

2.8.9.3 Impact on TOE

- 615 This attack is of a logical nature. However, it is aimed at bypassing a mechanism designed to provide the primary protection. This means that finding an actual

vulnerability and exploiting it is the work of experts.

2.8.9.4 Characteristics of the Attack

- 616 The attacks are logical; the test environment consists of a smart card reader connected to a PC. The PC runs communication software and several dedicated tests scripts including dedicated applications for attacks. We can say that the tools used for this attack are standard but the tests scripts or applications that have to be developed are dedicated to these kind of attacks and could be considered as specialized.
- 617 The attacker must be an expert in software, and more precisely Java Card when the TOE is a Java Card, and have excellent programming skills. We therefore rate the expertise as 'Expert'. Because the attacker will need to bypass two defence mechanisms, this will take a significant amount of time. The knowledge about the card management system and the firewall mechanism may be public, but the knowledge about the applet is not (rating: 'sensitive'). The attack is of a logical nature; therefore the number of TOEs used is less than ten.

2.8.9.5 Countermeasures

- 618 The separation of execution domains for the applets is a countermeasure in itself. Careful programming of the mechanism must ensure that it secures itself and its own execution domain.

2.8.9.6 Example of rating

- 619 In this example, we take the assumption that the attacker has the possibility to download code on the TOE.
- 620 The attacker succeeds in finding a flaw in the firewall: he can create an application allowing reading a certain area of memory through illegal jump code, allowing to disclose an asset stored in memory.

Factor	Comment	Ident'n	Exploit'n (loading authorized)
Elapsed Time	The identification phase lasts more than 1 week, in order to run a test suite of software attacks, and then to tune each test to exploit an identified vulnerability. The exploitation is less than 1 day, as except a few parameters (references), every step of the attack is scripted.	< 1 month (2)	< 1 day (3)
Expertise	For Identification, the attacker must be an Expert in Java Card. For the usage of attack scripts, a proficient is enough.	Expert (5)	Proficient (2)
Knowledge of TOE	The knowledge of the applet to be attacked is sensitive, the rest is public.	Sensitive (4)	Public (0)
Access to TOE	The attack is of a logical nature; therefore the number of TOEs used is less than ten	< 10 Samples (0)	< 10 Samples (0)
Open Samples /	To set-up the attack, open samples are	Public	Public

Known Key	necessary (to download applications on the card). It depends on the card targeted, but some of them are publicly available and can be bought on the internet.	(0)	(0)
Equipment	The tests scripts or applications that have to be developed are dedicated to these kind of attacks, needs significant development and could be considered as specialized.	Specialized (3)	Specialized (4)
Sub Total		14	9
Total		23	

621 Note: In exploitation, we assume that the application loading does not require any additional attack, or any additional information. Otherwise, the rating have to be updated accordingly.

2.9 Application isolation

2.9.1 Overview of the section

- 622 The present section describes the attacks related to the application isolation. It starts with a precise explanation of the context of application isolation: the terminology, the assets, the threats and an overview of existing countermeasures.
- 623 The section continues with a description of the partial attacks specific to each countermeasure. These partial attacks are only one component of a full attack path used to compromise an asset. The section ends with examples of full attack paths in the context of application isolation.

The index table of the section is the following:

2.9	Application isolation	144
2.9.1	Overview of the section.....	144
2.9.2	References	144
2.9.3	Definitions	145
2.9.4	Introduction	145
2.9.5	Assets	146
2.9.6	Threats	147
2.9.7	List of relevant technologies and standards for application isolation	147
2.9.7.1	GlobalPlatform	147
2.9.7.2	Bytecode verifier offcard / oncard	149
2.9.7.3	Defensive virtual machine.....	150
2.9.7.4	Java Card Firewall.....	151
2.9.7.5	Multos.....	151
2.9.8	Rating of attacks defeating application isolation	153
2.9.9	Partial attack bricks applicable to the countermeasures.....	153
2.9.9.1	Partial attacks involving GlobalPlatform	153
2.9.9.2	Partial attacks on bytecode verifier	156
2.9.9.3	Partial attacks on Defensive virtual machine	162
2.9.9.4	Partial attacks on Java Card Firewall (with HW).....	166
2.9.9.5	Partial attacks on Java Card Firewall (SW based)	169
2.9.9.6	Partial attacks on Multos mechanisms	171
2.9.10	Full attack path	173
2.9.10.1	Attacks on memory management (getting a resource from another application)	173
2.9.10.2	Attacks on code execution (calling a code from another application) ...	178
2.9.10.3	Attacks on memory management with ill-formed applet.....	180

2.9.2 References

- [PPJCS] Sun Microsystems, Inc., Java Card System Open Configuration Protection Profile, April 2010, ANSSI-CC-PP-2010/03.

2.9.3 Definitions

Terminology	Definition
Application	Software running above an operating system.
Platform	IC and operating system. Can handle several applications
Application isolation	Unless explicitly authorized, the isolation ensures that application can't access data or methods from other applications.

2.9.4 Introduction

624 A multi-application platform describes a set of hardware and software built with the aim
to run more than one application at the same time.

625 While using more than one application in the same hardware, new items and a new
architecture concept have been introduced in the security problem definition:

- IC hardware: Electronic component designed to perform processing and/or memory function, based on a processing unit, security components, I/O ports and volatile and non-volatile memories.
- Dedicated Software, it is proprietary software embedded in a security IC (also known as IC firmware) and developed by the IC Developer. Such software is required for testing purposes (IC Dedicated Test Software) but may provide additional services to facilitate usage of the hardware and/or to provide additional services such as crypto libraries (IC Dedicated Support Software).
- Operating system: software in charge of performing the generic functions, routines and interpreters, related to the handling of hardware components and logical structures.
- Loaded-Application system Interface: It consists of the application loader, the virtual machine and the context manager.

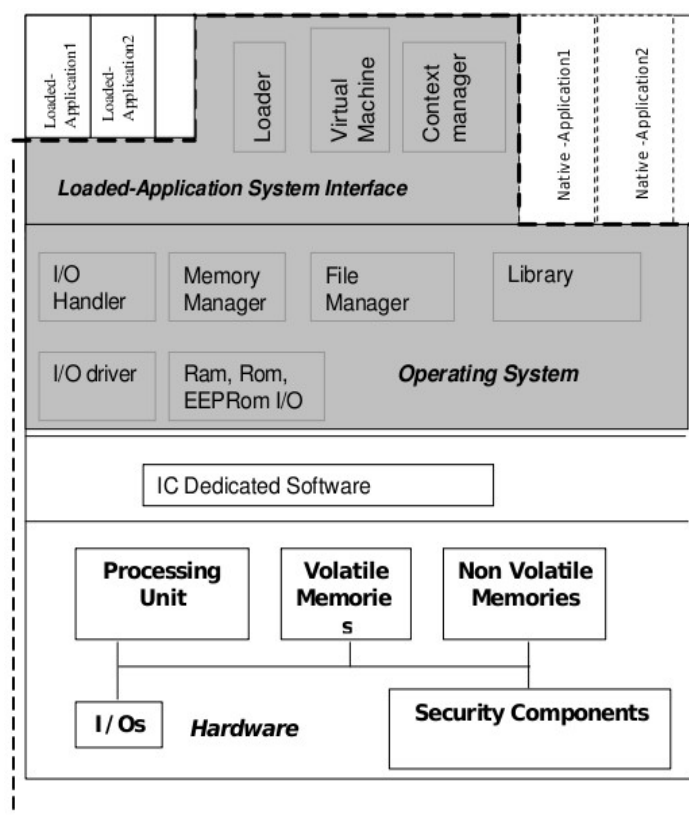


Figure 1 : Example of Multi-application platform

- 626 *Note: the layering presented in the previous diagram is an example. In some cases, the layering is not strict, for example native applications may access directly to the IC dedicated software, or the operating system can access directly to the hardware.*
- 627 The combination of the physical and logical measures of the multi-application platform allows the applet isolation, which might be defined as: all the security measures and mechanisms which protect the sensitive assets from the application and the platform against modification and/or disclosure.

2.9.5 Assets

- 628 The assets that may need to be protected in a multi-application environment are:
- Loaded application data (including keys). Application data must be protected according to:
 - The interfaces provided to access the data from outside the card.
 - The interfaces provided to access the data from within the card (interfaces shared with other applications, internal interfaces to be used by the same application package that implements it).
 - Loaded application code. The application code can neither be retrieved from outside the card, nor from another application.
 - Underlying platform data. The platform data must be protected according to:
 - The interfaces provided to access the data from outside the card (native interfaces for functionalities such as patching or native data storage operations).
 - The interfaces provided to access the data from loaded applets (native API

offered to applications).

629 This list is given as an indication and has to be detailed according to the TOE.

2.9.6 Threats

630 The multi-application platform is subject to the typical Smart Card attacks, such as:

- Physical attacks,
- Perturbation attacks,
- Side Channel attacks and,
- Software attacks, which may or may not be combined with the above-mentioned attacks, detailed as:
 - Unauthorized disclosure of Loaded Application data (such as application data, code and keys).
 - Unauthorized use of instructions, commands or sequence of commands.
 - Bypassing the Administrator restrictions by loading a malicious application on the multi-application platform.
 - Read confidential data or code belonging to another Loaded-Application without authorization by using a Loaded-Application.
 - Modify data or code belonging to another Loaded-Application without its authorization, by using a Loaded-Application.
 - Access the confidential data of system resources (like a system patch), by using the Loaded-Application.
 - Reverse-engineer the abstraction layer mechanisms by Using the Loaded-Application

631 The potential use of these techniques is very specific and special attention should be paid during the appraisal of coding assumptions.

2.9.7 List of relevant technologies and standards for application isolation

632 The present section details a set of countermeasures applicable to ensure the isolation of applications.

633 These countermeasures are usually specified in standards, and can be combined in smart card devices.

2.9.7.1 GlobalPlatform

2.9.7.1.1 Background

634 Smart cards are now divided between closed proprietary systems and true open systems on which various applications with different functions and potentially different providers may coexist. In this latter context, security is particularly important as the safe coexistence between applications must be guaranteed.

635 Java Card technology does address security issues concerned with applet isolation and inter applet communication. However Java Card provides such management on a low level, i.e. it states how memory objects are managed, but not how applications are managed.

636 Typically the JCRE specification does not give a model or a particular management of the applet or the card life cycles. Although the Java Card specification does provide a high level description of the main functional requirements about applet maintenance

(e.g. applet installation, deletion) it is mainly concerned with how such maintenance interacts with the core components of JCRE (the persistent memory, the firewall etc.). The JCRE specification actually states that such applet maintenance components are optional and the specific feature that those would have are not further detailed. Finally, although Java Card has been designed to be a multi-application framework it does not come with a concrete infrastructure to resolve problems arising from the secure coexistence of applications from different providers on the card.

637 With the fast technology development and the evolution of customer requirements, needs have arisen to reply to these questions. The role of the GlobalPlatform organization (GP for short) is to fill in this gap by addressing the following issues

- Standardization,
- Interoperability between applications,
- Trust management between the different smart card actors (card issuer and application providers),
- A general framework for card content management both off and on card during the whole lifecycle of the card,
- Secure communication.

2.9.7.1.2 Principle

638 The GP standard comes with the definition of a framework for application interoperability and management. This framework is specified by the GP specification and we can identify the main components as follows:

- Open GlobalPlatform Environment (OPEN)
OPEN is an additional layer to the JCRE which provides extra management functionalities to the card. If present on the card, the OPEN is responsible for command dispatching, (optional) multiple logical channel management, management of application and card lifecycle.

The OPEN provides also a concrete management for performing the installation and deletion of applications. For this, the GP specification defines the notion of a security domain.

- Security domain (SD)
A security domain represents a smart card actor on the card. Three main actors may be present on the card: an application provider (AP), a card issuer (CI) and a controlling authority (CA). SD is a special kind of a smart card application which provides common security services for applications which are associated to it e.g. various kinds of cryptographic services, secure messaging as well as application personalization. A SD stores cryptographic secrets of the actor which it represents. A GP card is always provided with a CI security domain. One of the benefits which SDs bring to security providers is that an AP may benefit of a certain independence with respect to the CI mainly for provisioning of security services (e.g. secure messaging, or application personalization) to its associated applications. A SD may be granted further privileges which would enable it to perform Card Content Management (application load, installation etc.).
- Cardholder verification methods
These are the common security services which the card provides to all applications. In particular, this gives the possibility for a unique user PIN number to be used by all applications.

2.9.7.1.3 Threats covered

639 GlobalPlatform participates to the authenticity, confidentiality and integrity of the applications loaded and installed.

2.9.7.2 Bytecode verifier offcard / oncard

2.9.7.2.1 History

640 All byte codes must be verified before their execution in order to avoid the execution of malformed applets. In the Java technology, the Byte Code Verifier is used to verify the class file on the Java Virtual Machine and is operating dynamically (ex: applied each time a class is loaded). However, the full Byte Code Verifier is often not implemented in a Java Card due to its limitation in processing power and memory size. There are several solutions to resolve this issue:

- The application can be verified off-card by an Off-Card Verifier which is not limited by Java Card constraints.
- The application can be verified on-card with a specific On-Card Verifier designed for Java Card.

2.9.7.2.2 Principle

641 These two kinds of byte code verifiers have a different behaviour:

- The On-Card Verifier performs its checks during the applet installation phase (link operation), at which point it can perform a structural verification of the CAP file and type verification. Note that dynamic verification performed on card is covered in section 2.9.7.3 "Defensive virtual machine".
- The Off-Card Verifier induces different organizational issues, as there is a need to guarantee that the application loaded on the card was actually checked by the off-card verifier. The off-card verifier avoids the constraints of having an on-card verifier implementation on cards. Off card verifier performs a structural analysis of the CAP file and a type verification (it simulates the execution of byte code).

642 A structural verification of the CAP file includes for example:

- CAP file component validity
 - Inside a component: conform to the Java Card Virtual Machine specification
 - Between components: for instance:
 - Between Descriptor component and Method component in order to compare method return type and last method's instruction
 - Between the Method component and the Constant Pool Component to detect invalid reference
 - Between the Method component and the Reference Location to detect address forging
- Binary compatibility between CAP file information and previous file loaded on the card
- Validity of export files used to generate the CAP file
- Verification of Java Card and Java constraints (for example, interface's methods are "abstract", final method are not overridden)

643 Type verification includes for example:

- No invalid usage of instructions: instruction operands must be conformed to instruction semantic.
- No Stack Overflow
- No Stack Underflow
- No Invalid Jump

2.9.7.2.3 Threats covered

644 The Byte Code Verifier detects malicious application. For instance, the following threats are covered by this mechanism, for a detailed description see [PPJCS]:

- CONFIDENTIALITY
 - The attacker executes an application without authorization to disclose the Java Card System code.
 - The attacker executes an application without authorization to disclose data belonging to another application.
 - The attacker executes an application without authorization to disclose data belonging to the Java Card System.
- INTEGRITY
 - The attacker executes an application to alter (part of) its own or another application's code.
 - The attacker executes an application to alter (part of) the Java Card System code.
 - The attacker executes an application to alter (part of) another application's data.
 - The attacker executes an application to alter (part of) Java Card System or API data.
- UNAUTHORIZED EXECUTION
 - An applet performs an unauthorized execution of a method.
 - An applet performs an unauthorized execution of a method fragment or arbitrary data.

2.9.7.3 Defensive virtual machine**2.9.7.3.1 History**

645 There are two approaches in maintaining type safety within virtual machines.

646 Semi-defensive virtual machine: all byte codes are either verified before or during installation (off-card or on-card).

647 Defensive virtual machine: type safety is enforced at run-time because the virtual machine only references typed data.

2.9.7.3.2 Principle**2.9.7.3.2.1 Semi-defensive virtual machine**

648 The semi-defensive virtual machine prevents type confusion by disallowing certain byte code execution sequences. Both virtual machines with off-card and on-card byte code verifiers are considered semi-defensive virtual machines. For example, the following byte code sequence is considered illegal:

```
sload_1
areturn
```

649 A virtual method implementing the above byte codes accepts an argument of type „short“, and returns the same value type cast to „reference“ (generic type used for all objects), leading to type confusion. The off-card verifier delivered with the Java Card Kit rejects this byte code sequence.

2.9.7.3.2.2 Defensive virtual machine

650 The defensive virtual machine²⁸ can analyze the byte code dynamically during the APDU execution (ex: type verification and structural verification) and does not require off- or on-card byte code analysis to prevent type confusion. In the implementation of a defensive virtual machine, all data is stored typed. This means that data value of type „short“ does not only consist of 16 data bits, but also some meta data (or tag) specifying the data value is in fact of type „short“. The same tagging applies for all data values for all types. Given this mechanism of tagged data values, the defensive virtual machine detects type confusion at run time, and can then act accordingly.

651 In the example given in the previous section, a type confusion is detected at runtime by the defensive virtual machine when a data value, tagged as „short“ is attempted to return as type „reference“.

652 A defensive virtual machine is slow compared to a semi-defensive virtual machine, hence it is likely that an implementation of a virtual machine might borrow some defensive concepts, but will not be a fully defensive virtual machine.

2.9.7.3.3 Threats covered

653 The semi-defensive and defensive virtual machine ensures type safety of the virtual machine without making assumptions on the byte codes it executes.

2.9.7.4 Java Card Firewall

2.9.7.4.1 History

654 The Java Card Operating System is designed to run all applets in a single virtual machine. It does not have resources to provide a per-application virtual machine, which would provide an isolated runtime environment for each applet. The Java Card firewall is introduced to provide the sandbox environment for applets running in the same virtual machine.

2.9.7.4.2 Principle

655 The Java Card firewall limits access to object references by their context. Only objects created within the same context can be referenced. Access to resources outside the context of an object is possible through the Java Card Firewall by means of the Shareable Interface Object mechanism. Static members are excluded from firewall control and their accessibility does not depend on contexts.

2.9.7.4.3 Threats covered

656 Access or modification to public and protected members by applets not within the context of the owner applet.

2.9.7.5 Multos

2.9.7.5.1 History

657 MULTOS platform provides a secure environment for application execution and data storage. It is a multi-applications operating system enforcing applications segregation. MULTOS applications can be develop in C language, in MULTOS Assembler (MEL) or in Java.

658 MULTOS does not have a verifier tool for MEL code because this language is less complex. However, MULTOS has similar security mechanisms such as firewall and secure application loading. In fact, all firewall checks are performed at run time by the

²⁸ There is no longer any definition of the defensive virtual machine in the version 2.6 of the Java Card system protection profile.

on-card virtual machine that is why the Off-Card byte code verification is useless.

2.9.7.5.2 Principle

659 MULTOS implements the following countermeasures:

1. Instructions, primitives and APDU commands do not allow addresses manipulation. In fact we cannot assign a new address to a variable contrary to Java Card (for instance: `aload_1 astore_3`)
2. The Firewall: applet isolation, code space and data space isolation (for instance, we cannot perform a jump from code to data). That is why an application cannot access to another application space and so cannot be accessed by other applications.
3. The Application loaded on card can contain:
 - MEL instructions
 - Data
 - DIR record: information about the name of the application when loaded on the card
 - FCI record: information that is returned when a MEL application is selected
 - Application signature (if exist)
 - KTU (if exist)
 - ...

It is not possible to manipulate components contrary to Java Card (for instance in order to forge an address by deleting an element in the Reference location).

4. The MULTOS Application Abstract Machine provides each application with its own memory space. In fact, the memory space is always relative to the current running application. Tagged addresses are used instead of absolute addresses. This tagged address consists of:
 - A register: ST and SB for static memory, DT, DB and LB for dynamic memory, PB and PT for public memory
 - An offset

A different instruction will be generated depending on register used. For instance:

- Instruction “LOAD SB[1], 0x10” will be “39 10 00 01”.
- Instruction “LOAD PB[1], 0x10” will be “3E 10 00 01”.

5. The Loaded application can be encrypted

2.9.7.5.3 Threats covered

660 MULTOS covers the following threats:

- CONFIDENTIALITY
 - The attacker tries to read data/code of the MULTOS system.
 - The attacker tries to read data/code of a codelet.
 - The attacker tries to read data/code of another application.
 - The attacker tries to reverse the application source code before/during the application loading process.
- INTEGRITY
 - The attacker tries to modify the code/data of the MULTOS system.
 - The attacker tries to modify the code/data of another application.
 - The attacker tries to modify the code/data of a codelet.
 - The attacker tries to modify the code of the application before/during the application loading process
- UNAUTHORIZED EXECUTION
 - The attacker tries to perform an unauthorized execution of a method.
 - The attacker tries to perform an unauthorized execution of a method fragment or arbitrary data.

2.9.8 Rating of attacks defeating application isolation

- 661 As presented in previous sections, security functions and countermeasures ensuring the isolation of application code and data may involve several technologies/standards and underlying mechanisms. To ease the work of laboratories during Common Criteria certification and to allow them to individually test the resistance of some mechanisms, it will be requested to developers to provide specific product configuration or to provide relevant key materials to proceed with the tests.
- 662 Such configuration is therefore considering that laboratories are using a “privileged attacker” profile for their tests.
- 663 When vulnerabilities are demonstrated in such test settings, laboratories will make a partial attack rating with the following constraints:
- The “privileged attacker” profile is explicitly detailed prior to the rating table
 - This profile is not reflected within the rating criteria (neither in “Knowledge of the TOE”, neither in “Open samples”).
- 664 More precisely for each partial attack, an example of rating is provided only to give an indication on the necessary efforts to perform the partial attack. The number of given points is not the number of points of a full attack path. A full attack path is constructed by joining all required partial attack paths. However, this does not mean that for rating the full attack path the points of all those partial attacks paths can simply be added. On the contrary, the rating of the full attack path is generally substantially less than the sum.
- 665 For “privileged attacker” profile, it has been decided by the group that giving “points” to such profile will be only mandated in case of full attack path rating.

2.9.9 Partial attack bricks applicable to the countermeasures

- 666 The partial attacks described in this section only target a specific technology/standard (see 2.9.7 "List of relevant technologies and standards for application isolation") and are therefore not sufficient to demonstrate exploitable vulnerabilities. A full attack path in the context of applet isolation needs to combine several partial attacks (see "Full attack path" Section 2.9.10).

2.9.9.1 Partial attacks involving GlobalPlatform

- 667 The aim of attacking GlobalPlatform is to allow an attacker to illegally load an application onto the TOE, *i.e.*, without knowing the loading keys values.

2.9.9.1.1 Attack the execution flow

2.9.9.1.1.1 Description of Attack

- 668 The attack is not performed on the cryptographic computations involved in the GlobalPlatform mutual authentication process and subsequent secure messaging commands.
- 669 The attack is performed on the code execution of the security domain with content management privilege. The attack exploits here a potential vulnerability in the robustness of the code execution flow against perturbation attacks. The idea is here to force the execution of any content management APDU command (`INSTALL [for load]`, `LOAD`, etc) whereas no secure channel has been opened. If the implementation is basic, this may consist in a simple verification such as:

```

    if (securityLevel == SecureChannel.NO_SECURITY_LEVEL)
        ISOException.throwIt(0x6985);

```

670 Then, the attacker simply needs to send a content management command (without previous INITIALIZE UPDATE and EXTERNAL AUTHENTICATE command) and if it specifies a 0x80 CLA byte, no secure channel unwrapping will be processed: therefore no cryptographic computations have to be attacked.

671 For a successful applet loading and installation, two distinct sequences are required:

- A sequence for code loading, consisting in an INSTALL [for load] and one or several LOAD commands;
- A sequence for applet instantiation, consisting in an INSTALL [for install & make selectable] command;

672 In the first sequence, the attacker shall be able to reproduce the attack successfully on all the commands within the sequence as the loading operation is atomic (at least 2^{29}). If the attack fails (which generally implies a power off), the sequence shall restart from the initial INSTALL [for load] command.

2.9.9.1.1.2 Effect of attack

673 The attack has several advantages. First, since it does not attack cryptographic operations (MAC computations) as well as GlobalPlatform key set derivation (EXTERNAL AUTHENTICATE); it is less likely that velocity counters will be incremented, allowing the attacker to repeat the attack as many times as necessary until it succeeds.

674 Finally, the partial attack could be extended to any other similar verification of the platform or of applications.

2.9.9.1.1.3 Impact on TOE

675 The direct impact is that the TOE may contain malicious code that could disclose or alter others applications.

2.9.9.1.1.4 Characteristics of the Attack

676 This is a direct attack (laser beam perturbation) on the GlobalPlatform security domain execution flow, aimed at bypassing security mechanisms.

677 The observation of the power consumption (SPA) or of electro-magnetic emanations (SEMA) is used to synchronize the attack. The identification of the test condition can be performed by comparing a loading with correct secure channel set-up and a loading with missing secure channel. Advanced synchronization may be required to reproduce automatically the attack, in particular if there is a lot of noise or jitter.

2.9.9.1.1.5 Countermeasures

678 The direct countermeasure relies on the security domain execution flow protection (by software). It generally consists in redundant verifications (double checks, sequencers, etc).

679 For instance, the security domain could not only ensure security level is appropriate according to the card state (in SECURED state at least C_MAC is required) during the secure channel establishment (*i.e.*, EXTERNAL AUTHENTICATE) but it could redundantly verify the security level in commands (e.g., LOAD).

2.9.9.1.1.6 Example of rating

680 The rating described here has been demonstrated on a card with quite low security protections and with a basic implementation as described in “Description of Attack”, Section 2.9.9.1.1.1.

681 The list of assumptions is the following:

²⁹ An optimized malicious applet that is able to execute code loaded into the heap (e.g., in arrays content) can fit in a single LOAD APDU command. Otherwise, an average of about two or three LOAD commands is to be considered.

Attacker profile	
GlobalPlatform version of Card Manager	2.x
GlobalPlatform life cycle (ex: OP_READY, SECURED, etc.)	OP_READY or INITIALIZED for identification SECURED for exploitation
Configuration of the Card Manager:	delegated management or option requiring controlling authority signature verification (DAP) are not present
Byte code verifier or not	Not applicable (attack concerns the loading capability only)
Knowledge of the keys (ex: Card Manager, command keys, SD keys, etc.)	The attack requires a valid trace of a correct loading in the card used in identification phase.
Possibility to sign/authenticate an applet (ex: to have a valid DAP, valid Token)	Not required
Presence of security countermeasures	
Software countermeasure	No software countermeasure at Card Manager level or Virtual Machine level against perturbation
	No velocity counter
Defensive virtual machine	Not applicable
Hardware countermeasure	Jitter and noise

- 682 The attack has consisted into loading a Java Card library (fits two LOAD commands) and an applet (fits in three LOAD commands).
- 683 The rating considers a success rate of about 20% per APDU command to successfully bypass the verification.
- 684 This rating considers a TOE without countermeasures against perturbation attacks. The evaluator should refer to "

Perturbation Attacks", see Section 2.3, to ensure a consistent rating when countermeasures are implemented in the TOE.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Less than one week for identification (includes applet generation, chip preparation, perturbation location and timing) Less than one week for exploitation	< 1 week (2)	< 1 week (4)
Expertise	Expert for identification Proficient for exploitation as it is automated.	Expert (5)	Proficient (2)
Knowledge of TOE	Public information	Public (0)	Public (0)
Access to TOE	Only few samples are required	< 10 samples (0)	<10 samples (0)
Open Samples / Known Key	No open samples	Public (0)	0
Equipment	Laser equipment and various tools	Specialized (3)	Specialized (4)
Sub Total		10	10
Total of partial attack		20	

2.9.9.2 Partial attacks on bytecode verifier

685 The Java Card Protection profile specifies that all byte code should be verified before its execution. In fact, if the platform has no On-Card verifier or is not a defensive platform, the application should be at least verified by an Off-Card Verifier. If this verification succeeds, the CAP file can be loaded without additional verification onto the card.

686 In this context, the TOE is the smart card because all the assets to protect are in it. There is no asset in the Off-Card verifier. In fact, this attack allows an attacker to ask for loading its malicious applet without being detected.

687 Basic type confusion attacks modify the reference of an object by the reference of another object. For instance, we can assign the address of a byte array to a short array in order to dump memory located after the byte array. The two following attacks are based on type confusing:

- The aim of the first attack is to create a type confusion not detected by an On-Card Verifier enabling us to dump and modify a part of the memory content.
- The second one uses a well-formed CAP file and abuses the transaction mechanism in order to create type confusion.

2.9.9.2.1 Type confusion not detected by an On-Card Verifier

2.9.9.2.1.1 Description of Attack

688 The aim of this attack is to create a type confusion not detected by an On-Card Verifier enabling us to dump and modify a part of the memory content.

689 The attacker will modify the CAP file in order to create a type confusion.

690 Several steps must be necessary for this attack. First, we need to characterize the On-Card Verifier, in order to understand its behavior and to analyze the checks performed by this tool. Second, we need to write the malicious applet by creating a type confusion not detected by the On-Card Verifier.

691 The main assumption of this attack is that the application could be loaded on card. If it is not the case, this attack will become a combined attack. In fact, the evaluator should

use the attack as described in the section 2.9.9.1 “Partial attacks involving GlobalPlatform”, in order to bypass the loading mechanism.

2.9.9.2.1.2 Effect of attack

692 This attack implements a “mechanism” allowing the application to bypass the check of the On-Card Verifier. In fact, the application contains, for instance, a type confusion between two arrays (a byte array and a short array) that will not be checked by the On-Card Verifier.

693 During the application execution, the short array will be located at the same address as the byte array.

2.9.9.2.1.3 Impact on TOE

694 This attack allows bypassing the On-Card Verifier in order to impose our malicious applet as a well-formed one.

695 The impact of the type confusion attack is dependent of software implementation. The main impacts are:

- Retrieve secret data (such as cryptographic keys).
- Read data/code outside our context.
- Modify data of another application.
- Modify the source code of another application.

2.9.9.2.1.4 Characteristics of the Attack

696 This attack requires performing a deep characterization of the On-Card Verifier, in order to isolate its potential vulnerabilities. In fact, depending on software implementation, some type confusion could not be exploitable.

2.9.9.2.1.5 Countermeasures

697 Several on-card countermeasures could detect our attack. In fact, the platform can implement some software countermeasures like dynamic type checks, HW design and so-on which could detect this type confusion attack.

2.9.9.2.1.6 Example of rating

698 This rating is based on the attack of the On-Card Verifier and of the smart card. The identification and exploitation parts take into account these two steps.

699 Moreover, this rating assumes that there is no specific security mechanism using over data (for instance, it is assumed that the design is not based on MMU and that data are not encrypted).

700 The On-Card Verifier used to check the applet contains vulnerabilities allowing performing a type confusion attack.

701 The main assumption of this attack is that the application could be loaded on card. If it is not the case, this attack will become a combined attack. In fact, the evaluator should use the attack as described in “Partial attacks involving GlobalPlatform”, Section 2.9.9.1, in order to bypass the loading mechanism.

Step 1: Characterization of the On-Card Verifier

702 Several applications need to be developed in order to determine:

- Order of the treatment applied to instructions.
- Checks applied on instructions which performed a read/write on a local variable.
- Checks applied on instructions which performed a read/write on global.
- Checks applied to method signature (parameter, return type).

703 Results obtain in these previous steps allow to isolate a vulnerability in the On-Card

Verifier.

Note that the use of the side-channel might help the attacker.

Step 2: Application development

- 704 Application will contain the type confusion attack between a byte array and a short
array, and the “mechanism” allowing bypassing the On-Card Verifier.
- 705 The identification of this vulnerability is included in the elapsed time of the
identification part.
- 706 Several APDU commands will be available in order to:
- Read the memory located after the byte array.
 - Modify the memory located after the byte array.
- 707 A memory dump should be performed by using the first APDU command. The memory
dump should be analyzed in order to retrieve secret data.
- 708 The list of assumptions for the rating is:

Attacker profile	
GlobalPlatform version of Card Manager	Not applicable
GlobalPlatform life cycle (ex: OP_READY, SECURED, etc.)	Not applicable
Configuration of the Card Manager:	Not applicable
Byte code verifier or not	On-Card Verifier
Knowledge of the keys (ex: Card Manager, command keys, SD keys, etc.)	The attack requires to be able to load applet onto the card
Possibility to sign/authenticate an applet (ex: to have a valid DAP, valid Token)	Not required
Presence of security countermeasures	
Software countermeasure	No software countermeasure at On-Card Verifier level or Virtual Machine level against perturbation
Defensive virtual machine	The virtual machine is unable to detect a type confusion
Hardware countermeasure	Jitter and noise

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Characterization of the On-Card Verifier, application development, potential on-card modifications in order to dump more memory. For exploitation, APDU commands must be send in order to dump memory. Moreover, result must be analyzed in order to identify secret data.	>1 month (5)	< 1 week (4)
Expertise	Expert is needed but no multi-expert. Exploiting the attack is at Proficient level since secrets are in clear in the dump (Expert if secret is encrypted).	Expert (5)	Proficient (2)
Knowledge of TOE	Information is in the public domain	Public (0)	Public (0)
Access to TOE	A few samples are sufficient to adjust the attack. For exploitation, the attack is performed on one sample.	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	The assumption is that an applet could be loaded, so, open samples are not used.	0	0
Equipment	For this attack, a PC and smart card reader are needed.	Standard (1)	Standard (2)
Sub Total		11	8
Total of partial attack		19	

2.9.9.2.2 Abusing the Transaction Mechanism

2.9.9.2.2.1 Description of Attack

709 The aim of this attack is to create a type confusion using a weakness implementation of the platform enabling us to dump and modify a part of the memory content. This type of attack uses a well-formed CAP file and abuses the transaction mechanism in order to create type confusion.

710 The main assumption of this attack is that the application could be loaded on card. If it is not the case, this attack will become a combined attack and may use the attack described in section 2.9.9.1 “Partial attacks involving GlobalPlatform” in order to bypass the loading mechanism.

2.9.9.2.2.2 Effect of attack

711 This attack exploits a vulnerability of the platform. The well-formed CAP file contains a transaction. During this transaction, the evaluator allocates a byte array *GlobalArrayB* using the instruction “new” and finally aborts this transaction:

```

short[] GlobalArrayS;
byte[] GlobalArrayB;
...
process()
{
...
    byte[] LocalArrayB = null;
    JCSystem.beginTransaction();
        GlobalArrayB = new byte[1];
        LocalArrayB = GlobalArrayB;
    JCSystem.abortTransaction();
    GlobalArrayS = new short[1];

    //The following condition is true
    if (GlobalArrayS==LocalArrayB)
...
}

```

712 Concerning the JCSys~~tem~~.abortTransaction call, the Java Card specifications states that a cleanup shall be done, but that muting is an acceptable alternative. Although many cards today would choose muting, this is not a Java Card requirement.

713 Some platforms have a vulnerable implementation of this transaction mechanism. In fact, if the evaluator allocates a short array (*GlobalArrayS*) just after the end of the transaction, it will be located at the same address as the byte array *LocalArrayB* and so *GlobalArrayB*.

2.9.9.2.2.3 Impact on TOE

714 This attack uses a well-formed application.

715 The impact of the type confusion attack is dependent of software implementation.

716 The main impacts are:

- Retrieve secret data (such as cryptographic keys).
- Read data/code outside our context.
- Modify data of another application.
- Modify the source code of another application.

2.9.9.2.2.4 Characteristics of the Attack

717 This attack exploits a vulnerability of the platform. Not all platforms will be vulnerable to this attack; some may implement a mechanism to detect it.

718 An exploitable type confusion must be found. In fact, depending on software implementation, some type confusion could not be exploitable. In this case, the

evaluator needs to modify the malicious application and to find an exploitable attack path.

2.9.9.2.2.5 Countermeasures

719 Several on-card countermeasures could detect the attack. In fact, the platform can implement some software countermeasures like byte code verification or other defensive mechanism (such as firewall), which could detect this type confusion attack.

720 The evaluator needs to find a way to bypass these countermeasures.

2.9.9.2.2.6 Example of rating

721 The identification and exploitation parts take into account these two steps.

722 The attack is performed on a card which has a vulnerability in the implementation of the transaction mechanism.

723 Moreover, this rating assumes that there is no specific security mechanism using over data (for instance, it is assumed that the design is not based on MMU and that data are not encrypted).

724 In some case, the developer only has the CAP file of the application of the third party. In order to ensure that this application is not a malicious one, it performs an Off-Card Verification. If this verification succeeds, it will load the application on card.

725 That's why, we will take the hypothesis, that the CAP file, evaluated as well-formed, is automatically loaded on card (the authentication keys must not be recovered).

726 If the applet could not be loaded, this attack becomes a combined attack. In fact, the evaluator should use the attack described in section 2.9.9.1 "Partial attacks involving GlobalPlatform" in order to bypass the loading mechanism.

Step 1: Application development

727 Application will contain the type confusion attack between a byte array and a short array, and exploit the transaction mechanism.

728 Several APDU commands will be available in order to:

- Read the memory located after the byte array.
- Modify the memory located after the byte array.

Step 2: Exploitation of type confusion

729 A memory dump should be performed by using the first APDU command. The memory dump should be analyzed in order to retrieve secret data.

730 If the memory dump cannot be exploitable, there could be two solutions:

- Perform some memory modification in order to dump more memory. In fact, the type confusion allowing to read and write into the memory. For instance, if the dump contains the header of an array, it will be possible to modify the size of this array directly in memory: since we are able to write into the memory the attacker will be able to change/write the new size. Then, by reading this array, we can dump more memory.
- Perform another type confusion (for instance between a class and an array).

731 The list of assumptions for the rating is:

Attacker profile	
GlobalPlatform version of Card Manager	Not applicable
GlobalPlatform life cycle (ex: OP_READY, SECURED, etc.)	Not applicable
Configuration of the Card Manager:	Not applicable
Byte code verifier or not	Not applicable
Knowledge of the keys (ex: Card Manager, command keys, SD keys, etc.)	Not applicable for exploitation since it is a valid applet
Possibility to sign/authenticate an applet (ex: to have a valid DAP, valid Token)	Not applicable for exploitation since it is a valid applet
Presence of security countermeasures	
Software countermeasure	No software countermeasure
Defensive virtual machine	The virtual machine is unable to detect such transaction mechanism abuse.
Hardware countermeasure	Not applicable

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Application development, potential attack modification in order to dump more memory. For exploitation, APDU commands must be send in order to dump memory. Moreover, result must be analyzed.	< 1 week (2)	< 1 week (4)
Expertise	Expert is needed but no multi-expert. Exploiting the attack is at Proficient level since secrets are in clear in the dump (Expert if secret is encrypted).	Expert (5)	Proficient (2)
Knowledge of TOE	Information is in the public domain.	Public (0)	Public (0)
Access to TOE	A few samples are sufficient to adjust the attack. For exploitation, the attack is performed on one sample.	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	The assumption is that an applet could be loaded, so, open samples are not used.	0	0
Equipment	For this attack, a PC and smart card reader are needed.	Standard (1)	Standard (2)
Sub Total		8	8
Total of partial attack		16	

2.9.9.3 Partial attacks on Defensive virtual machine

⁷³² The goal of an attack on a defensive virtual machine is to trick the virtual machine in allowing types to be confused. Such an attack may be possible when the defensive virtual machine is implemented only partially.

2.9.9.3.1 Type confusion attack with illegal byte code order

2.9.9.3.1.1 Description of Attack

733 An ill-formed applet containing byte codes in illegal order is loaded onto the target which then, when defensive checks are not present or incomplete, causes type confusion. This type confusion can then possibly be used to read persistent and transient data of the JCRE and other contexts not belonging to attacker's context.

734 A fully fledged type confusion attack uses the type confusion attack itself, the knowledge of the virtual machine meta data, and its application in a single attack applet able to read or write persistent and transient memory.

2.9.9.3.1.2 Effect of attack

735 The immediate effect of a type confusion attack is access to object meta data. Depending on the implementation of the virtual machine, this might also mean access to data outside the attacker's context, including the JCRE or native patches on the JCRE/JVM/underlying OS.

2.9.9.3.1.3 Impact on TOE

736 The attack is directed against other applications installed on the TOE, or the operating system. The main impacts are:

- Access to secret data
- Modification of functions and status

737 As the internal representation of data is not public the attacker should have critical knowledge of the TOE to interpret the retrieved data, or by experimental analysis on open samples derive the meaning of the data.

2.9.9.3.1.4 Characteristics of the Attack

738 The attack comprises three steps:

739 **Step 1: create illegal byte codes**

The Java Card Kit converter, which creates the CAP files to be loaded on the target, does not emit byte codes in illegal order. Therefore, the CAP file must be modified after being generated by the converter. An alternative format to the CAP format is the JCA, or Java Card Assembly format. This is the human readable equivalent of the CAP file. The JCA file can be modified at will, and converted into the CAP file format without limitation on byte code order, using the tools in Sun's Java Card Kit.

740 **Step 2: perform the type confusion**

The code snippet shown in 2.9.7.3 is the simplest example that, when it successfully runs on a target, shows that the virtual machine is actually not performing defensive checks at all.

Loading the applet requires knowledge of the card management keys, or a perturbation attack. Possibly when the target is implementing a fully defensive virtual machine, performing the type confusion is not possible using logical attacks alone, and a perturbation attack might be required. Depending on the implementation of the virtual machine, the perturbation attack must be performed once (to get the pointer), or many times (for each access to the pointer) during the attack. The latter attack is of course much more difficult, or even impractical.

741 **Step 3: use the type confusion**

The result of the type confusion is that object pointers can be created from meta data under the control of the attacker. For example, some virtual machines store arrays using the following meta data format:

<type of array> <length> <array data>

742 The length field of such an array can be set to an arbitrary large value, allowing access to data outside the attacker's context. Similarly, meta data formats for all interesting types (such as transient objects) can be mapped. The technique to realize these modifications may use one byte array and ill-formed methods (as in 2.9.7.3) to transform references into shorts and vice versa. First the byte array reference is turned into a short, and then the short is increased and turned back into a reference. The new reference points now to a byte array a little further in memory. By setting data bytes of the original byte array the meta data of the new byte array is modified. This way the new array can be assigned any type or given any length.

As the actual format of the meta data is implementation dependent the attacker may need to conduct multiple experiments to study the specific coding used in the implementation. This is part of the identification phase.

2.9.9.3.1.5 Countermeasures

743 The defensive virtual machine (when fully implemented) is a countermeasure against a type confusion attack using illegal byte codes. As already mentioned, a fully defensive virtual machine is rather slow and usually only some defensive concepts are used. Typically, cards will not disallow type confusion as such, but implement counter measures to prevent modification of byte codes by perturbation and limit the effect, such as ownership checks of E2P/RAM areas to prevent dumping/modification.

2.9.9.3.1.6 Example of rating

744 For the example rating we assume that open samples are used for identification and that perturbation is needed for exploitation to by-pass authentication, and/or integrity checks when loading an ill-formed applet. The attack requires therefore both perturbation and Java Card expertise.

745 This rating considers a TOE without countermeasures against perturbation attacks. The evaluator should refer to “

Perturbation Attacks”, see Section 2.3, to ensure a consistent rating when countermeasures are implemented in the TOE.

746 The list of assumptions for the rating is:

Attacker profile	
GlobalPlatform version of Card Manager	2.x
GlobalPlatform life cycle (ex: OP_READY, SECURED, etc.)	OP_READY or INITIALIZED for identification SECURED for exploitation
Configuration of the Card Manager:	No specific configuration needed
Byte code verifier or not	Not applicable (attack bypasses off-card verification)
Knowledge of the keys (ex: Card Manager, command keys, SD keys, etc.)	Only for identification.
Possibility to sign/authenticate an applet (ex: to have a valid DAP, valid Token)	Not required
Presence of security countermeasures	
Software countermeasure	The card management mechanism is not perturbation resistant
Defensive virtual machine	The virtual machine is unable to detect illegal byte code sequences
Hardware countermeasure	The chip is not perturbation resistant

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Tuning a perturbation attack to load an ill-formed applet, and interpret data retrieved after the attack.	< 1 Month (3)	< 1 week (4)
Expertise	Attacker must understand perturbation, be able to create an ill-formed applet and interpret the data retrieved from the TOE In this example, there are two distinct expertise required for identification: Java Card security of internal architectures and hardware attacks (perturbation). The rating is therefore multiple expert. The attack can be partly scripted, so proficient is sufficient for exploitation.	Multiple expert (7)	Proficient (2)
Knowledge of TOE	Only public knowledge required	Public (0)	Public (0)
Access to TOE	Several samples may be needed for identification due to the risk of card corruption. Repeating the attack may still require a few samples due to non-deterministic target behavior	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	No open samples	Public (0)	NA
Equipment	Laser perturbation equipment and basic Java Card tools	Specialized (3)	Specialized (4)

Sub Total		13	10
Total of partial attack		23	

2.9.9.4 Partial attacks on Java Card Firewall (with HW)

2.9.9.4.1 Description of Attack

747 Malicious applets in the Java Card environment could be used to challenge the restrictions imposed by the Java Card Firewall by attacking the context switching mechanisms. These malicious applets are well-formed and do pass byte-code verification. This attack may be easier to mount than ill-formed applet attacks as a malicious applet attack cannot be detected by byte code verification. On the other hand, this attack can only succeed if the firewall of the TOE is flawed.

2.9.9.4.2 Effect of attack

748 Various types of attacks can be performed on the Java Card Firewall depending on the intended effect and the resilience of the Firewall against the attack. The attacks have to be implemented in applets loaded on the card, but not necessarily in the same package. Malicious applets may try to delude the firewall by accessing data in unconventional ways or prevent a context switch. Example mechanisms are:

- Storage of global or temporary references in static fields for future access. Static fields are not controlled by the firewall, so if restricted data can be referenced via a static field this would create a loophole. This attack may disclose information belonging to an applet in a different context
- Object typecasting. A security breach would occur if the firewall agrees to typecast a restricted object to another object in a different context. This attack may disclose members that should not be accessible outside the context of the owner.
- Exception handling. By careful selection of input data an unexpected exception may be triggered during method execution. This attack may cause the Firewall not to switch the execution context when an exception is thrown and caught by an external caller.

2.9.9.4.3 Impact on TOE

749 The attack is directed against other applications installed on the TOE. The main impacts are:

- Access to secret data of the target applet,
- Modification of applet functions and status

750 The potential use of these techniques is specialized and has to be considered in the context of each evaluation. As the internal representation of data is not public the attacker should have critical knowledge of the TOE to interpret the retrieved data, or by experimental analysis on open samples derive the meaning of the data.

2.9.9.4.4 Characteristics of the Attack

751 This type of attack requires detailed information on the target applet and the Java Card platform hosting it. The information is needed to develop an applet that implements the attack. Consequently, the applet needs to be installed and executed on the host. If the detailed information is not available, it may be obtained by reverse engineering. The reverse engineering effort may be split in two stages:

- The first stage targets the host Java Card. The Java Card Firewall is tested in the attack areas described in section 2.9.7.4. This stage requires installation capabilities for the Java Card.
- The second stage targets the applet. It is aimed to find the interesting members to attack and tested for successful attacks. This stage requires access to the applet code.

752 While the development of the attack applet may occur on open samples, the exploitation phase requires installation of the attack applet on the operational product as we are considering here the case of a non-sensitive applet used to attack the target applet. A suggested method could be to incorporate the malicious applet in a 'Trojan horse', or use perturbation for loading the malicious applet.

753 The equipment needed to develop an attack applet consists of a development environment for Java Card applets. Tailored export files for the Java Card may be needed, which would be part of the information gathering phase on the target.

754 Note that vulnerability coming from malicious applets may be covered by organizational means, for example by ensuring that all applets downloaded on the platform are evaluated before to check if they exploit the identified vulnerability.

2.9.9.4.5 Countermeasures

755 A fully functional firewall will detect the access violations and prevent the attacks.

2.9.9.4.6 Example of rating

756 For this example we assume an attacker using perturbation to by-pass authentication for applet loading. The attack requires therefore both perturbation and Java Card expertise.

757 This rating considers a TOE without countermeasures against perturbation attacks. The evaluator should refer to "

Perturbation Attacks", see Section 2.3, to ensure a consistent rating when countermeasures are implemented in the TOE.

758 The list of assumptions for the rating is:

Attacker profile	
GlobalPlatform version of Card Manager	2.x
GlobalPlatform life cycle (ex: OP_READY, SECURED, etc.)	OP_READY or INITIALIZED for identification SECURED for exploitation
Configuration of the Card Manager:	No specific configuration needed
Byte code verifier or not	Not applicable (attack bypasses off-card verification)
Knowledge of the keys (ex: Card Manager, command keys, SD keys, etc.)	Only for identification.
Possibility to sign/authenticate an applet (ex: to have a valid DAP, valid Token)	Not required
Presence of security countermeasures	
Software countermeasure	The card management mechanism is not perturbation resistant
Defensive virtual machine	The firewall implemented in the runtime environment is unable to detect certain violations. The VM may be fully defensive.
Hardware countermeasure	The chip is not perturbation resistant

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Tuning a perturbation attack to load a malicious applet, and interpret data retrieved after the attack.	< one month (3)	< 1 week (4)
Expertise	Attacker must understand perturbation, be able to create a malicious applet and interpret the data retrieved from the TOE In this example, there are two distinct expertise required for identification: Java Card security of internal architectures and hardware attacks (perturbation). The rating is therefore multiple expert. The attack can be partly scripted, so proficient is sufficient for exploitation.	Multiple expert (7)	Proficient (2)
Knowledge of TOE	Knowledge of the targeted applet code is required for exploitation.	Sensitive (4)	Public (0)
Access to TOE	Several samples may be needed for identification due to the risk of card corruption. Repeating the attack may still require a few samples due to non-deterministic target behaviour	<10 samples (0)	<10 samples (0)
Open Samples / Known Key	No open samples	Public (0)	not rated
Equipment	Laser perturbation equipment and basic Java	Specialized	Specialized

	Card tools	(3)	(4)
Sub Total		17	10
Total of partial attack		27	

2.9.9.5 Partial attacks on Java Card Firewall (SW based)

2.9.9.5.1 Description of Attack

- 759 A firewall is generally present on the multi-application card to confine each applet and the data owned by that applet to its own execution domain. However, sharing of data and functions is usually a feature offered with a high functional interest, but such mechanism also opens a channel of communication with a sensitive application through a mean which is different from sending APDU commands. Such sharing is therefore a target for the attackers.
- 760 The purpose of the attack is to access through non-sensitive applications to data belonging to sensitive applications, to modify the behaviour of these sensitive applications.
- 761 This could be done either by a malicious application downloaded by the attacker (see for example "Malicious code execution", section 2.8.7), but also through a non-sensitive application which has no security in it and is exploited by the attacker aware of this.

2.9.9.5.2 Effect of attack

- 762 There are several forms of firewall bypass:
- Access to functions or methods belonging to a sensitive application, such as:
 - Executing them from a unauthorized application;
 - Modifying them from a unauthorized application to change their behaviour (for example modifying a retrieving key method by making it copying the key value in a public buffer);
 - Force context switches;
 - Access to data belonging to a sensitive application, such as:
 - Unexpected sharing of data
 - Residual data exploitation (for example data in APDU buffer coming from another application)
 - Usage of shared resources to influence the behaviour of the sensitive application, such as:
 - Shared functions or methods having an output used by both sensitive and non-sensitive application;
 - Shared buffer which content may have an impact on the calculations performed within the sensitive application

2.9.9.5.3 Impact on TOE

- 763 This attack is of a logical nature. However, it is aimed at bypassing a mechanism designed to provide the primary protection. This means that finding an actual vulnerability and exploiting it is the work of experts.

2.9.9.5.4 Characteristics of the Attack

- 764 The attacks are logical; the test environment consists of a smart card reader connected to a PC. The PC runs communication software and several dedicated tests scripts including dedicated applications for attacks. We can say that the tools used for this

attack are standard but the tests scripts or applications that have to be developed are dedicated to these kinds of attacks and could be considered as specialized.

- 765 The attacker must be an expert in software, and more precisely Java Card when the TOE is a Java Card, and have excellent programming skills. We therefore rate the expertise as 'Expert'. Because the attacker will need to bypass two defence mechanisms, this will take a significant amount of time. The knowledge about the card management system and the firewall mechanism may be public, but the knowledge about the applet is not (rating: 'sensitive'). The attack is of a logical nature; therefore the number of TOEs used is less than ten.

2.9.9.5.5 Countermeasures

- 766 The separation of execution domains for the applets is a countermeasure in itself. Careful programming of the mechanism must ensure that it secures itself and its own execution domain.

2.9.9.5.6 Example of rating

- 767 For this example we assume a privileged attacker that is able to download code on the TOE. The attack requires therefore only Java Card expertise.
- 768 The attacker succeeds in finding a flaw in the firewall: he can create an application allowing to read a certain area of memory through illegal jump code, allowing to disclose an asset stored in memory.
- 769 The list of assumptions for the rating is:

Attacker profile	
GlobalPlatform version of Card Manager	2.x
GlobalPlatform life cycle (ex: OP_READY, SECURED, etc.)	OP_READY or INITIALIZED for identification SECURED for exploitation
Configuration of the Card Manager:	No specific configuration needed
Byte code verifier or not	Not applicable (attack bypasses off-card verification)
Knowledge of the keys (ex: Card Manager, command keys, SD keys, etc.)	SD keys are known (privileged attacker)
Possibility to sign/authenticate an applet (ex: to have a valid DAP, valid Token)	Not required
Presence of security countermeasures	
Software countermeasure	None
Defensive virtual machine	The firewall implemented in the runtime environment is unable to detect certain violations. The VM may be fully defensive.
Hardware countermeasure	None

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	The identification phase lasts more than 1 week, in order to run a test suite of software attacks, and then to tune each test to exploit an identified vulnerability. The exploitation is less than 1 day, as except a few parameters (references), every step of the attack is scripted.	< 1 month (3)	< 1 day (3)

Expertise	For Identification, the attacker must be an Expert in Java Card. For the usage of attack scripts, a proficient is enough.	Expert (5)	Proficient (2)
Knowledge of TOE	The knowledge of the applet to be attacked is sensitive, the rest is public.	Sensitive (4)	Public (0)
Access to TOE	The attack is of a logical nature; therefore the number of TOEs used is less than ten	< 10 Samples (0)	< 10 Samples (0)
Open Samples / Known Key	No open samples	Public (0)	Public (0)
Equipment	The tests scripts or applications that have to be developed are dedicated to these kind of attacks, needs significant development and could be considered as specialized.	Specialized (3)	Specialized (4)
Sub Total		15	9
Total of partial attack		24	

Note: In exploitation, we assume that the application loading does not require any additional attack, or any additional information. Otherwise, the rating has to be updated accordingly.

2.9.9.6 Partial attacks on Multos mechanisms

2.9.9.6.1 Attacking the Firewall

2.9.9.6.1.1 Description of Attack

770 This attack is a combined attack. Its aim is to attempt to read a block of data with an invalid size (a great one) and to perform a fault injection in order to bypass the firewall.

771 The firewall ensures that an application cannot access to another application space. If the following instruction is executed: "LOAD SB[1], 0xFF", the firewall will detect that the current application attempts to access to other application space and so will return an error. The evaluator needs to perform a fault injection in order to bypass this check and so succeeding to dump a part of memory.

2.9.9.6.1.2 Effect of attack

772 This attack is a combined attack. The application contains an instruction which attempt to read a block of data with an invalid block length. This attack will be normally detected by the firewall.

773 The fault injection will be performed during this block length verification performed by the firewall. To succeed, this attack must bypass the security countermeasures in order to avoid the firewall detection. The fault injection will perturb the IC behaviour in order, for instance, to skip the good instruction.

2.9.9.6.1.3 Impact on TOE

774 The main impacts of this attack are:

- Retrieve secret data (such as cryptographic keys).
- Read data/code outside our context.

2.9.9.6.1.4 Characteristics of the Attack

775 The platform must be sensitive to fault injection.

776 In first time, a side channel analysis must be performed in order to retrieve the time where the check of the length was made. This step achieved, the fault injection could be

performed.

2.9.9.6.1.5 Countermeasures

777 Some countermeasure can be implemented to detect this kind of attack. For instance, the platform can implement hardware countermeasures against perturbation attacks. Moreover, additional software countermeasure should be needed (for instance, redundancy checks).

2.9.9.6.1.6 Example of rating

778 We suppose that the implementation of the security countermeasure which detects our attack is sensitive to fault injection. Moreover, we suppose that the platform does not implement additional security countermeasures against this kind of attack.

779 This rating assumes that there is no specific security mechanism using over data (for instance, it is assumed that the design is not based on MMU and that data are not encrypted).

780 **Step 1: Application development**

Develop a MEL application containing the following instruction “LOAD SB[1], 0xFF”. The identification of this vulnerability is included in the elapsed time of the identification part.

781 **Step 2: Side Channel**

Perform the Side Channel analysis in order to retrieve the time where the firewall checks the block length.

782 **Step 3: Fault injection**

Perform the Fault Injection in order to bypass the check of the firewall and analyse result of memory dump to retrieve secret data.

783 This rating considers a TOE without countermeasures against perturbation attacks. The evaluator should refer to “

Perturbation Attacks”, see Section 2.3, to ensure a consistent rating when countermeasures are implemented in the TOE.

784 The list of assumptions for the rating is:

Attacker profile	
Knowledge of the keys (ex: Card Manager, command keys, SD keys, etc.)	The assumption is that an application could be loaded in MULTOS world
Possibility to sign/authenticate an applet (ex: to have a valid DAP, valid Token)	The assumption is that an application could be loaded in MULTOS world
Presence of security countermeasures	
Software countermeasure	The firewall mechanism is not perturbation resistant
Defensive virtual machine	Not applicable
Hardware countermeasure	The chip is not perturbation resistant

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Attack identification SEMA, reverse engineering, documentation analysis. For exploitation: attack (sample preparation, fault injection ...).	< 1 week (2)	< 1 week (4)
Expertise	Expert is needed but no multi-expert. Exploiting the attack is at Proficient level since secrets are in clear in the dump (Expert possible if secret is encrypted).	Expert (5)	Proficient (2)
Knowledge of TOE	The implementation representation (Design and Source Code) is needed to perform the perturbation attack (in order to know how to perform this attack and to identify the code which should be bypassed). For exploitation, no particular documentation is needed once the setup is in place.	Critical (6)	Public (0)
Access to TOE	A few samples are sufficient to adjust the attack. For exploitation, the attack is performed on one sample.	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	Open samples are not used.	0	0
Equipment	Oscilloscope, Laser, ...	3 (specialized)	4 (specialized)
Sub Total		16	10
Total of partial attack		26	

2.9.10 Full attack path

2.9.10.1 Attacks on memory management (getting a resource from another application)

785 This section describes an attack that combines the attack scenarios detailed in
Section 2.9.9 to illegally get resources (e.g., key values) across applet isolation.

2.9.10.1.1 Description of Attack

786 The attack is a close variation to the attacks described in the article "Combined Attacks
and Countermeasures", from E. Vétillard and A. Ferrari, Cardis 2010 (133-147).

787 This attack is the combination of:

- 1) Getting a memory dump to locate assets through "Attacks on the I/O routines".
The attacker is able through physical perturbation during bytes emission to
force the TOE outputting more bytes than expected. The memory dumps
obtained, for instance during the ATR or in public APDU commands returning
a significant number of bytes, may allow the attacker to identify assets of
others applications and their respective addresses in memory.
It shall be noticed that software attacks such as those described in "Partial
attacks on bytecode verifier" or "Partial attacks on Defensive virtual machine"
could be used to perform such memory dump instead.
- 2) Loading an applet through "Partial attacks involving GlobalPlatform".
The attacker is able through this attack to load a malicious application onto the
TOE.
- 3) Type confusion to manipulate the objects identified in step 2 through "Partial
attacks on bytecode verifier" or "Partial attacks on Defensive virtual machine".

The attacker is able in the malicious applet to illegally manipulate a memory
address of an object of another context. In this description, this is achieved
through type confusions attacks.

- 4) Attack on the firewall to execute the getKey method on the object through
"Java Card Firewall" or "Attacking the Firewall"
The attacker uses physical perturbations to bypass Java Card/Multos Firewall
restrictions while manipulating objects out of the legitimate bounds. On Java
Card platform, the malicious applet may illegally invoke the getKey method on
an address of an object of another context.

788 Steps 1 and 2 are used to calibrate the attack. Steps 3 and 4 are detailed here because in
the partial attacks described in the previous sections, we assume that a single malicious
applet can perform every operation whereas in more realistic examples, a malicious
applet can only handle its own objects. That's why here a perturbation is used to bypass
the firewall restriction.

2.9.10.1.2 Effect of attack

789 The attack combines software and hardware based attacks. The combination may vary
depending on platform implementation: on defensive virtual machine, some above listed
software only attacks may be replaced by software/hardware combined or hardware
only attacks (for instance, there are several ways to get a memory dump on TOE).

790 The effect is that TOE has additional attack path to protect in its implementation.

2.9.10.1.3 Impact on TOE

791 Any platform security mechanisms could be bypassed to disclose or alter secrets since
security routines (decrypt, update, etc) are forced to be legally exercised in a context
belonging to the attacked application.

2.9.10.1.4 Characteristics of the Attack

792 This attack path combines several expertises and techniques.

- Fault injection through laser beam, for bypassing Issuer Security Domain and Firewall conditions and for performing a memory dump.
- Type confusion attack to illegally load and manipulate an address of an object of another context.

2.9.10.1.5 Countermeasures

- 793 The implementer has first to prevent the full attack path, by preventing the loading of the malicious application, by ensuring no memory dumps (in particular across several contexts) cannot be achieved, by guaranteeing that type confusion attacks are ineffective or by forcing static verification of applets provisioned.
- 794 The defensiveness of the virtual machine could be increased, in particular:
- The Firewall design.
The memory architecture on the use of hardware based mechanisms (for example MMU).
 - Redundant dynamic checks.
Reintroduce some dynamic verifications (types, range of jump addresses, etc).

2.9.10.1.6 Example of rating

795 In this example, the assumptions are:

Attacker profile	
GlobalPlatform version of Card Manager	2.x
GlobalPlatform life cycle (ex: OP_READY, SECURED, etc.)	OP_READY or INITIALIZED for open samples OP_READY, INITIALIZED or SECURED for TOE in exploitation.
Configuration of the Card Manager:	delegated management or option requiring controlling authority signature verification (DAP) are not present
Byte code verifier or not	No
Knowledge of the keys (ex: Card Manager, command keys, SD keys, etc.)	None, apart from open samples (see rating).
Possibility to sign/authenticate an applet (ex: to have a valid DAP, valid Token)	Not required. The attack requires a valid trace of a correct loading in the card used in identification phase.
Presence of security countermeasures	
Software countermeasure	No countermeasure on the GlobalPlatform content management commands
	No countermeasure against memory dumps in the I/O routines (allow to get the full memory dump)
	No countermeasure in the firewall checks
Defensive virtual machine	No defensive virtual machine or existence of a vulnerability in the defensive Java Virtual Machine (for manipulation of a type confusion attack)
Hardware countermeasure	None

796 As additional assumptions, to simplify the analysis of the dump, we consider that for the attack identification, all cards have the same configuration (same applications loaded, same personalization).

797 The precise path considered for this attack derived from the steps described in Section 2.9.10.1.1 "Description of Attack" are:

- 1) Getting a full memory dump through perturbation during ATR (perturbation based).
- 2) Loading one applet using a GlobalPlatform attack (perturbation based). If attack attempts during this step destroy the tested card, the attacker restarts from the same step because the assumption is that every cards have the same memory configuration.
- 3) Type confusion attack (software attack based), using the loaded applet which is a malicious applet. This step cannot be destructive for the cards tested since the assumption is there is no defensive virtual machine or a vulnerability exists in it.
- 4) Firewall attack (perturbation based), still using the code executed by the loaded

applet but with a different APDU command as in step 3. If attack attempts during this step destroy the tested card, the attacker restarts from step 2, by reinstalling the same applet.

- 798 Warm or cold reset of the card between the steps has no impact.
- 799 Several weeks are required for tuning physical attacks. Several weeks are required for software attacks.
- 800 Multiple levels of expertise are required: in particular, a good level of expertise on platform architectures, memory organization and physical aspects.
- 801 Information is gathered by the attacker and more than ten samples are required due to the multiple steps requiring perturbations.
- 802 Finally, calibration of the attack (identification of type confusion attacks) may require access to open samples to load applets.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Due to the number of separate attack steps, each lasting about a week, more than a month for identification is required: time required for determination of component behavior under laser perturbations, application loading attack, memory dump, for type confusion attack identification, firewall attack. In exploitation, there are also several complex steps to perform in a row to succeed the attack. Each of these steps takes at least one or 2 days to be done, leading to more than a week to succeed the attack, but substantially less than one month – hence an intermediate rating is chosen.	> one month (5)	More than one week but substantially less than a month (5)
Expertise	In this example, there are two distinct expertise required for identification: Java Card security of internal architectures and hardware attacks (perturbation). The rating is therefore multiple experts. In exploitation, expertise required here because the full attack path contains a certain number of perturbations on one sample (so difficult for a proficient in hardware) and also requires the analysis of attack steps results that may depend from the memory configuration of each sample tested (so expert in Java Card architecture is required).	Multiple Expert (7)	Expert (4)
Knowledge of TOE	Information is gathered by the dump analysis and the Java Card specifications are public.	Public (0)	Public (0)
Access to TOE	In identification, there are two different steps requesting perturbations, so it is reasonable to consider that more than ten samples are necessary. Exploitation can be performed on less than ten	More than 10 samples, but less than 30 (1)	<10 samples (0)

	samples.		
Open Samples / Known Key	Open samples considered as restricted	Restricted (2)	0
Equipment	Oscilloscope, laser, ...	Specialized (3)	Specialized (4)
Sub Total		18	13
Total		31	

2.9.10.2 Attacks on code execution (calling a code from another application)

803 This section describes an attack similar to the previous one but here applied on a non-shared method of another applet.

2.9.10.2.1 Description of Attack

804 The same combination of attacks is required, with the following modifications:

- 1) Getting a memory dump to locate assets through "Attacks on the I/O routines". Compared to 1.9.1, the attacker should not only locate objects (and their respective references) but also reverse part of the code to identify private routines to be called (for instance to reset a security counter or to disable a security mechanism).
- 2) Loading an applet through "Partial attacks involving GlobalPlatform". Same.
- 3) Type confusion to manipulate the objects identified in step 2 through "Partial attacks on bytecode verifier" or "Partial attacks on Defensive virtual machine". Same.
- 4) Attack on the firewall to execute a private method on the object through "Java Card Firewall" or "Attacking the Firewall".
The attacker uses physical perturbations to bypass Java Card/Multos Firewall restrictions while manipulating objects out of the legitimate bounds. On Java Card platform, the malicious applet may illegally invoke an arbitrary method on an address of an object of another context.
However, several perturbations may be required to allow invoking a method on an object not owned by the current context through firewall restrictions as during a method execution, object not owned by the current context may be accessed several times, with each time a firewall check³⁰.

805 Steps 1 and 2 are used to calibrate the attack. Steps 3 and 4 are not recalled here because they are similar compared to those defined in Section 2.9.10.1. It shall be noticed that performing several perturbations is difficult, however still feasible as the firewall check operation can be identified through a synchronisation on the bytecode execution.

2.9.10.2.2 Effect of attack

806 Same as stated in Section 2.9.10.1.

2.9.10.2.3 Impact on TOE

807 A malicious application may access private routines allowing to reset counters or to deactivate security mechanisms.

³⁰ In Section **Erreur ! Source du renvoi introuvable.**, since getKey is implemented at platform level, there is a great chance that the code is in native language and therefore only a single firewall check should be performed.

2.9.10.2.4 Characteristics of the Attack

808 This attack path combines several expertises and techniques.

- Fault injection through laser beam, for bypassing Issuer Security Domain and Firewall conditions and for performing a memory dump.
- Type confusion attack to illegally load and manipulate an address of an object of another context.

2.9.10.2.5 Countermeasures

809 Same as defined in Section 2.9.10.1.

2.9.10.2.6 Example of rating

810 The rating in Section 2.9.10.1 is modified taking into account the additional characteristics of this attack.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Due to the number of separate attack steps each lasting about a week, more than a month for identification: time is required for determination of component behaviour under laser perturbations, application loading attack, type confusion attack identification, firewall attack with multiple perturbations. In exploitation, there are also several complex steps to perform in a row to succeed the attack. Even if the attack is more complex than in the previous section (more perturbations in last step here), each of these steps continue to take at least one or 2 days to be done, but not weeks, leading to more than a week to succeed the attack..	> one month (5)	More than one week but substantially less than one month (5)
Expertise	In this example, there are two distinct expertise required for identification: Java Card security of internal architectures and hardware attacks (perturbation). The rating is therefore multiple expert. In exploitation, expertis required here because the full attack path contains a certain number of perturbations on one sample (so difficult for a proficient in hardware) and also requires the analysis of attack steps results that may depend from the memory configuration of each sample tested (so expert in Java Card architecture is required).	Multiple Expert (7)	Expert (4)
Knowledge of TOE	Information is gathered by the dump analysis.	Public (0)	Public (0)
Access to TOE	In identification, there are two different steps requesting perturbations, even multiple perturbations, so more than ten samples are	More than 10 samples (1)	More than 10 samples (2)

	necessary, but less than 30. Even in exploitation, with two steps requesting multiple perturbations, more than ten samples will be necessary.		
Open Samples / Known Key	Open samples considered as restricted.	Restricted (2)	0
Equipment	Oscilloscope, laser, ...	Specialized (3)	Specialized (4)
Sub Total		18	15
Total		33	

2.9.10.3 Attacks on memory management with ill-formed applet

- 811 In this example, the attacker loads an ill-formed application on the Java Card platform. This application creates a table in memory and reads the table cells with a data type larger in memory than the table initially created. The result is a partial memory dump, where data after the actual table is read.
- 812 Then, by loading applications creating objects and by modifying the objects value several times, the attacker uses their Java Card expertise to discover through the memory dumps how data is managed, in particular how application data is stored (including integrity checking and encryption).
- 813 In this example, we suppose that the attacker is able to find which data encryption is used and where the encryption key is.
- 814 This attack is performed on samples with a known loading key. An application on real samples (with unknown keys) could be an exploitation after performing a memory dump by physical attacks, or might be carried out using the same technique after developing an appropriate ill-formed applet by using public development samples. The result obtained on these samples allows the attacker to decrypt dumped data and discover sensitive data.
- 815 It is possible to obtain such samples for Java Card platforms, in particular in the form of development kits.

2.9.10.3.1 Attack path

- 816 Here we consider three variations of the ill-formed application attack. In each variation, a different scenario is used to enable the modified application to be loaded:
- using a perturbation attack to overcome a check on whether application load is allowed. In this case we assume that the attacker uses a laser perturbation when the card checks whether an application should be loaded or not. The perturbation causes the application to be loaded even though the check should fail (e.g. because the attacker does not have valid keys for authentication);
 - using a software bypass technique in which the attacker edits a sequence of application load commands in order to substitute the modified application for the intended (authorised) application;
 - using access to the loading keys, gained by collusion with an authorised person in the issuing environment.
- 817 The combined attack therefore consists of creating the modified application, causing the application to be loaded by one of the three situations above, and then executing the modified application to obtain data outside the permitted memory boundaries.
- 818 The rating below is based on the first scenario.

819 The attack takes place in two main parts: the development of the modified application, and the creation of the fault to enable the application to be loaded. The execution of the attack and collection of the results is assumed to require very little time (of the order of minutes). The analysis of the data is then assumed to be based on Critical knowledge of the software in order to interpret the data and, for example, recognise a secret/private key contained in it.

820 In this case we take a set of attack parameters for the application modification to be as below (note that no final attack potential is valid yet, since this is not yet a complete attack – the sub-totals are included purely for interest).

2.9.10.3.2 Example rating without countermeasures

821 First step: developing an attack applet

Factors	Comments		
		Ident'n	Exploit'n
Elapsed Time	Running ill-formed applications, analyzing results and fine tuning the applications can take close to a month. Exploitation is fast, but results can depend on the memory content and configuration.	< 1 month (3)	< 1 week (4)
Expertise	In order to find vulnerabilities in the mechanism, an expert is needed. If well documented during the identification phase, the exploitation needs proficient expertise.	Expert (5)	Proficient (2)
Knowledge of TOE	Critical knowledge is required for ill-formed application results analysis.	Critical (6)	Public (0)
Access to TOE	As this is a logical attack, access to TOE will in practice always be in the order of less than 10 samples.	< 10 (0)	< 10 (0)
Open Samples / Known Key	No open samples	Public (0)	NA
Equipment	The equipment needed is standard: PC and communication software. The ill-formed applications test suite used in the identification phase is considered as specialized. The development of the ill-formed final application forms part of the Identification phase and these are not therefore considered as 'Equipment' in exploitation phase.	Specialized (3)	Standard (2)
Sub Total		17	8
Total of partial attack		25	

822 This step does not give a rating to the TOE since it only involves preparation for the full attack – it is not an attack in itself.

823 Step two: application loading on a target card (using perturbation to overcome loading

checks):

824 We assume that the perturbation is based on small modifications to the attack rating described in Section 2.3.6. The modifications assume that:

- The attack is based on attempting and observing a successful load of an unauthorised application – this makes no difference to the points in the rating of the attack potential.
- Because we assume that the attacker uses development versions of the Java Card in developing the attack, the knowledge of the TOE for Identification is public.

825 Again, the final attack potential is not given at this stage but sub-totals are included for interest.

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	We assume the de-processing and initial testing to identify the failure combined with demonstrating the vulnerability take less than 1 week. We assume Exploitation based on a description of the attack and the commands to use.	< 1 week (2)	< 1 day (3)
Expertise	The attacker can recognise the fault by simple observation of output, however the attack requires several perturbations. A Proficient rating is required for Exploitation, because of the de-processing techniques and equipment operation required.	Expert (5)	Proficient (2)
Knowledge of TOE	Both attack steps assume Public information for Identification and Exploitation.	Public (0)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 samples (0)	< 10 samples (0)
Open Samples / Known Key	No open samples	Public (0)	NA
Equipment	The dominant attack step is the perturbation part, requiring specialised equipment for both steps.	Specialized (3)	Specialized (4)
Sub Total		9	9
Total of partial attack		18	

826 If the ST identifies the unauthorized uploading of an application as a threat, then only this part of this attack needs to be considered for the rating.

827 Otherwise, we combine the two parts of the attack to give a single table for the full attack path:

Factor	Comment	Ident'n	Exploit'n
Elapsed Time	Both parts of the attack are done in sequence (not in parallel), so the times for the two attack steps are added ³¹ .	< 1 month (3)	< 1 week (4)
Expertise	The higher rating of 'Expert' is required for Identification of the application modification, and both steps require Proficient for Exploitation.	Expert (5)	Proficient (2)
Knowledge of TOE	The attack is based on using a Java Card with a development kit, but the application modification uses Critical knowledge to interpret the results of the application modification.	Critical (6)	Public (0)
Access to TOE	More than one sample may be necessary, but less than ten.	< 10 samples (0)	< 10 samples (0)
Open Sample / Known Key	Open samples considered as restricted.	Restricted (2)	NA
Equipment	Minimum requirement is equipment to de-process the chip, bond out the pads, and then to generate and analyse the required commands to drive the IC. A laser and optical microscope are required to generate the perturbations, and a digital oscilloscope is used to identify and repeat the attack timing. In the second step, an ill-formed applications test suite (specialized also) is used for the logical attack.	Specialized (3)	Specialized (4)
Sub Total		19	10
Total		29	

828 This TOE would be resistant for this attack to an attacker with moderate attack potential (VAN.4).

2.9.10.3.3 Required demonstration of attack to justify this rating

829 Demonstration of this attack requires both parts of the attack to be completed. The perturbation step needs to demonstrate that authentication can be bypassed in the way required to load an ill-formed applet. A demonstration applet then needs to be executed to show that the relevant behaviour (e.g. dumping memory content) can be carried out.

2.9.10.3.4 Countermeasures

830 Possible countermeasures to the software part of the attack could be:

- On-board bytecode verifier

³¹ It is noted that adding two values such as '< 1 week' together could give a result that is still '<1 week' or else a result at the next level (i.e. '<1 month' with the values mentioned). Similarly, adding '<1 week' and '<1 month' could give a result of '<1 month' or '> 1 month'. In this example, it is assumed that there is enough difficulty in each of the attack steps, with no realistic potential for reducing effort, to justify increasing the Exploitation points to the higher level, but that the Identification (by an Expert) for both steps could be achieved within the higher of the values for the individual steps.

- Defensive Java Virtual Machine (protection against stack overflow, inconsistency in variable type managed by instructions, etc.)
- 831 For the perturbation part of the attack, the countermeasures discussed in section 2.3 also apply.
- 832 A TOE with countermeasures is assumed to render the attack impossible (or at least not feasible) and hence no separate attack potential calculation is given.

3 Countermeasure list

3.1 Hardware

- 833 Environmental controlling countermeasures
- Low and high voltage sensors
 - High and low frequency sensors
 - Glitch Sensor
 - Input filters
 - Temperature sensor
 - Light Sensor
- 834 Security optimized layout and manufacture
- Randomised location of components
 - High density co-location
 - Non-standard dedicated Smart Card CPU
 - Active shield, or barrier shielding
 - Minimised feature size
 - Multi-layer interconnect
 - Wide Bus width
- 835 Secure Delivery Processes
- Unique chip identification number for each chip
 - Secure Test modes
- 836 Side channel resistance
- Current balancing
 - Current smoothing
 - Randomization
- 837 Randomised internal execution
- Clock
 - Code
- 838 Operational Integrity Checking
- Trap system for not permitted states
 - Hardware ECC for memories

- Hardware fault detection for memories (that are effective in detecting when code is not read correctly from memory, leading to, e.g., a reset, but not necessarily a non-functional TOE)
- Redundancy
- Hardware Memory Management and Protection Unit

839 Internal Encryption

- On-chip encryption of internally handled data
- Hardware encryption of memories

3.2 Software countermeasures

840 The following list identifies countermeasures implemented in software that help to prevent not only the pure software attacks, but also some of the hardware attacks identified elsewhere in other sections.

841 This list gives a guide of software aspects that should be considered during an evaluation.

842 Access Control

- Memory access control management
- Access conditions to APDU commands (Lifecycle, authentication first)
- Access conditions associated to files (authentication status)
- Flow control (MAC) to APDU command

843 Integrity

- Integrity of sensitive assets
 - Start-up integrity check (data)
 - Data and data structure (TLV) integrity check during usage
 - Data integrity check during creation, importation, update (counters, lifecycle states...)
 - Applet loading integrity
 - Software Integrity (patch table integrity, applet table integrity, applet integrity, patch integrity)
- File system integrity check during creation and erasure
- Stack integrity check
- VM firewall (stack overflow, Illegal instructions, unauthorized memory access...)
- Key generation and pair verification
- Transaction mechanism (including roll-back mechanism)
- Secure branching against execution perturbation and fault attacks
- Multiple check of sensitive data (e.g. lifecycle information)
- Secure messaging for integrity (MAC-ing)

- Continuous random testing
- Onboard bytecode verifier

844 Confidentiality

- Sensitive data creation, access and clearing
 - Transient data (arrays in RAM, clear on reset, clear on applet selection)
 - APDU buffer
 - Cryptographic buffer of the crypto processor, managed by the cryptographic library
 - Keys, PINs, session keys (renewal, clearing)
- Data manipulation hiding
- Constant time code execution
- Randomized algorithm execution
- Randomized data initialization and erasure
- Randomized data comparison
- Memory allocation quota management
- Secure channel for confidentiality (encryption + MAC+ counter)
- PIN retry counter check
- Data encryption, secure storage
- Noise during sensitive computation or comparison

845 Non repudiation

- Diversification by unique chip identification number
- Role authentication, binding between role and access condition
- User attribute verification and link to roles
- Event logging and audit

846 Availability

- Start-up (cryptographic services, sensor) operational service check
- Muting mechanism
- Blocking card mechanism
- Exception handling
- Irreversible card termination
- Session closing, session keys erasure
- Access rights management and clearing
- Key zeroisation

847 Non Replay

- Sequence numbers with integrity/authenticity protection
- Retry counters for encryption, signature, session keys, secure messaging, authentication, transactions.

848 The effect of these countermeasures will generally be to increase the ratings in the tables. In particular, the elapsed time for identification and exploitation may well be extended. The expertise required may increase, and if countermeasures are used which kill the device then a higher number of samples may be required.

4 Glossary

AES	Advanced Encryption Standard
AFM	Atomic Force Microscope
APDU	Application Protocol Data Unit
API	Application Programming Interface
CAP	Converted Applet
CB	Certification Body
CC	Common Criteria
CEM	Common Methodology for Information Technology Security Evaluation
CMOS	Complementary Metal Oxide Semiconductor
CPU	Central Processing Unit
CRT	Chinese Remainder Theorem
DEMA	Differential Electromagnetic Analysis
DES	Data Encryption Standard
DFA	Differential Fault Analysis
DPA	Differential Power Analysis
ECC	Elliptic Curve Cryptography
ECC	Error Correcting Code (eg. For memory)
EEPROM	Electrically Erasable Programmable ROM
EMA	Electromagnetic Analysis
EMV	Europay, Mastercard, VISA
ETR	Evaluation Test Report
FIB	Focused Ion Beam
FPGA	Field-Programmable Gate Array
FSP	Functional Specification
HRNG	Hybrid Random Number Generator
HW	Hardware
IC	Integrated Circuit
ISO	International Organization for Standardization
ITSEF	Information Technology Security Evaluation Facility
JCRE	Java Card Runtime Environment
JIL	Joint Interpretation Library
MAC	Message Authentication Code
NFC	Near Field Communication
OS	Operating System
PIN	Personal Identification Number
PK	Public Key
PRNG	Pseudo Random Number Generator
PTC	PIN Try Counter
RAM	Random Access Memory
RIE	Reactive Ion Etch
RNG	Random Number Generator
ROM	Read Only Memory
RSA	Rivest Shamir Adleman
S/N	Signal to Noise ratio
SEM	Scanning Electron Microscope
SEMA	Simple Electromagnetic Analysis

SFI	Single Fault Injection
SK	Secret Key
SPA	Simple Power Analysis
SW	Software
TLV	Type Length Value
TOE	Target of Evaluation
TRNG	True Random Number Generator
VM	Virtual Machine

5 References

[AIS20]	Functionality classes and evaluation methodology for deterministic random number generators, (BSI).
[AIS31]	Functionality classes and evaluation methodology for physical random number generator, Version 3, 15.05.2013 (BSI).
[COMPO]	Composite product evaluation for Smart Cards and similar devices, Version 1.5.1, May 2018, Joint Interpretation Library.
[ISO/IEC CD 18031]	ISO/IEC SC27 WG2 (Security Techniques and Mechanisms) Meeting Report. CD 18031 is about Random bit generation standard. This standard has been completed and was officially published 11 November 2005
[JIL-AP]	Application of Attack Potential to Smartcards, version 3.0, April 2019, Joint Interpretation Library.
[PP0084]	Security IC Platform Protection Profile with Augmentation Packages, BSI-CC-PP-0084-2014, Version 1.0, 13.01.2014.
[RankDist]	https://eprint.iacr.org/2016/491 , D. Martin et al, Characterisation and Estimation of the Key Rank Distribution in the Context of Side Channel Evaluations, Asiacrypt 2016
[Knapsack]	http://eprint.iacr.org/2015/689 - D. Martin et al. How to Enumerate Your Keys Accurately and Efficiently After a Side Channel Attack.
[SKEA]	http://eprint.iacr.org/2015/795 - A. Bogdanov et al. Fast and Memory-Efficient Key Recovery in Side-Channel Attacks.